

CHAPTER

9

TURING MACHINES

CHAPTER SUMMARY

Here we introduce the important concept of a *Turing machine*, a finite-state control unit to which is attached a one-dimensional, unbounded tape. Even though a Turing machine is still a very simple structure, it turns out to be very powerful and lets us solve many problems that cannot be solved with a pushdown automaton. This leads to *Turing's thesis*, which claims that Turing machines are the most general types of automata, in principle as powerful as any computer.

In our discussion so far we have encountered some fundamental ideas, in particular the concepts of regular and context-free languages and their association with finite automata and pushdown accepters. Our study has revealed that the regular languages form a proper subset of the context-free languages and, therefore, that pushdown automata are more powerful than finite automata. We also saw that context-free languages, while fundamental to the study of programming languages, are limited in scope. This was made clear in the last chapter, where our results showed that some simple languages, such as $\{a^n b^n c^n\}$ and $\{ww\}$, are not context-free. This prompts us to look beyond context-free languages and investigate how one might define new language families that include these examples. To do so, we return to the general picture of an automaton. If we compare finite automata with pushdown automata, we see that the nature of

the temporary storage creates the difference between them. If there is no storage, we have a finite automaton; if the storage is a stack, we have the more powerful pushdown automaton. Extrapolating from this observation, we can expect to discover even more powerful language families if we give the automaton more flexible storage. For example, what would happen if, in the general scheme of Figure 1.4, we used two stacks, three stacks, a queue, or some other storage device? Does each storage device define a new kind of automaton and through it a new language family? This approach raises a large number of questions, most of which turn out to be uninteresting. It is more instructive to ask a more ambitious question and consider how far the concept of an automaton can be pushed. What can we say about the most powerful of automata and the limits of computation? This leads to the fundamental concept of a **Turing machine** and, in turn, to a precise definition of the idea of a mechanical or algorithmic computation.

We begin our study with a formal definition of a Turing machine, then develop some feeling for what is involved by doing some simple programs. Next we argue that, while the mechanism of a Turing machine is quite rudimentary, the concept is broad enough to cover very complex processes. The discussion culminates in the **Turing thesis**, which maintains that any computational process, such as those carried out by present-day computers, can be done on a Turing machine.

9.1 THE STANDARD TURING MACHINE

Although we can envision a variety of automata with complex and sophisticated storage devices, a Turing machine's storage is actually quite simple. It can be visualized as a single, one-dimensional array of cells, each of which can hold a single symbol. This array extends indefinitely in both directions and is therefore capable of holding an unlimited amount of information. The information can be read and changed in any order. We will call such a storage device a **tape** because it is analogous to the magnetic tapes used in older computers.

Definition of a Turing Machine

A Turing machine is an automaton whose temporary storage is a tape. This tape is divided into cells, each of which is capable of holding one symbol. Associated with the tape is a **read-write head** that can travel right or left on the tape and that can read and write a single symbol on each move. To deviate slightly from the general scheme of Chapter 1, the automaton that we use as a Turing machine will have neither an input file nor any special output mechanism. Whatever input and output is necessary will be done on the machine's tape. We will see later that this modification of our general model in Section 1.2 is of little consequence. We could retain the input file

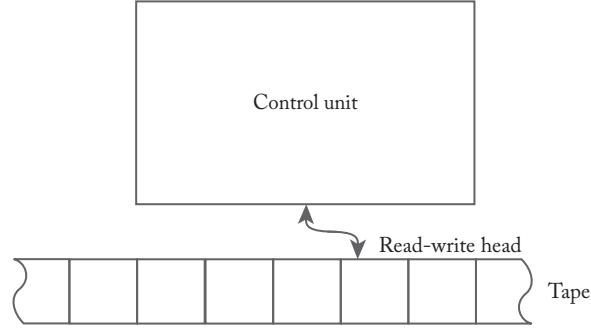


FIGURE 9.1

and a specific output mechanism without affecting any of the conclusions we are about to draw, but we leave them out because the resulting automaton is a little easier to describe.

A diagram giving an intuitive visualization of a Turing machine is shown in Figure 9.1. Definition 9.1 makes the notion precise.

DEFINITION 9.1

A Turing machine M is defined by

$$M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F),$$

where

Q is the set of internal states,

Σ is the input alphabet,

Γ is a finite set of symbols called the **tape alphabet**,

δ is the transition function,

$\square \in \Gamma$ is a special symbol called the **blank**,

$q_0 \in Q$ is the initial state,

$F \subseteq Q$ is the set of final states.

In the definition of a Turing machine, we assume that $\Sigma \subseteq \Gamma - \{\square\}$, that is, that the input alphabet is a subset of the tape alphabet, not including the blank. Blanks are ruled out as input for reasons that will become apparent shortly. The transition function δ is defined as

$$\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}.$$

In general, δ is a partial function on $Q \times \Gamma$; its interpretation gives the principle by which a Turing machine operates. The arguments of δ are the current state of the control unit and the current tape symbol being read. The result is a new state of the control unit, a new tape symbol, which replaces the old one, and a move symbol, L or R . The move symbol indicates whether the read-write head moves left or right one cell after the new symbol has been written on the tape.

EXAMPLE 9.1

Figure 9.2 shows the situation before and after the move

$$\delta(q_0, a) = (q_1, d, R).$$

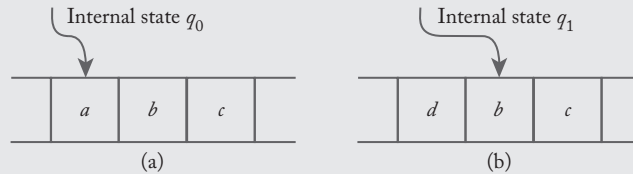


FIGURE 9.2 The situation (a) before the move and (b) after the move.

We can think of a Turing machine as a rather simple computer. It has a processing unit, which has a finite memory, and in its tape, it has a secondary storage of unlimited capacity. The instructions that such a computer can carry out are very limited: It can sense a symbol on its tape and use the result to decide what to do next. The only actions the machine can perform are to rewrite the current symbol, to change the state of the control, and to move the read-write head. This small instruction set may seem inadequate for doing complicated things, but this is not so. Turing machines are quite powerful in principle. The transition function δ defines how this computer acts, and we often call it the “program” of the machine.

As always, the automaton starts in the given initial state with some information on the tape. It then goes through a sequence of steps controlled by the transition function δ . During this process, the contents of any cell on the tape may be examined and changed many times. Eventually, the whole process may terminate, which we achieve in a Turing machine by putting it into a **halt state**. A Turing machine is said to halt whenever it reaches a configuration for which δ is not defined; this is possible because δ is a partial function. In fact, we will assume that no transitions are defined for any final state, so the Turing machine will halt whenever it enters a final state.

EXAMPLE 9.2

Consider the Turing machine defined by

$$\begin{aligned} Q &= \{q_0, q_1\}, \\ \Sigma &= \{a, b\}, \\ \Gamma &= \{a, b, \square\}, \\ F &= \{q_1\}, \end{aligned}$$

and

$$\begin{aligned} \delta(q_0, a) &= (q_0, b, R), \\ \delta(q_0, b) &= (q_0, b, R), \\ \delta(q_0, \square) &= (q_1, \square, L). \end{aligned}$$

If this Turing machine is started in state q_0 with the symbol a under the read-write head, the applicable transition rule is $\delta(q_0, a) = (q_0, b, R)$. Therefore, the read-write head will replace the a with a b , then move right on the tape. The machine will remain in state q_0 . Any subsequent a will also be replaced with a b , but b 's will not be modified. When the machine encounters the first blank, it will move left one cell, then halt in final state q_1 .

Figure 9.3 shows several stages of the process for a simple initial configuration.



FIGURE 9.3 A sequence of moves.

As before, we can use transition graphs to represent Turing machines. Now we label the edges of the graph with three items: the current tape symbol, the symbol that replaces it, and the direction in which the read-write head is to move. The Turing machine in Example 9.2 is represented by the transition graph in Figure 9.4.

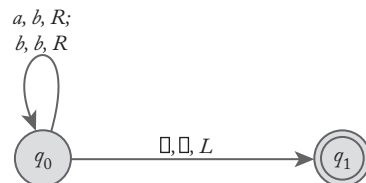
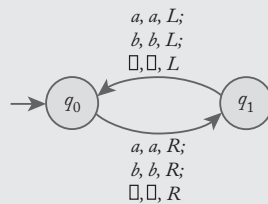


FIGURE 9.4

EXAMPLE 9.3

Look at the Turing machine in Figure 9.5. To see what will happen, we can trace a typical case. Suppose that the tape initially contains $ab\dots$, with the read-write head on the a . The machine then reads the a , but does not change it. Its next state is q_1 , and the read-write head moves right, so that it is now over the b . This symbol is also read and left unchanged. The machine goes back into state q_0 , and the read-write head moves left. We are now back exactly in the original state, and the sequence of moves starts again. It is clear from this that the machine, whatever the initial information on its tape, will run forever, with the read-write head moving alternately right then left, but making no modifications to the tape. This is an instance of a Turing machine that does not halt. In analogy with programming terminology, we say that the Turing machine is in an **infinite loop**.

**FIGURE 9.5**

Since one can make several different definitions of a Turing machine, it is worthwhile to summarize the main features of our model, which we will call a **standard Turing machine**:

1. The Turing machine has a tape that is unbounded in both directions, allowing any number of left and right moves.
2. The Turing machine is deterministic in the sense that δ defines at most one move for each configuration.
3. There is no special input file. We assume that at the initial time the tape has some specified content. Some of this may be considered input. Similarly, there is no special output device. Whenever the machine halts, some or all of the contents of the tape may be viewed as output.

These conventions were chosen primarily for the convenience of subsequent discussion. In Chapter 10, we will look at other versions of Turing machines and discuss their relation to our standard model.

Here, as in the case of pda's, the most convenient way to exhibit a sequence of configurations of a Turing machine uses the idea of an instantaneous description. Any configuration is completely determined by the

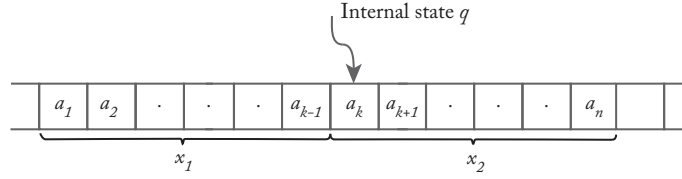


FIGURE 9.6

current state of the control unit, the contents of the tape, and the position of the read-write head. We will use the notation in which

$$x_1 q x_2$$

or

$$a_1 a_2 \cdots a_{k-1} q a_k a_{k+1} \cdots a_n$$

is the instantaneous description of a machine in state q with the tape depicted in Figure 9.6. The symbols $a_1, \dots, a_n$ show the tape contents, while q defines the state of the control unit. This convention is chosen so that the position of the read-write head is over the cell containing the symbol immediately following q .

The instantaneous description gives only a finite amount of information to the right and left of the read-write head. The unspecified part of the tape is assumed to contain all blanks; normally such blanks are irrelevant and are not shown explicitly in the instantaneous description. If the position of blanks is relevant to the discussion, however, the blank symbol may appear in the instantaneous description. For example, the instantaneous description $q \square w$ indicates that the read-write head is on the cell to the immediate left of the first symbol of w and that this cell contains a blank.

EXAMPLE 9.4

The pictures drawn in Figure 9.3 correspond to the sequence of instantaneous descriptions $q_0 aa$, $bq_0 a$, $bbq_0 \square$, $bq_1 b$.

A move from one configuration to another will be denoted by $\vdash$. Thus, if

$$\delta(q_1, c) = (q_2, e, R),$$

then the move

$$abq_1 cd \vdash abeq_2 d$$

is made whenever the internal state is q_1 , the tape contains $abcd$, and the read-write head is on the c . The symbol $\vdash^*$ has the usual meaning of an arbitrary number of moves. Subscripts, such as $\vdash_M$, are used in arguments to distinguish between several machines.

EXAMPLE 9.5

The action of the Turing machine in Figure 9.3 can be represented by

$$q_0aa \vdash bq_0a \vdash bbq_0\Box \vdash bq_1b$$

or

$$q_0aa \vdash^* bq_1b.$$

For further discussion, it is convenient to summarize the various observations just made in a formal way.

DEFINITION 9.2

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, \Box, F)$ be a Turing machine. Then any string $a_1 \cdots a_{k-1} q_1 a_k a_{k+1} \cdots a_n$, with $a_i \in \Gamma$ and $q_1 \in Q$, is an instantaneous description of M . A move

$$a_1 \cdots a_{k-1} q_1 a_k a_{k+1} \cdots a_n \vdash a_1 \cdots a_{k-1} b q_2 a_{k+1} \cdots a_n$$

is possible if and only if

$$\delta(q_1, a_k) = (q_2, b, R).$$

A move

$$a_1 \cdots a_{k-1} q_1 a_k a_{k+1} \cdots a_n \vdash a_1 \cdots q_2 a_{k-1} b a_{k+1} \cdots a_n$$

is possible if and only if

$$\delta(q_1, a_k) = (q_2, b, L).$$

M is said to halt starting from some initial configuration $x_1 q_i x_2$ if

$$x_1 q_i x_2 \vdash^* y_1 q_j a y_2$$

for any q_j and a , for which $\delta(q_j, a)$ is undefined. The sequence of configurations leading to a halt state will be called a **computation**.

Example 9.3 shows the possibility that a Turing machine will never halt, proceeding in an endless loop from which it cannot escape. This situation plays a fundamental role in the discussion of Turing machines, so we use a special notation for it. We will represent it by

$$x_1qx_2 \vdash^* \infty,$$

indicating that, starting from the initial configuration x_1qx_2 , the machine goes into a loop and never halts.

Turing Machines as Language Accepters

Turing machines can be viewed as accepters in the following sense. A string w is written on the tape, with blanks filling out the unused portions. The machine is started in the initial state q_0 with the read-write head positioned on the leftmost symbol of w . If, after a sequence of moves, the Turing machine enters a final state and halts, then w is considered to be accepted.

DEFINITION 9.3

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ be a Turing machine. Then the language accepted by M is

$$L(M) = \left\{ w \in \Sigma^+ : q_0w \vdash^* x_1q_fx_2 \text{ for some } q_f \in F, x_1, x_2 \in \Gamma^* \right\}.$$

This definition indicates that the input w is written on the tape with blanks on either side. The reason for excluding blanks from the input now becomes clear: It assures us that all the input is restricted to a well-defined region of the tape, bracketed by blanks on the right and left. Without this convention, the machine could not limit the region in which it must look for the input; no matter how many blanks it saw, it could never be sure that there was not some nonblank input somewhere else on the tape.

Definition 9.3 tells us what must happen when $w \in L(M)$. It says nothing about the outcome for any other input. When w is not in $L(M)$, one of two things can happen: The machine can halt in a nonfinal state, or it can enter an infinite loop and never halt. Any string for which M does not halt is by definition not in $L(M)$.

EXAMPLE 9.6

For $\Sigma = \{0, 1\}$, design a Turing machine that accepts the language denoted by the regular expression 00^* .

This is an easy exercise in Turing machine programming. Starting at the left end of the input, we read each symbol and check that it is a 0. If it is, we continue by moving right. If we reach a blank without encountering anything but 0, we terminate and accept the string. If the input contains a 1 anywhere, the string is not in $L(00^*)$, and we halt in a nonfinal state. To keep track of the computation, two internal states $Q = \{q_0, q_1\}$ and one final state $F = \{q_1\}$ are sufficient. As transition function we can take

$$\begin{aligned}\delta(q_0, 0) &= (q_0, 0, R), \\ \delta(q_0, \square) &= (q_1, \square, R).\end{aligned}$$

As long as a 0 appears under the read-write head, the head will move to the right. If at any time a 1 is read, the machine will halt in the nonfinal state q_0 , since $\delta(q_0, 1)$ is undefined. Note that the Turing machine also halts in a final state if started in state q_0 on a blank. We could interpret this as acceptance of λ , but for technical reasons the empty string is not included in Definition 9.3.

The recognition of more complicated languages is more difficult. Since Turing machines have a primitive instruction set, the computations that we can program easily in a higher-level language are often cumbersome on a Turing machine. Still, it is possible, and the concept is easy to understand, as the next examples illustrate.

EXAMPLE 9.7

For $\Sigma = \{a, b\}$, design a Turing machine that accepts

$$L = \{a^n b^n : n \geq 1\}.$$

Intuitively, we solve the problem in the following fashion. Starting at the leftmost a , we check it off by replacing it with some symbol, say x . We then let the read-write head travel right to find the leftmost b , which in turn is checked off by replacing it with another symbol, say y . After that, we go left again to the leftmost a , replace it with an x , then move to the leftmost b and replace it with y , and so on. Traveling back and forth this way, we match each a with a corresponding b . If after some time no a 's or b 's remain, then the string must be in L .

Working out the details, we arrive at a complete solution for which $Q = \{q_0, q_1, q_2, q_3, q_4\}$, $F = \{q_4\}$, $\Sigma = \{a, b\}$, $\Gamma = \{a, b, x, y, \square\}$. The transitions can be broken into several parts.

The set

$$\begin{aligned}\delta(q_0, a) &= (q_1, x, R), \\ \delta(q_1, a) &= (q_1, a, R), \\ \delta(q_1, y) &= (q_1, y, R), \\ \delta(q_1, b) &= (q_2, y, L)\end{aligned}$$

replaces the leftmost a with an x , then causes the read-write head to travel right to the first b , replacing it with a y . When the y is written, the machine enters a state q_2 , indicating that an a has been successfully paired with a b .

The next set of transitions reverses the direction until an x is encountered, repositions the read-write head over the leftmost a , and returns control to the initial state.

$$\begin{aligned}\delta(q_2, y) &= (q_2, y, L), \\ \delta(q_2, a) &= (q_2, a, L), \\ \delta(q_2, x) &= (q_0, x, R).\end{aligned}$$

We are now back in the initial state q_0 , ready to deal with the next a and b .

After one pass through this part of the computation, the machine will have carried out the partial computation

$$q_0aa \cdots abb \cdots b \stackrel{*}{\vdash} xq_0a \cdots ayb \cdots b,$$

so that a single a has been matched with a single b . After two passes, we will have completed the partial computation

$$q_0aa \cdots abb \cdots b \stackrel{*}{\vdash} xxq_0 \cdots ayy \cdots b,$$

and so on, indicating that the matching process is being carried out properly.

When the input is a string $a^n b^n$, the rewriting continues this way, stopping only when there are no more a 's to be erased. When looking for the leftmost a , the read-write head travels left with the machine in state q_2 . When an x is encountered, the direction is reversed to get the a . But now, instead of finding an a , it will find a y . To terminate, a final check is made to see if all a 's and b 's have been replaced (to detect input where an a follows a b). This can be done by

$$\begin{aligned}\delta(q_0, y) &= (q_3, y, R), \\ \delta(q_3, y) &= (q_3, y, R), \\ \delta(q_3, \square) &= (q_4, \square, R).\end{aligned}$$

If we input a string not in the language, the computation will halt in a nonfinal state. For example, if we give the machine a string $a^n b^m$, with $n > m$, the machine will eventually encounter a blank in state q_1 . It will halt because no transition is specified for this case. Other input not in the language will also lead to a nonfinal halting state (see Exercise 4 at the end of this section).

The particular input $aabb$ gives the following successive instantaneous descriptions:

$$\begin{aligned} q_0 aabb &\vdash xq_1 abb \vdash xaq_1 bb \vdash xq_2 ayb \\ &\vdash q_2 xayb \vdash xq_0 ayb \vdash xxq_1 yb \\ &\vdash xxyq_1 b \vdash xxq_2 yy \vdash xq_2 xyy \\ &\vdash xxq_0 yy \vdash xxyq_3 y \vdash xxyyq_3 \square \\ &\vdash xxyy \square q_4 \square. \end{aligned}$$

At this point the Turing machine halts in a final state, so the string $aabb$ is accepted.

You are urged to trace this program with several more strings in L , as well as with some not in L .

EXAMPLE 9.8

Design a Turing machine that accepts

$$L = \{a^n b^n c^n : n \geq 1\}.$$

The ideas used in Example 9.7 are easily carried over to this case. We match each a , b , and c by replacing them in order with x , y , and z , respectively. At the end, we check that all original symbols have been rewritten. Although conceptually a simple extension of the previous example, writing the actual program is tedious. We leave it as a somewhat lengthy but straightforward exercise. Notice that even though $\{a^n b^n\}$ is a context-free language and $\{a^n b^n c^n\}$ is not, they can be accepted by Turing machines with very similar structures.

One conclusion we can draw from this example is that a Turing machine can recognize some languages that are not context free, a first indication that Turing machines are more powerful than pushdown automata.

Turing Machines as Transducers

We have had little reason so far to study transducers; in language theory, accepters are quite adequate. But as we will shortly see, Turing machines are

not only interesting as language accepters, they also provide us with a simple abstract model for digital computers in general. Since the primary purpose of a computer is to transform input into output, it acts as a transducer. If we want to model computers using Turing machines, we have to look at this aspect more closely.

The input for a computation will be all the nonblank symbols on the tape at the initial time. At the conclusion of the computation, the output will be whatever is then on the tape. Thus, we can view a Turing machine transducer M as an implementation of a function f defined by

$$\hat{w} = f(w),$$

provided that

$$q_0 w \vdash_M^* q_f \hat{w},$$

for some final state q_f .

DEFINITION 9.4

A function f with domain D is said to be **Turing-computable** or just **computable** if there exists some Turing machine $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ such that

$$q_0 w \vdash_M^* q_f f(w), \quad q_f \in F,$$

for all $w \in D$.

As we will shortly claim, all the common mathematical functions, no matter how complicated, are Turing-computable. We start by looking at some simple operations, such as addition and arithmetic comparison.

EXAMPLE 9.9

Given two positive integers x and y , design a Turing machine that computes $x + y$.

We first have to choose some convention for representing positive integers. For simplicity, we will use unary notation in which any positive integer x is represented by $w(x) \in \{1\}^+$, such that

$$|w(x)| = x.$$

We must also decide how x and y are placed on the tape initially and how their sum is to appear at the end of the computation. We will

assume that $w(x)$ and $w(y)$ are on the tape in unary notation, separated by a single 0, with the read-write head on the leftmost symbol of $w(x)$. After the computation, $w(x+y)$ will be on the tape followed by a single 0, and the read-write head will be positioned at the left end of the result. We therefore want to design a Turing machine for performing the computation

$$q_0 w(x) 0 w(y) \vdash^* q_f w(x+y) 0,$$

where q_f is a final state. Constructing a program for this is relatively simple. All we need to do is to move the separating 0 to the right end of $w(y)$, so that the addition amounts to nothing more than the coalescing of the two strings. To achieve this, we construct $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$, with $Q = \{q_0, q_1, q_2, q_3, q_4\}$, $F = \{q_4\}$, and

$$\begin{aligned} \delta(q_0, 1) &= (q_0, 1, R), \\ \delta(q_0, 0) &= (q_1, 1, R), \\ \delta(q_1, 1) &= (q_1, 1, R), \\ \delta(q_1, \square) &= (q_2, \square, L), \\ \delta(q_2, 1) &= (q_3, 0, L), \\ \delta(q_3, 1) &= (q_3, 1, L), \\ \delta(q_3, \square) &= (q_4, \square, R). \end{aligned}$$

Note that in moving the 0 right we temporarily create an extra 1, a fact that is remembered by putting the machine into state q_1 . The transition $\delta(q_2, 1) = (q_3, 0, L)$ is needed to remove this at the end of the computation. This can be seen from the sequence of instantaneous descriptions for adding 111 to 11:

$$\begin{aligned} q_0 111011 &\vdash 1q_0 11011 \vdash 11q_0 1011 \vdash 111q_0 011 \\ &\vdash 1111q_1 11 \vdash 11111q_1 1 \vdash 111111q_1 \square \\ &\vdash 111111q_2 1 \vdash 1111q_3 10 \\ &\vdash^* q_3 \square 1111110 \vdash q_4 1111110. \end{aligned}$$

Unary notation, although cumbersome for practical computations, is very convenient for programming Turing machines. The resulting programs are much shorter and simpler than if we had used another representation, such as binary or decimal.

Adding numbers is one of the fundamental operations of any computer, one that plays a part in the synthesis of more complicated instructions. Other basic operations are copying strings and simple comparisons. These can also be done easily on a Turing machine.

EXAMPLE 9.10

Design a Turing machine that copies strings of 1's. More precisely, find a machine that performs the computation

$$q_0 w \vdash^* q_f w w,$$

for any $w \in \{1\}^+$.

To solve the problem, we implement the following intuitive process:

1. Replace every 1 by an x .
2. Find the rightmost x and replace it with 1.
3. Travel to the right end of the current nonblank region and create a 1 there.
4. Repeat Steps 2 and 3 until there are no more x 's.

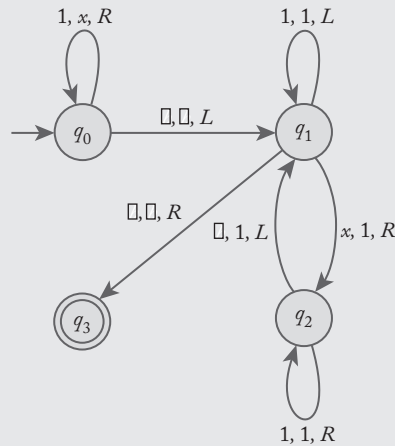


FIGURE 9.7

The solution is shown in the transition graph in Figure 9.7. It may be a little hard to see at first that the solution is correct, so let us trace the program with the simple string 11. The computation performed in this case is

$$\begin{aligned}
 q_0 11 &\vdash x q_0 1 \vdash x x q_0 \square \vdash x q_1 x \\
 &\vdash x 1 q_2 \square \vdash x q_1 11 \vdash q_1 x 11 \\
 &\vdash 1 q_2 11 \vdash 11 q_2 1 \vdash 111 q_2 \square \\
 &\vdash 11 q_1 11 \vdash 1 q_1 111 \\
 &\vdash q_1 1111 \vdash q_1 \square 1111 \vdash q_3 1111.
 \end{aligned}$$

EXAMPLE 9.11

Let x and y be two positive integers represented in unary notation. Construct a Turing machine that will halt in a final state q_y if $x \geq y$, and that will halt in a nonfinal state q_n if $x < y$. More specifically, the machine is to perform the computation

$$\begin{aligned}
 q_0 w(x) 0 w(y) &\vdash^* q_y w(x) 0 w(y) && \text{if } x \geq y, \\
 q_0 w(x) 0 w(y) &\vdash^* q_n w(x) 0 w(y) && \text{if } x < y.
 \end{aligned}$$

To solve this problem, we can use the idea in Example 9.7 with some minor modifications. Instead of matching a 's and b 's, we match each 1 on the left of the dividing 0 with the 1 on the right. At the end of the matching, we will have on the tape either

$$xx \cdots 110xx \cdots x \square$$

or

$$xx \cdots xx0xx \cdots x11 \square,$$

depending on whether $x > y$ or $y > x$. In the first case, when we attempt to match another 1, we encounter the blank at the right of the working space. This can be used as a signal to enter the state q_y . In the second case, we still find a 1 on the right when all 1's on the left have been replaced. We use this to get into the other state q_n . The complete program for this is straightforward and is left as an exercise.

This example makes the important point that a Turing machine can be programmed to make decisions based on arithmetic comparisons.

This kind of simple decision is common in the machine language of computers, where alternate instruction streams are entered, depending on the outcome of an arithmetic operation.

INTRODUCTORY EXERCISES

1. Describe the action of the Turing machine

$$\begin{aligned}\delta(q_0, a) &= (q_1, a, R), & \delta(q_1, a) &= (q_2, a, R), \\ \delta(q_2, a) &= (q_0, b, R)\end{aligned}$$

if it starts in state q_0 and input $w = a^n$.

2. Describe the action of the Turing machine

$$\begin{aligned}\delta(q_0, a) &= (q_0, a, R), & \delta(q_0, \square) &= (q_1, \square, L), \\ \delta(q_1, a) &= (q_1, a, L), & \delta(q_1, \square) &= (q_0, \square, R)\end{aligned}$$

if it starts in state q_0 and input is $w = a^n$.

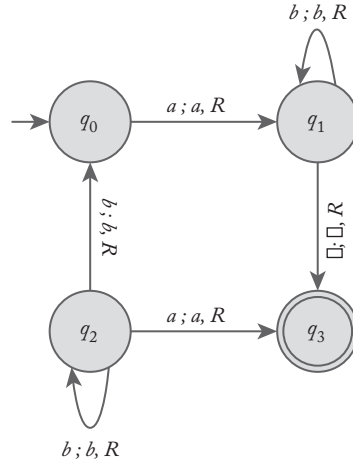
3. Write a program that scans input $w \in \{a, b\}^+$, replacing every b with a c , and then stops.
4. Construct a standard Turing machine that accepts $L = \{a^{2n}, n \geq 1\}$.
5. Design a standard Turing machine that accepts $L = \{(ab)^n, n \geq 1\}$.
6. Write a program that scans the input $w \in \{a, b\}^+$ and checks the number of b 's. If this number is odd, accept the input, otherwise reject.
7. Describe in words or pseudocode how one can construct a Turing machine program that appends $|w|$ in unary notation to an input $w \in \{a\}^+$.



EXERCISES

1. Construct a Turing machine that accepts the language $L = L(aaaa^*b^*)$.
2. Construct a Turing machine that accepts the complement of the language $L = L(aaaa^*b^*)$. Assume that $\Sigma = \{a, b\}$.
3. Design a Turing machine with no more than three states that accepts the language $L(a(a+b)^*)$. Assume that $\Sigma = \{a, b\}$. Is it possible to do this with a two-state machine?
4. Determine what the Turing machine in Example 9.7 does when presented with the inputs aba and $aaabbbb$.
5. Is there any input for which the Turing machine in Example 9.7 goes into an infinite loop?

6. What language is accepted by the Turing machine whose transition graph is in the figure below?



7. What happens in Example 9.10 if the string w contains any symbol other than 1?
8. Construct Turing machines that will accept the following languages on $\{a, b\}$:
- $L = L(aaba^*b)$.
 - $L = \{w : |w| \text{ is odd}\}$.
 - $L = \{w : |w| \text{ is a multiple of } 4\}$.
 - $L = \{a^n b^m : n \geq 2, n = m\}$.
 - $L = \{w : n_a(w) \neq n_b(w)\}$.
 - $L = \{a^n b^m a^{n+m} : n \geq 0, m \geq 1\}$.
 - $L = \{a^n b^n a^n b^n : n \neq 0\}$.
 - $L = \{a^n b^{2n} : n \geq 1\}$.

For each problem, give a transition graph; then check your answers by tracing several test examples.

9. Design a Turing machine that accepts the language

$$L = \{ww : w \in \{a, b\}^+\}.$$

10. Construct a Turing machine to compute the function

$$f(w) = w^R,$$

where $w \in \{0, 1\}^+$.

11. Design a Turing machine that computes the function for w , a positive integer represented in unary

$$\begin{aligned} f(w) &= 1 \text{ if } w \text{ is even} \\ &= 0 \text{ if } w \text{ is odd.} \end{aligned}$$

12. Design a Turing machine that computes the function for x , a positive integer represented in unary

$$\begin{aligned} f(x) &= x - 2 \text{ if } x > 2 \\ &= 0 \text{ if } x \leq 2. \end{aligned}$$

13. Design Turing machines to compute the following functions for x and y positive integers represented in unary:

(a) $f(x) = 2x + 1.$

(b) $f(x, y) = x + 2y.$

(c) $f(x, y) = 2x + 3y.$

(d) $f(x) = x \bmod 5.$

(e) $f(x) = \lfloor \frac{x}{2} \rfloor$, where $\lfloor \frac{x}{2} \rfloor$ denotes the largest integer less than or equal to $\frac{x}{2}$.

14. Design a Turing machine with $\Gamma = \{0, 1, \square\}$, which, when started on any cell containing a blank or a 1, will halt if and only if its tape has a 0 somewhere on it.
15. Write out a complete solution for Example 9.8.
16. Show the sequence of instantaneous descriptions that the Turing machine in Example 9.10 goes through when presented with the input 111. What happens when this machine is started with 110 on its tape?
17. Give convincing arguments that the Turing machine in Example 9.10 does in fact carry out the indicated computation.
18. Complete the details in Example 9.11.
19. Suppose that in Example 9.9 we had decided to represent x and y in binary. Write a Turing machine program for doing the indicated computation in this representation.
20. You may have noticed that all the examples in this section had only one final state. Is it generally true that for any Turing machine, there exists another one with only one final state that accepts the same language?
-

9.2 COMBINING TURING MACHINES FOR COMPLICATED TASKS

We have shown explicitly how some important operations found in all computers can be done on a Turing machine. Since, in digital computers, such primitive operations are the building blocks for more complex instructions, let us see how these basic operations can also be put together on a Turing machine. To demonstrate how Turing machines can be combined, we follow a practice common in programming. We start with a high-level description, then refine it successively until the program is in the actual language with which we are working. We can describe Turing machines several ways at a high level; block diagrams or pseudocode are the two approaches we will use most frequently in subsequent discussions. In a block diagram, we encapsulate computations in boxes whose function is described, but whose interior details are not shown. By using such boxes, we implicitly claim that they can actually be constructed. As a first example, we combine the machines in Examples 9.9 and 9.11.

EXAMPLE 9.12

Design a Turing machine that computes the function

$$\begin{aligned} f(x, y) &= x + y && \text{if } x \geq y, \\ &= 0 && \text{if } x < y. \end{aligned}$$

For the sake of discussion, assume that x and y are positive integers in unary representation. The value zero will be represented by 0, with the rest of the tape blank.

The computation of $f(x, y)$ can be visualized at a high level by means of the diagram in Figure 9.8. The diagram shows that we first use a comparing machine, like that in Example 9.11, to determine whether or not $x \geq y$. If so, the comparer sends a start signal to the adder, which then computes $x + y$. If not, an erasing program is started that changes every 1 to a blank.

In subsequent discussions, we will often use such high-level, black-diagram representations of Turing machines. It is certainly quicker and clearer than the corresponding extensive set of δ 's. Before we accept this high-level view, we must justify it. What, for example, is meant by saying that the comparer sends a start signal to the adder? There is nothing in Definition 9.1 that offers that possibility. Nevertheless, it can be done in a straightforward way.

The program for the comparer C is written as suggested in Example 9.11, using a Turing machine having states indexed with C . For the adder, we use the idea in Example 9.9, with states indexed with A .

For the eraser E , we construct a Turing machine having states indexed with E . The computations to be done by C are

$$q_{C,0}w(x)0w(y) \vdash^* q_{A,0}w(x)0w(y) \quad \text{if } x \geq y,$$

and

$$q_{C,0}w(x)0w(y) \vdash^* q_{E,0}w(x)0w(y) \quad \text{if } x < y.$$

If we take $q_{A,0}$ and $q_{E,0}$ as the initial states of A and E , respectively, we see that C starts either A or E .

The computations performed by the adder will be

$$q_{A,0}w(x)0w(y) \vdash^* q_{A,f}w(x+y)0,$$

and that of the eraser E will be

$$q_{E,0}w(x)0w(y) \vdash^* q_{E,f}0.$$

The result is a single Turing machine that combines the action of C , A , and E as indicated in Figure 9.8.

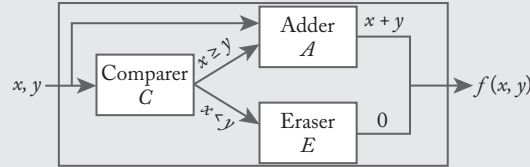


FIGURE 9.8

Another useful, high-level view of Turing machines involves pseudocode. In computer programming, pseudocode is a way of outlining a computation using descriptive phrases whose meaning we claim to understand. While this description is not usable on the computer, we assume that we can translate it into the appropriate language when needed. One simple kind of pseudocode is exemplified by the idea of a macroinstruction, which is a single-statement shorthand for a sequence of lower-level statements. We first define the macroinstruction in terms of the lower-level language. We then use the macroinstruction in a program with the assumption that the relevant low-level code is substituted for each occurrence of the macroinstruction. This idea is very useful in Turing machine programming.

EXAMPLE 9.13

Consider the macroinstruction

$$\text{if } a \text{ then } q_j \text{ else } q_k,$$

with the following interpretation. If the Turing machine reads an a , then regardless of its current state, it is to go into state q_j without changing the tape content or moving the read-write head. If the symbol read is not an a , the machine is to go into state q_k without changing anything.

To implement this macroinstruction requires several relatively obvious steps of a Turing machine.

$$\begin{aligned}\delta(q_i, a) &= (q_{j0}, a, R) \text{ for all } q_i \in Q, \\ \delta(q_i, b) &= (q_{k0}, b, R) \text{ for all } q_i \in Q \text{ and all } b \in \Gamma - \{a\}, \\ \delta(q_{j0}, c) &= (q_j, c, L) \text{ for all } c \in \Gamma, \\ \delta(q_{k0}, c) &= (q_k, c, L) \text{ for all } c \in \Gamma.\end{aligned}$$

The states q_{j0} and q_{k0} are new states, introduced to take care of complications arising from the fact that in a standard Turing machine the read-write head changes position in each move. In the macroinstruction, we want to change the state but leave the read-write head where it is. We let the head move right but put the machine into a state q_{j0} or q_{k0} . This indicates that a left move must be made before entering the desired state q_j or q_k .

Going a step further, we can replace macroinstructions with subprograms. Normally, a macroinstruction is replaced by actual code at each occurrence, whereas a subprogram is a single piece of code that is invoked repeatedly whenever needed. Subprograms are fundamental to high-level programming languages, but they can also be used with Turing machines. To make this plausible, let us outline briefly how a Turing machine can be used as a subprogram that can be invoked repeatedly by another Turing machine. This requires a new feature: the ability to store information on the calling program's configuration so the configuration can be recreated on return from the subprogram. For example, say machine A in state q_i invokes machine B . When B is finished, we would like to resume program A in state q_i , with the read-write head (which may have moved during B 's operation) in its original place. At other times, A may call B from state q_j , in which case control should return to this state. To solve the control transfer problem, we must be able to pass information from A to B , and vice versa, to be able to recreate A 's configuration when it recovers control from B and ensure that the temporarily suspended computations of A are

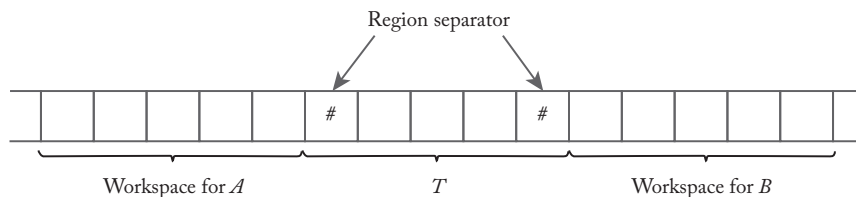


FIGURE 9.9

not affected by the execution of B . To solve this, we can divide the tape into several regions as shown in Figure 9.9.

Before A calls B , it writes the information needed by B (e.g., A 's current state, the arguments for B) on the tape in some region T . A then passes control to B by making a transition to the start state of B . After transfer, B will use T to find its input. The workspace for B is separate from T and from the workspace for A , so no interference can occur. When B is finished, it will return relevant results to region T , where A will expect to find it. In this way, the two programs can interact in the required fashion. Note that this is very similar to what actually happens in a real computer when a subprogram is called.

We can now program Turing machines in pseudocode, provided that we know (in theory at least) how to translate this pseudocode into an actual Turing machine program.

EXAMPLE 9.14

Design a Turing machine that multiplies two positive integers in unary notation.

A multiplication machine can be constructed by combining the ideas we encountered in adding and copying. Let us assume that the initial and final tape contents are to be as indicated in Figure 9.10. The process of multiplication can then be visualized as a repeated copying of the multiplicand y for each 1 in the multiplier x , whereby the string y is added the appropriate number of times to the partially computed product. The following pseudocode shows the main steps of the process.

1. Repeat the following steps until x contains no more 1's.
 Find a 1 in x and replace it with another symbol a .
 Replace the leftmost 0 by $0y$.
2. Replace all a 's with 1's.

Although this pseudocode is sketchy, the idea is simple enough that there should be no doubt that it can be done.

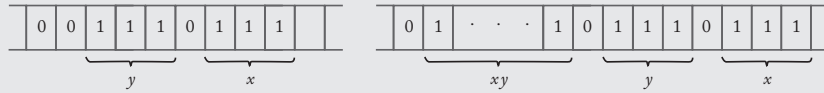


FIGURE 9.10

In spite of the descriptive nature of these examples, it is not too far-fetched to conjecture that Turing machines, while rather primitive in principle, can be combined in many ways to make them quite powerful. Our examples were not general and detailed enough for us to claim that we have proved anything, but it should be plausible at this point that Turing machines can do some quite complicated things.

EXERCISES

1. Write out the complete solution to Example 9.14.
2. Establish a convention for representing positive and negative integers in unary notation. With your convention, sketch the construction of a *subtractor* for computing $x - y$.
3. Using adders, subtracters, comparers, copiers, or multipliers, draw block diagrams for Turing machines that compute the following functions for all positive integers n :

(a) $f(n) = n(n + 2)$.

(b) $f(n) = n^4$.

(c) $f(n) = 2^n$.

(d) $f(n) = n!$.

(e) $f(n) = 2^{n!}$.

4. Use a block diagram to sketch the implementation of a function f defined for all $w_1, w_2, w_3 \in \{1\}^+$ by

$$f(w_1, w_2, w_3) = i,$$

where i is such that $|w_i| = \max(|w_1|, |w_2|, |w_3|)$ if no two w 's have the same length, and $i = 0$ otherwise.

5. Provide a "high-level" description for Turing machines that accept the following languages on $\{a, b\}$. For each problem, define a set of appropriate macroinstructions that you feel are reasonably easy to implement. Then use them for the solution.

(a) $L = \{ww^Rw\}$.

(b) $L = \{w_1w_2 : w_1 \neq w_2 : |w_1| = |w_2|\}$.

- (c) The complement of the language in part (b).
 - (d) $L = \{a^n b^m : n = m^2, m \geq 1\}$.
 - (e) $L = \{a^n : n \text{ is a prime number}\}$.
6. Suggest a method for representing rational numbers on a Turing machine, then sketch a method for adding and subtracting such numbers.
 7. Sketch the construction of a Turing machine that can perform the addition and multiplication of positive integers x and y given in the usual decimal notation.
 8. Give an implementation of the macroinstruction

$$\text{searchleft}(a, q_i, q_j),$$

which indicates that the machine is to search its tape to the left of the current position for the first occurrence of the symbol a . If an a is encountered before a blank, the machine is to go into state q_i , otherwise it is to go into state q_j .

9. Use the macroinstruction `searchleft` to design a Turing machine on $\Sigma = \{a, b\}$ that accepts the language $L(ab^*ab^*a)$.
-

9.3 TURING'S THESIS

The preceding discussion not only shows how a Turing machine can be constructed from simpler parts, but it also illustrates a negative aspect of working with such low-level automata. While it takes very little imagination or ingenuity to translate a block diagram or pseudocode into the corresponding Turing machine program, actually doing it is time-consuming, error-prone, and adds little to our understanding. The instruction set of a Turing machine is so restricted that any argument, solution, or proof for a nontrivial problem is quite tedious.

We now face a dilemma: We want to claim that Turing machines can perform not only the simple operations for which we have provided explicit programs, but also more complex processes as well, describable by block diagrams or pseudocode. To defend such claims against challenge, we should show the relevant programs explicitly. But doing so is unpleasant and distracting and ought to be avoided if possible. Somehow, we would like to find a way of carrying out a reasonably rigorous discussion of Turing machines without having to write lengthy, low-level code. There is unfortunately no completely satisfactory way of getting out of the predicament; the best we can do is to reach a reasonable compromise. To see how we might achieve such a compromise, we turn to a somewhat philosophical issue.

We can draw some simple conclusions from the examples in the previous section. The first is that Turing machines appear to be more powerful than

pushdown automata (for a comment on this, see the exercise at the end of this section). In Example 9.8, we sketched the construction of a Turing machine for a language that is not context free and for which, consequently, no pushdown automaton exists. Examples 9.9, 9.10, and 9.11 show that Turing machines can do some simple arithmetic operations, perform string manipulations, and make some simple comparisons. The discussion also illustrates how primitive operations can be combined to solve more complex problems, how several Turing machines can be composed, and how one program can act as a subprogram for another. Since very complex operations can be built this way, we might suspect that a Turing machine begins to approach a typical computer in power.

Suppose we were to make the conjecture that, in some sense, Turing machines are equal in power to a typical digital computer? How could we defend or refute such a hypothesis? To defend it, we could take a sequence of increasingly more difficult problems and show how they are solved by some Turing machine. We might also take the machine language instruction set of a specific computer and design a Turing machine that can perform all the instructions in the set. This would undoubtedly tax our patience, but it ought to be possible in principle if our hypothesis is correct. Still, while every success in this direction would strengthen our conviction of the truth of the hypothesis, it would not lead to a proof. The difficulty lies in the fact that we don't know exactly what is meant by "a typical digital computer" and that we have no means for making a precise definition.

We can also approach the problem from the other side. We might try to find some procedure for which we can write a computer program, but for which we can show that no Turing machine can exist. If this were possible, we would have a basis for rejecting the hypothesis. But no one has yet been able to produce a counterexample; the fact that all such tries have been unsuccessful must be taken as circumstantial evidence that it cannot be done. Every indication is that Turing machines are in principle as powerful as any computer.

Arguments of this type led A. M. Turing and others in the mid-1930s to the celebrated conjecture called the **Turing thesis**. This hypothesis states that any computation that can be carried out by mechanical means can be performed by some Turing machine.

This is a sweeping statement, so it is important to keep in mind what Turing's thesis is. It is not something that can be proved. To do so, we would have to define precisely the term "mechanical means." This would require some other abstract model and leave us no further ahead than before. The Turing thesis is more properly viewed as a definition of what constitutes a mechanical computation: A computation is mechanical if and only if it can be performed by some Turing machine.

If we take this attitude and regard the Turing thesis simply as a definition, we raise the question as to whether this definition is sufficiently broad. Is it far-reaching enough to cover everything we now do (and conceivably might do in the future) with computers? An unequivocal “yes” is not possible, but the evidence in its favor is very strong. Some arguments for accepting the Turing thesis as the definition of a mechanical computation are

1. Anything that can be done on any existing digital computer can also be done by a Turing machine.
2. No one has yet been able to suggest a problem, solvable by what we intuitively consider an algorithm, for which a Turing machine program cannot be written.
3. Alternative models have been proposed for mechanical computation, but none of them is more powerful than the Turing machine model.

These arguments are circumstantial, and Turing's thesis cannot be proved by them. In spite of its plausibility, Turing's thesis is still an assumption. But viewing Turing's thesis simply as an arbitrary definition misses an important point. In some sense, Turing's thesis plays the same role in computer science as do the basic laws of physics and chemistry. Classical physics, for example, is based largely on Newton's laws of motion. Although we call them laws, they do not have logical necessity; rather, they are plausible models that explain much of the physical world. We accept them because the conclusions we draw from them agree with our experience and our observations. Such laws cannot be proved to be true, although they can possibly be invalidated. If an experimental result contradicts a conclusion based on the laws, we might begin to question their validity. On the other hand, repeated failure to invalidate a law strengthens our confidence in it. This is the situation for Turing's thesis, so we have some reason for considering it a basic law of computer science. The conclusions we draw from it agree with what we know about real computers, and so far, all attempts to invalidate it have failed. There is always the possibility that someone will come up with another definition that will account for some subtle situations not covered by Turing machines but which still fall within the range of our intuitive notion of mechanical computation. In such an eventuality, some of our subsequent discussions would have to be modified significantly. However, the likelihood of this happening seems to be very small.

Having accepted Turing's thesis, we are in a position to give a precise definition of an algorithm.

DEFINITION 9.5

An **algorithm** for a function $f: D \rightarrow R$ is a Turing machine M , which given as input any $d \in D$ on its tape, eventually halts with the correct answer $f(d) \in R$ on its tape. Specifically, we can require that

$$q_0 d \vdash_M^* q_f f(d), q_f \in F,$$

for all $d \in D$.

Identifying an algorithm with a Turing machine program allows us to prove rigorously such claims as “there exists an algorithm . . .” or “there is no algorithm. . . .” However, to construct explicitly an algorithm for even relatively simple problems is a very lengthy undertaking. To avoid such unpleasant prospects, we can appeal to Turing’s thesis and claim that anything we can do on any computer can also be done on a Turing machine. Consequently, we could substitute “C program” for “Turing machine” in Definition 9.5. This would ease the burden of exhibiting algorithms considerably. Actually, as we have already done, we will go one step further and accept verbal descriptions or block diagrams as algorithms on the assumption that we could write a Turing machine program for them if we were challenged to do so. This greatly simplifies the discussion, but it obviously leaves us open to criticism. While “C program” is well defined, “clear verbal description” is not, and we are in danger of claiming the existence of nonexistent algorithms. But this danger is more than offset by the facts that we can keep the discussion simple and intuitively clear and that we can give concise descriptions for some rather complex processes. The reader who has any doubts about the validity of these claims can dispel them by writing a suitable program in some programming language.

EXERCISES

1. In the above discussion, we stated at one point that Turing machines appear to be more powerful than pushdown automata. Because the tape of a Turing machine can always be made to behave like a stack, it would seem that we could have actually claimed that Turing machines are more powerful. What important factor was not taken into account in the argument that must be addressed before such a claim is justified?

DEFINITION 10.3

A nondeterministic Turing machine M is said to *accept* a language L if, for all $w \in L$, at least one of the possible configurations accepts w . There may be branches that lead to nonaccepting configurations, while some may put the machine into an infinite loop. But these are irrelevant for acceptance.

A nondeterministic Turing machine M is said to *decide* a language L if, for all $w \in \Sigma^*$, there is a path that leads either to acceptance or rejection.

EXERCISES

1. Discuss the simulation of a nondeterministic Turing machine by a deterministic one. Indicate explicitly how new machines are created, how active machines are identified, and how machines that halt are removed from further consideration.
2. Write programs for nondeterministic Turing machines that accept the languages below. In each case, explain if and how the nondeterminism simplifies the task.
 - (a) $L = \{ww : w \in \{a, b\}^+\}$.
 - (b) $L = \{ww^Rw : w \in \{a, b\}^+\}$.
 - (c) $L = \{xww^Ry : x, y, w \in \{a, b\}^+, |x| \geq |y|\}$.
 - (d) $L = \{w : n_a(w) = n_b(w) = n_c(w)\}$.
 - (e) $L = \{a^n : n \text{ is not a prime number}\}$.
3. A two-stack automaton is a nondeterministic pushdown automaton with two independent stacks. To define such an automaton, we modify Definition 7.1 so that

$$\delta : Q \times (\Sigma \cup \{\lambda\}) \times \Gamma \times \Gamma \rightarrow \text{finite subsets of } Q \times \Gamma^* \times \Gamma^*.$$

A move depends on the tops of the two stacks and results in new values being pushed on these two stacks. Show that the class of two-stack automata is equivalent to the class of Turing machines.

10.4 A UNIVERSAL TURING MACHINE

Consider the following argument against Turing's thesis: "A Turing machine as presented in Definition 9.1 is a special-purpose computer. Once δ is defined, the machine is restricted to carrying out one particular type of computation. Digital computers, on the other hand, are general-purpose machines that can be programmed to do different jobs at different times.

Consequently, Turing machines cannot be considered equivalent to general-purpose digital computers.”

This objection can be overcome by designing a reprogrammable Turing machine, called a **universal Turing machine**. A universal Turing machine M_u is an automaton that, given as input the description of any Turing machine M and a string w , can simulate the computation of M on w . To construct such an M_u , we first choose a standard way of describing Turing machines. We may, without loss of generality, assume that

$$Q = \{q_1, q_2, \dots, q_n\},$$

with q_1 the initial state, q_2 the single final state, and

$$\Gamma = \{a_1, a_2, \dots, a_m\},$$

where a_1 represents the blank. We then select an encoding in which q_1 is represented by 1, q_2 is represented by 11, and so on. Similarly, a_1 is encoded as 1, a_2 as 11, etc. The symbol 0 will be used as a separator between the 1's. With the initial and final state and the blank defined by this convention, any Turing machine can be described completely with δ only. The transition function is encoded according to this scheme, with the arguments and result in some prescribed sequence. For example, $\delta(q_1, a_2) = (q_2, a_3, L)$ might appear as

$$\dots 1011011011011010\dots.$$

It follows from this that any Turing machine has a finite encoding as a string on $\{0, 1\}^+$ and that, given any encoding of M , we can decode it uniquely. Some strings will not represent any Turing machine (e.g., the string 00011), but we can easily spot these, so they are of no concern.

A universal Turing machine M_u then has an input alphabet that includes $\{0, 1\}$ and the structure of a multitape machine, as shown in Figure 10.16.

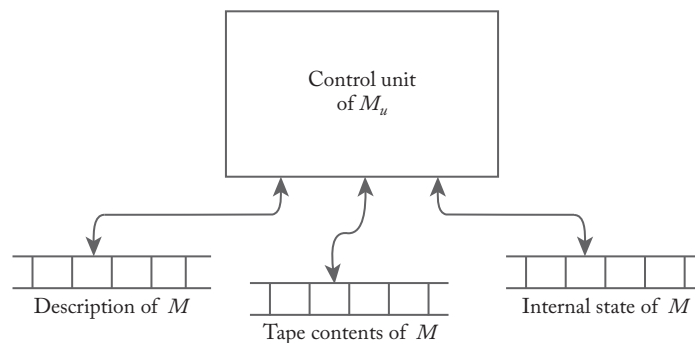


FIGURE 10.16

For any input M and w , tape 1 will keep an encoded definition of M . Tape 2 will contain the tape contents of M , and tape 3 the internal state of M . M_u looks first at the contents of tapes 2 and 3 to determine the configuration of M . It then consults tape 1 to see what M would do in this configuration. Finally, tapes 2 and 3 will be modified to reflect the result of the move.

It is within reason to construct an actual universal Turing machine (see, for example, Denning, Dennis, and Qualitz 1978), but the process is uninteresting. We prefer instead to appeal to Turing's hypothesis. The implementation clearly can be done using some programming language. Therefore, we expect that it can also be done by a standard Turing machine. We are then justified in claiming the existence of a Turing machine that, given any program, can carry out the computations specified by that program and that is therefore a proper model for a general-purpose computer.

The observation that every Turing machine can be represented by a string of 0's and 1's has important implications. But before we explore these implications, we need to review some results from set theory.

Some sets are finite, but most of the interesting sets (and languages) are infinite. For infinite sets, we distinguish between sets that are **countable** and sets that are **uncountable**. A set is said to be countable if its elements can be put into a one-to-one correspondence with the positive integers. By this we mean that the elements of the set can be written in some order, say, $x_1, x_2, x_3, \dots$, so that every element of the set has some finite index. For example, the set of all even integers can be written in the order $0, 2, 4, \dots$. Since any positive integer $2n$ occurs in position $n + 1$, the set is countable. This should not be too surprising, but there are more complicated examples, some of which may seem counterintuitive. Take the set of all quotients of the form p/q , where p and q are positive integers. How should we order this set to show that it is countable? We cannot use the sequence

$$\frac{1}{1}, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \dots$$

because then $\frac{2}{3}$ would never appear. This does not imply that the set is uncountable; in this case, there is a clever way of ordering the set to show that it is in fact countable. Look at the scheme depicted in Figure 10.17, and write down the element in the order encountered following the arrows. This gives us

$$\frac{1}{1}, \frac{1}{2}, \frac{2}{1}, \frac{3}{1}, \frac{2}{2}, \dots$$

Here the element $\frac{2}{3}$ occurs in the seventh place, and every element has some position in the sequence. The set is therefore countable.

We see from this example that we can prove that a set is countable if we can produce a method by which its elements can be written in some sequence. We call such a method an **enumeration procedure**. Since an

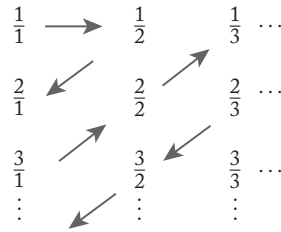


FIGURE 10.17

enumeration procedure is some kind of mechanical process, we can use a Turing machine model to define it formally.

DEFINITION 10.4

Let S be a set of strings on some alphabet Σ . Then an enumeration procedure for S is a Turing machine that can carry out the sequence of steps

$$q_0 \square \vdash^* q_s x_1 \# s_1 \vdash^* q_s x_2 \# s_2 \dots,$$

with $x_i \in \Gamma^* - \{\#\}$, $s_i \in S$, in such a way that any s in S is produced in a finite number of steps. The state q_s is a state signifying membership in S ; that is, whenever q_s is entered, the string following $\#$ must be in S .

Not every set is countable. As we will see in the next chapter, there are some uncountable sets. But any set for which an enumeration procedure exists is countable because the enumeration gives the required sequence.

Strictly speaking, an enumeration procedure cannot be called an algorithm since it will not terminate when S is infinite. Nevertheless, it can be considered a meaningful process because it produces well-defined and predictable results.

EXAMPLE 10.3

Let $\Sigma = \{a, b, c\}$. We can show that the $S = \Sigma^+$ is countable if we can find an enumeration procedure that produces its elements in some order, say in the order in which they would appear in a dictionary. However, the order used in dictionaries is not suitable without modification. In a dictionary, all words beginning with a are listed before the string b . But when there are an infinite number of a words, we will never reach b , thus violating the condition of Definition 10.4 that any given string be listed after a finite number of steps.

Instead, we can use a modified order, in which we take the length of the string as the first criterion, followed by an alphabetic ordering of all equal-length strings. This is an enumeration procedure that gives the sequence

$$a, b, c, aa, ab, ac, ba, bb, bc, ca, cb, cc, aaa, \dots$$

As we will have several uses for such an ordering, we will call it the **proper order**.

An important consequence of the previous discussion is that Turing machines are countable.

THEOREM 10.3

The set of all Turing machines, although infinite, is countable.

Proof: We can encode each Turing machine using 0 and 1. With this encoding, we then construct the following enumeration procedure.

1. Generate the next string in $\{0, 1\}^+$ in proper order.
2. Check the generated string to see if it defines a Turing machine. If so, write it on the tape in the form required by Definition 10.4. If not, ignore the string.
3. Return to Step 1.

Since every Turing machine has a finite description, any specific machine will eventually be generated by this process. ■

The particular ordering of Turing machines depends on the encoding we use; if we use a different encoding, we must expect a different ordering. This is of no consequence, however, and shows that the ordering itself is unimportant. What matters is the existence of some ordering.

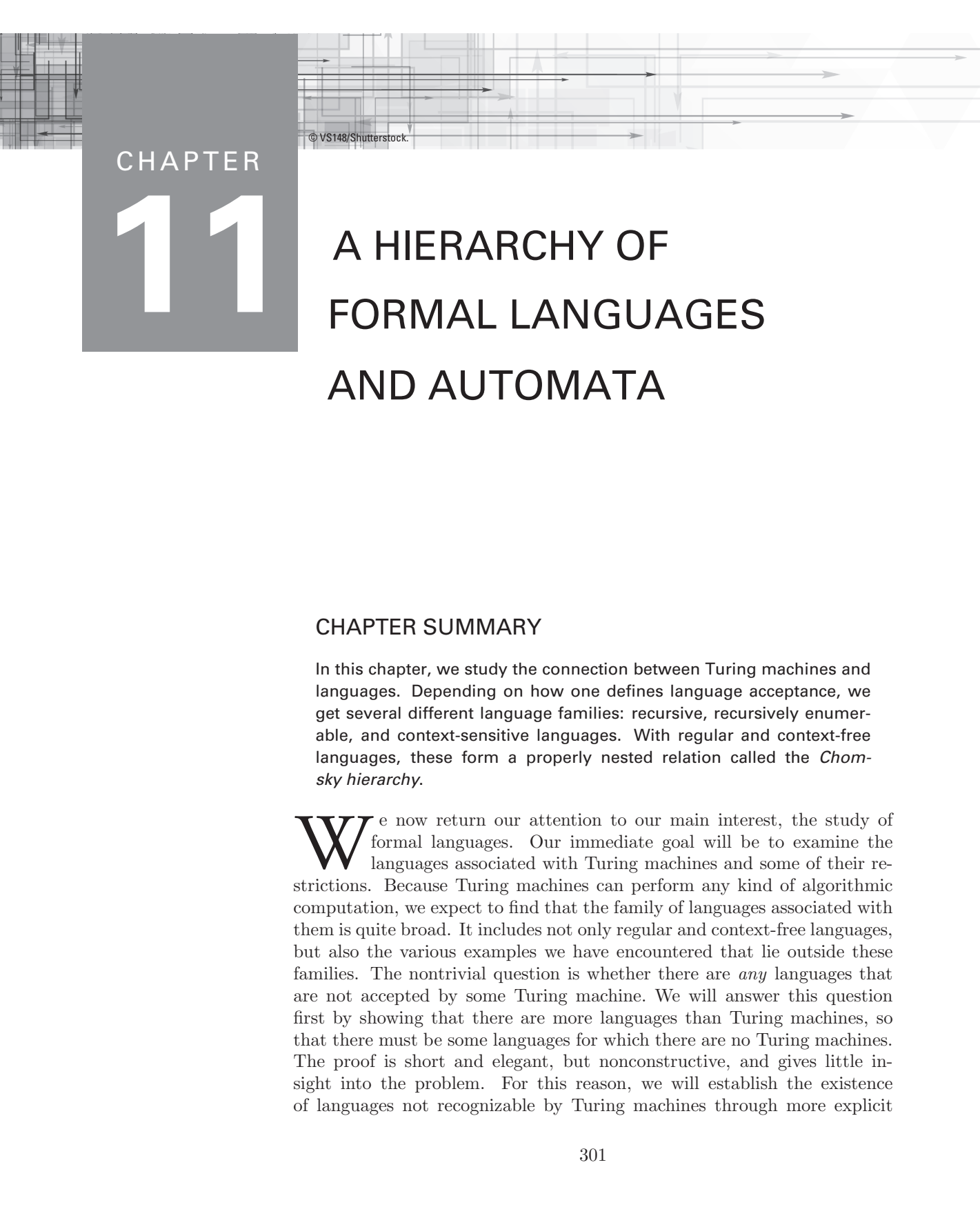
EXERCISES

1. Give the encoding, using the suggested method, for

$$\delta(q_1, a_1) = (q_1, a_1, R),$$

$$\delta(q_1, a_2) = (q_3, a_1, L),$$

$$\delta(q_3, a_1) = (q_2, a_2, L).$$



CHAPTER

11

A HIERARCHY OF FORMAL LANGUAGES AND AUTOMATA

CHAPTER SUMMARY

In this chapter, we study the connection between Turing machines and languages. Depending on how one defines language acceptance, we get several different language families: recursive, recursively enumerable, and context-sensitive languages. With regular and context-free languages, these form a properly nested relation called the *Chomsky hierarchy*.

We now return our attention to our main interest, the study of formal languages. Our immediate goal will be to examine the languages associated with Turing machines and some of their restrictions. Because Turing machines can perform any kind of algorithmic computation, we expect to find that the family of languages associated with them is quite broad. It includes not only regular and context-free languages, but also the various examples we have encountered that lie outside these families. The nontrivial question is whether there are *any* languages that are not accepted by some Turing machine. We will answer this question first by showing that there are more languages than Turing machines, so that there must be some languages for which there are no Turing machines. The proof is short and elegant, but nonconstructive, and gives little insight into the problem. For this reason, we will establish the existence of languages not recognizable by Turing machines through more explicit

examples that actually allow us to identify one such language. Another avenue of investigation will be to look at the relation between Turing machines and certain types of grammars and to establish a connection between these grammars and regular and context-free grammars. This leads to a hierarchy of grammars and through it to a method for classifying language families. Some set-theoretic diagrams illustrate the relationships between various language families clearly.

Strictly speaking, many of the arguments in this chapter are valid only for languages that do not include the empty string. This restriction arises from the fact that Turing machines, as we have defined them, cannot accept the empty string. To avoid having to rephrase the definition or having to add a repeated disclaimer, we make the tacit assumption that the languages discussed in this chapter, unless otherwise stated, do not contain λ . It is a trivial matter to restate everything so that λ is included, but we will leave this to the reader.

11.1 RECURSIVE AND RECURSIVELY ENUMERABLE LANGUAGES

We start with some terminology for the languages associated with Turing machines. In doing so, we must make the important distinction between languages for which there exists an accepting Turing machine and languages for which there exists a membership algorithm. Because a Turing machine does not necessarily halt on input that it does not accept, the first does not imply the second.

DEFINITION 11.1

A language L is said to be **recursively enumerable** if there exists a Turing machine that accepts it.

This definition implies only that there exists a Turing machine M , such that, for every $w \in L$,

$$q_0 w \vdash_M^* x_1 q_f x_2,$$

with q_f a final state. The definition says nothing about what happens for w not in L ; it may be that the machine halts in a nonfinal state or that it never halts and goes into an infinite loop. We can be more demanding and ask that the machine tell us whether or not any given input is in its language.

That there is also an enumeration procedure for every recursively enumerable language is not as easy to see. We cannot use the previous argument as it stands, because if some w_j is not in L , the machine M , when started with w_j on its tape, may never halt and therefore never get to the strings in L that follow w_j in the enumeration. To make sure that this does not happen, the computation is performed in a different way. We first get $\hat{M}$ to generate w_1 and let M execute one move on it. Then we let $\hat{M}$ generate w_2 and let M execute one move on w_2 , followed by the second move on w_1 . After this, we generate w_3 and do one step on w_3 , the second step on w_2 , the third step on w_1 , and so on. The order of performance is depicted in Figure 11.1. From this, it is clear that M will never get into an infinite loop. Since any $w \in L$ is generated by $\hat{M}$ and accepted by M in a finite number of steps, every string in L is eventually produced by M .

Definitions 11.1 and 11.2 give us very little insight into the nature of either recursive or recursively enumerable languages. These definitions

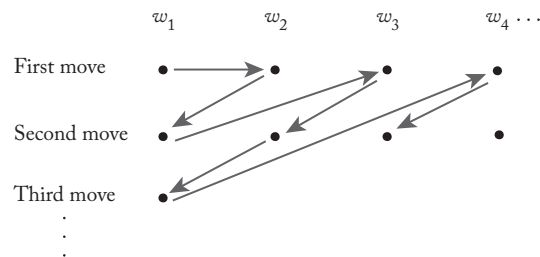


FIGURE 11.1

attach names to language families associated with Turing machines, but shed no light on the nature of representative languages in these families. Nor do they tell us much about the relationships between these languages or their connection to the language families we have encountered before. We are therefore immediately faced with questions such as “Are there languages that are recursively enumerable but not recursive?” and “Are there languages, describable somehow, that are not recursively enumerable?” While we will be able to supply some answers, we will not be able to produce very explicit examples to illustrate these questions, especially the second one.

Languages That Are Not Recursively Enumerable

We can establish the existence of languages that are not recursively enumerable in a variety of ways. One is very short and uses a very fundamental and elegant result of mathematics.

THEOREM 11.1

Let S be an infinite countable set. Then its powerset 2^S is not countable.

Proof: Let $S = \{s_1, s_2, s_3, \dots\}$. Then any element t of 2^S can be represented by a sequence of 0's and 1's, with a 1 in position i if and only if s_i is in t . For example, the set $\{s_2, s_3, s_6\}$ is represented by 01100100..., while $\{s_1, s_3, s_5, \dots\}$ is represented by 10101.... Clearly, any element of 2^S can be represented by such a sequence, and any such sequence represents a unique element of 2^S . Suppose that 2^S were countable; then its elements could be written in some order, say $t_1, t_2, \dots$, and we could enter these into a table, as shown in Figure 11.2. In this table, take the elements in the main diagonal, and complement each entry, that is, replace 0 with 1, and vice versa. In the example in Figure 11.2, the elements are 1100..., so we get 0011... as

t_1	1	0	0	0	0	...
t_2	1	1	0	0	0	...
t_3	1	1	0	1	0	...
t_4	1	1	0	0	1	...
.	.	.	.	.	.	.
.	.	.	.	.	.	.
.	.	.	.	.	.	.

FIGURE 11.2

the result. The new sequence along the diagonal represents some element of 2^S , say t_i for some i . But it cannot be t_1 because it differs from t_1 through s_1 . For the same reason it cannot be t_2, t_3 , or any other entry in the enumeration. This contradiction creates a logical impasse that can be removed only by throwing out the assumption that 2^S is countable. ■

This kind of argument, because it involves a manipulation of the diagonal elements of a table, is called **diagonalization**. The technique is attributed to the mathematician G. F. Cantor, who used it to demonstrate that the set of real numbers is not countable. In the next few chapters, we will see a similar argument in several contexts. Theorem 11.1 is diagonalization in its purest form.

As an immediate consequence of this result, we can show that, in some sense, there are fewer Turing machines than there are languages, so that there must be some languages that are not recursively enumerable.

THEOREM 11.2

For any nonempty Σ , there exist languages that are not recursively enumerable.

Proof: A language is a subset of Σ^* , and every such subset is a language. Therefore, the set of all languages is exactly 2^{Σ^*} . Since Σ^* is infinite, Theorem 11.1 tells us that the set of all languages on Σ is not countable. But the set of all Turing machines can be enumerated, so the set of all recursively enumerable languages is countable. By Exercise 16 at the end of this section, this implies that there must be some languages on Σ that are not recursively enumerable. ■

This proof, although short and simple, is in many ways unsatisfying. It is completely nonconstructive, and while it tells us of the existence of some languages that are not recursively enumerable, it gives us no feeling at all for what these languages might look like. In the next set of results, we investigate the conclusion more explicitly.

A Language That Is Not Recursively Enumerable

Since every language that can be described in a direct algorithmic fashion can be accepted by a Turing machine and hence is recursively enumerable, the description of a language that is not recursively enumerable must be indirect. Nevertheless, it is possible. The argument involves a variation on the diagonalization theme.

THEOREM 11.3

There exists a recursively enumerable language whose complement is not recursively enumerable.

Proof: Let $\Sigma = \{a\}$, and consider the set of all Turing machines with this input alphabet. By Theorem 10.3, this set is countable, so we can associate an order $M_1, M_2, \dots$ with its elements. For each Turing machine M_i , there is an associated recursively enumerable language $L(M_i)$. Conversely, for each recursively enumerable language on Σ , there is some Turing machine that accepts it.

We now consider a new language L defined as follows. For each $i \geq 1$, the string a^i is in L if and only if $a^i \in L(M_i)$. It is clear that the language L is well defined, since the statement $a^i \in L(M_i)$, and hence $a^i \in L$, must be either true or false. Next, we consider the complement of L ,

$$\overline{L} = \{a^i : a^i \notin L(M_i)\}, \quad (11.1)$$

which is also well defined but, as we will show, is not recursively enumerable.

We will show this by contradiction, starting from the assumption that $\overline{L}$ is recursively enumerable. If this is so, then there must be some Turing machine, say M_k , such that

$$\overline{L} = L(M_k). \quad (11.2)$$

Consider the string a^k . Is it in L or in $\overline{L}$? Suppose that $a^k \in \overline{L}$. By (11.2) this implies that

$$a^k \in L(M_k).$$

But (11.1) now implies that

$$a^k \notin \overline{L}.$$

Alternatively, if we assume that a^k is in L , then $a^k \notin \overline{L}$ and (11.2) implies that

$$a^k \notin L(M_k).$$

But then from (11.1) we get that

$$a^k \in \overline{L}.$$

The contradiction is inescapable, and we must conclude that our assumption that $\overline{L}$ is recursively enumerable is false.

To complete the proof of the theorem as stated, we must still show that L is recursively enumerable. For this we can use the known enumeration procedure for Turing machines. Given a^i , we first find i by counting the

number of a 's. We then use the enumeration procedure for Turing machines to find M_i . Finally, we give its description along with a^i to a universal Turing machine M_u that simulates the action of M on a^i . If a^i is in L , the computation carried out by M_u will eventually halt. The combined effect of this is a Turing machine that accepts every $a^i \in L$. Therefore, by Definition 11.1, L is recursively enumerable. ■

The proof of this theorem explicitly exhibits, through (11.1), a well-defined language that is not recursively enumerable. This is not to say that there is an easy, intuitive interpretation of $\overline{L}$; it would be difficult to exhibit more than a few trivial members of this language. Nevertheless, $\overline{L}$ is properly defined.

A Language That Is Recursively Enumerable but Not Recursive

Next, we show there are some languages that are recursively enumerable but not recursive. Again, we need do so in a rather roundabout way. We begin by establishing a subsidiary result.

THEOREM 11.4

If a language L and its complement $\overline{L}$ are both recursively enumerable, then both languages are recursive. If L is recursive, then $\overline{L}$ is also recursive, and consequently both are recursively enumerable.

Proof: If L and $\overline{L}$ are both recursively enumerable, then there exist Turing machines M and $\widehat{M}$ that serve as enumeration procedures for L and $\overline{L}$, respectively. The first will produce $w_1, w_2, \dots$ in L , the second $\widehat{w}_1, \widehat{w}_2, \dots$ in $\overline{L}$. Suppose now we are given any $w \in \Sigma^+$. We first let M generate w_1 and compare it with w . If they are not the same, we let $\widehat{M}$ generate $\widehat{w}_1$ and compare again. If we need to continue, we next let M generate w_2 , then $\widehat{M}$ generate $\widehat{w}_2$, and so on. Any $w \in \Sigma^+$ will be generated by either M or $\widehat{M}$, so eventually we will get a match. If the matching string is produced by M , w belongs to L , otherwise it is in $\overline{L}$. The process is a membership algorithm for both L and $\overline{L}$, so they are both recursive.

For the converse, assume that L is recursive. Then there exists a membership algorithm for it. But this becomes a membership algorithm for $\overline{L}$ by simply complementing its conclusion. Therefore, $\overline{L}$ is recursive. Since any recursive language is recursively enumerable, the proof is completed. ■

From this, we conclude directly that the family of recursively enumerable languages and the family of recursive languages are not identical. The language L in Theorem 11.3 is in the first but not in the second family.

THEOREM 11.5

There exists a recursively enumerable language that is not recursive; that is, the family of recursive languages is a proper subset of the family of recursively enumerable languages.

Proof: Consider the language L of Theorem 11.3. This language is recursively enumerable, but its complement is not. Therefore, by Theorem 11.4, it is not recursive, giving us the looked-for example. ■

We see from this that there are indeed well-defined languages for which one cannot construct a membership algorithm.

EXERCISES

1. Prove that the set of all real numbers is not countable.
2. Prove that the set of all languages that are not recursively enumerable is not countable.
3. Let L be a finite language. Show that L^+ is recursively enumerable. Suggest an enumeration procedure for L^+ .
4. Let L be a context-free language. Show that L^+ is recursively enumerable and suggest an enumeration procedure for it.
5. Show that if a language is not recursively enumerable, its complement cannot be recursive.
6. Show that the family of recursively enumerable languages is closed under union.
7. Is the family of recursively enumerable languages closed under intersection?
8. Show that the family of recursive languages is closed under union and intersection.
9. Show that the families of recursively enumerable and recursive languages are closed under reversal.
10. Is the family of recursive languages closed under concatenation?
11. Prove that the complement of a context-free language must be recursive.
12. Let L_1 be recursive and L_2 recursively enumerable. Show that $L_2 - L_1$ is necessarily recursively enumerable.
13. Suppose that L is such that there exists a Turing machine that enumerates the elements of L in proper order. Show that this means that L is recursive.
14. If L is recursive, is it necessarily true that L^+ is also recursive?
15. Choose a particular encoding for Turing machines, and with it, find one element of the language $\overline{L}$ in Theorem 11.3.

16. Let S_1 be a countable set, S_2 a set that is not countable, and $S_1 \subset S_2$. Show that S_2 must then contain an infinite number of elements that are not in S_1 .
17. In Exercise 16, show that in fact $S_2 - S_1$ cannot be countable.
18. Why does the argument in Theorem 11.1 fail when S is finite?
19. Show that the set of all irrational numbers is not countable.

11.2 UNRESTRICTED GRAMMARS

To investigate the connection between recursively enumerable languages and grammars, we return to the general definition of a grammar in Chapter 1. In Definition 1.1 the production rules were allowed to take any form, but various restrictions were later made to get specific grammar types. If we take the general form and impose no restrictions, we get unrestricted grammars.

DEFINITION 11.3

A grammar $G = (V, T, S, P)$ is called **unrestricted** if all the productions are of the form

$$u \rightarrow v,$$

where u is in $(V \cup T)^+$ and v is in $(V \cup T)^*$.

In an unrestricted grammar, essentially no conditions are imposed on the productions. Any number of variables and terminals can be on the left or right, and these can occur in any order. There is only one restriction: λ is not allowed as the left side of a production.

As we will see, unrestricted grammars are much more powerful than restricted forms like the regular and context-free grammars we have studied so far. In fact, unrestricted grammars correspond to the largest family of languages we can hope to recognize by mechanical means; that is, unrestricted grammars generate exactly the family of recursively enumerable languages. We show this in two parts; the first is quite straightforward, but the second involves a lengthy construction.

THEOREM 11.6

Any language generated by an unrestricted grammar is recursively enumerable.

Proof: The grammar in effect defines a procedure for enumerating all strings in the language systematically. For example, we can list all w in L such that

$$S \Rightarrow w,$$

that is, w is derived in one step. Since the set of the productions of the grammar is finite, there will be a finite number of such strings. Next, we list all w in L that can be derived in two steps

$$S \Rightarrow x \Rightarrow w,$$

and so on. We can simulate these derivations on a Turing machine and, therefore, have an enumeration procedure for the language. Hence it is recursively enumerable. ■

This part of the correspondence between recursively enumerable languages and unrestricted grammars is not surprising. The grammar generates strings by a well-defined algorithmic process, so the derivations can be done on a Turing machine. To show the converse, we describe how any Turing machine can be mimicked by an unrestricted grammar.

We are given a Turing machine $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ and want to produce a grammar G such that $L(G) = L(M)$. The idea behind the construction is relatively simple, but its implementation becomes notationally cumbersome.

Since the computation of the Turing machine can be described by the sequence of instantaneous descriptions

$$q_0 w \vdash^* x q_f y, \tag{11.3}$$

we will try to arrange it so that the corresponding grammar has the property that

$$q_0 w \Rightarrow^* x q_f y \tag{11.4}$$

if and only if (11.3) holds. This is not hard to do; what is more difficult to see is how to make the connection between (11.4) and what we really want, namely,

$$S \Rightarrow^* w$$

for all w satisfying (11.3). To achieve this, we construct a grammar that, in broad outline, has the following properties:

1. S can derive $q_0 w$ for all $w \in \Sigma^+$.
2. (11.4) is possible if and only if (11.3) holds.

3. When a string xq_fy with $q_f \in F$ is generated, the grammar transforms this string into the original w .

The complete sequence of derivations is then

$$S \xRightarrow{*} q_0w \xRightarrow{*} xq_fy \xRightarrow{*} w. \quad (11.5)$$

The third step in the above derivation is the troublesome one. How can the grammar remember w if it is modified during the second step? We solve this by encoding strings so that the coded version originally has two copies of w . The first is saved, while the second is used in the steps in (11.4). When a final configuration is entered, the grammar erases everything except the saved w .

To produce two copies of w and to handle the state symbol of M (which eventually has to be removed by the grammar), we introduce variables V_{ab} and V_{aib} for all $a \in \Sigma \cup \{\square\}$, $b \in \Gamma$, and all i such that $q_i \in Q$. The variable V_{ab} encodes the two symbols a and b , while V_{aib} encodes a and b as well as the state q_i .

The first step in (11.5) can be achieved (in the encoded form) by

$$S \rightarrow V_{\square\square}S|SV_{\square\square}|T, \quad (11.6)$$

$$T \rightarrow TV_{aa}|V_{a0a}, \quad (11.7)$$

for all $a \in \Sigma$. These productions allow the grammar to generate an encoded version of any string q_0w with an arbitrary number of leading and trailing blanks.

For the second step, for each transition

$$\delta(q_i, c) = (q_j, d, R)$$

of M , we put into the grammar productions

$$V_{aic}V_{pq} \rightarrow V_{ad}V_{pjq}, \quad (11.8)$$

for all $a, p \in \Sigma \cup \{\square\}$, $q \in \Gamma$. For each

$$\delta(q_i, c) = (q_j, d, L)$$

of M , we include in G

$$V_{pq}V_{aic} \rightarrow V_{pjq}V_{ad}, \quad (11.9)$$

for all $a, p \in \Sigma \cup \{\square\}$, $q \in \Gamma$.

If in the second step, M enters a final state, the grammar must then get rid of everything except w , which is saved in the first indices of the V 's. Therefore, for every $q_j \in F$, we include productions

$$V_{ajb} \rightarrow a, \quad (11.10)$$

for all $a \in \Sigma \cup \{\square\}$, $b \in \Gamma$. This creates the first terminal in the string, which then causes a rewriting in the rest by

$$cV_{ab} \rightarrow ca, \quad (11.11)$$

$$V_{abc} \rightarrow ac, \quad (11.12)$$

for all $a, c \in \Sigma \cup \{\square\}$, $b \in \Gamma$. We need one more special production

$$\square \rightarrow \lambda. \quad (11.13)$$

This last production takes care of the case when M moves outside that part of the tape occupied by the input w . To make things work in this case, we must first use (11.6) and (11.7) to generate

$$\square \dots \square q_0 w \square \dots \square,$$

representing all the tape region used. The extraneous blanks are removed at the end by (11.13).

The following example illustrates this complicated construction. Carefully check each step in the example to see what the various productions do and why they are needed.

EXAMPLE 11.1

Let $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ be a Turing machine with

$$\begin{aligned} Q &= \{q_0, q_1\}, \\ \Gamma &= \{a, b, \square\}, \\ \Sigma &= \{a, b\}, \\ F &= \{q_1\}, \end{aligned}$$

and

$$\begin{aligned} \delta(q_0, a) &= (q_0, a, R), \\ \delta(q_0, \square) &= (q_1, \square, L). \end{aligned}$$

This machine accepts $L(aa^*)$.

Consider now the computation

$$q_0 aa \vdash aq_0 a \vdash aaq_0 \square \vdash aq_1 a \square, \quad (11.14)$$

which accepts the string aa . To derive this string with G , we first use rules of the form (11.6) and (11.7) to get the appropriate starting string,

$$S \Rightarrow SV_{\square\square} \Rightarrow TV_{\square\square} \Rightarrow TV_{aa}V_{\square\square} \Rightarrow V_{a0a}V_{aa}V_{\square\square}.$$

The last sentential form is the starting point for the part of the derivation that mimics the computation of the Turing machine. It contains the original input $aa\Box$ in the sequence of first indices and the initial instantaneous description $q_0aa\Box$ in the remaining indices. Next, we apply

$$V_{a0a}V_{aa} \rightarrow V_{aa}V_{a0a}$$

and

$$V_{a0a}V_{\Box\Box} \rightarrow V_{aa}V_{\Box0\Box},$$

which are specific instances of (11.8), and

$$V_{aa}V_{\Box0\Box} \rightarrow V_{a1a}V_{\Box\Box}$$

coming from (11.9). Then the next steps in the derivation are

$$V_{a0a}V_{aa}V_{\Box\Box} \Rightarrow V_{aa}V_{a0a}V_{\Box\Box} \Rightarrow V_{aa}V_{aa}V_{\Box0\Box} \Rightarrow V_{aa}V_{a1a}V_{\Box\Box}.$$

The sequence of first indices remains the same, always remembering the initial input. The sequence of the other indices is

$$0aa\Box, a0a\Box, aa0\Box, a1a\Box,$$

which is equivalent to the sequence of instantaneous descriptions in (11.14).

Finally, (11.10) to (11.13) are used in the last steps

$$V_{aa}V_{a1a}V_{\Box\Box} \Rightarrow V_{aa}aV_{\Box\Box} \Rightarrow V_{aa}a\Box \Rightarrow aa\Box \Rightarrow aa.$$

The construction described in (11.6) to (11.13) is the basis of the proof of the following result.

THEOREM 11.7

For every recursively enumerable language L , there exists an unrestricted grammar G , such that $L = L(G)$.

Proof: The construction described guarantees that

$$x \vdash y,$$

then

$$e(x) \Rightarrow e(y),$$

where $e(x)$ denotes the encoding of a string according to the given convention. By an induction on the number of steps, we can then show that

$$e(q_0w) \xRightarrow{*} e(y)$$

if and only if

$$q_0w \vdash^* y.$$

We also must show that we can generate every possible starting configuration and that w is properly reconstructed if and only if M enters a final configuration. The details, which are not too difficult, are left as an exercise. ■

These two theorems establish what we set out to do. They show that the family of languages associated with unrestricted grammars is identical with the family of recursively enumerable languages.

EXERCISES

1. What language does the unrestricted grammar

$$\begin{aligned} S &\rightarrow S_1B, \\ S_1 &\rightarrow aS_1b, \\ bB &\rightarrow bbbB, \\ aS_1b &\rightarrow aa, \\ B &\rightarrow \lambda \end{aligned}$$

derive?

2. Construct a Turing machine for $L(01(01)^*)$, then find an unrestricted grammar for it using the construction in Theorem 11.7. Give a derivation for 0101 using the resulting grammar.
3. Use the construction in Theorem 11.7 to find unrestricted grammars for the following languages:
 - (a) $L = L(aaa^*bb^*)$.
 - (b) $L = L\{(ab)^*b^*\}$.
 - (c) $L = \{w : n_a(w) = n_b(w)\}$.
4. Consider a variation on grammars in which the starting point for any derivation can be a finite set of strings rather than a single variable. Formalize

$$(c) \quad L = \{a^n b^m c^n d^m : n \geq 1, m \geq 1\}.$$

$$(d) \quad L = \{ww : w \in \{a, b\}^+\}.$$

$$(e) \quad L = \{a^n b^n c^n d^n : n \geq 1\}.$$

3. Find context-sensitive grammars for the following languages:

$$(a) \quad L = \{w : n_a(w) = n_b(w) = n_c(w)\}.$$

$$(b) \quad L = \{w : n_a(w) = n_b(w) < n_c(w)\}.$$

4. Show that the family of context-sensitive languages is closed under union.
5. Show that the family of context-sensitive languages is closed under reversal.
6. For m in Theorem 11.10, give explicit bounds for m as a function of $|w|$ and $|V \cup T|$.
7. Without explicitly constructing it, show that there exists a context-sensitive grammar for the language $L = \{www^R : w, u \in \{a, b\}^+, |w| \geq |u|\}$.

11.4 THE CHOMSKY HIERARCHY

We have now encountered a number of language families, among them the recursively enumerable languages (L_{RE}), the context-sensitive languages (L_{CS}), the context-free languages (L_{CF}), and the regular languages (L_{REG}). One way of exhibiting the relationship between these families is by the **Chomsky hierarchy**. Noam Chomsky, a founder of formal language theory, provided an initial classification into four language types, type 0 to type 3. This original terminology has persisted, and one finds frequent references to it, but the numeric types are actually different names for the language families we have studied. Type 0 languages are those generated by unrestricted grammars, that is, the recursively enumerable languages. Type 1 consists of the context-sensitive languages, type 2 consists of the context-free languages, and type 3 consists of the regular languages. As we have seen, each language family of type i is a proper subset of the family of type $i - 1$. A diagram (Figure 11.3) exhibits the relationship clearly. Figure 11.3 shows the original Chomsky hierarchy. We have also met several other language families that can be fitted into this picture. Including the families of deterministic context-free languages (L_{DCF}) and recursive languages (L_{REC}), we arrive at the extended hierarchy shown in Figure 11.4.

Other language families can be defined and their place in Figure 11.4 studied, although their relationships do not always have the neatly nested structure of Figures 11.3 and 11.4. In some instances, the relationships are not completely understood.

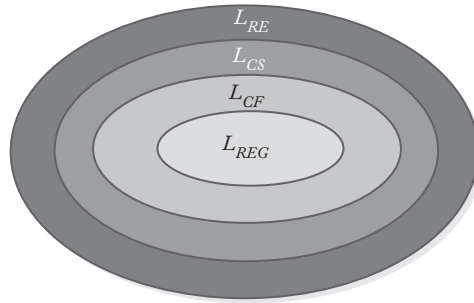


FIGURE 11.3

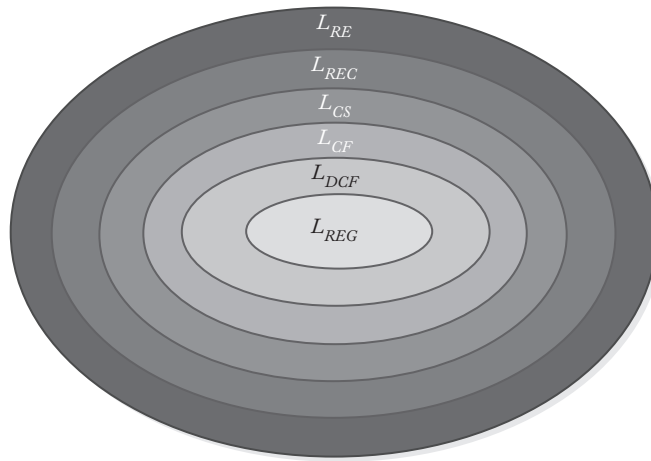


FIGURE 11.4

EXAMPLE 11.3

We have previously introduced the context-free language

$$L = \{w : n_a(w) = n_b(w)\}$$

and shown that it is deterministic, but not linear. On the other hand, the language

$$L = \{a^n b^n\} \cup \{a^n b^{2n}\}$$

is linear, but not deterministic. This indicates that the relationship between regular, linear, deterministic context-free, and nondeterministic context-free languages is as shown in Figure 11.5.

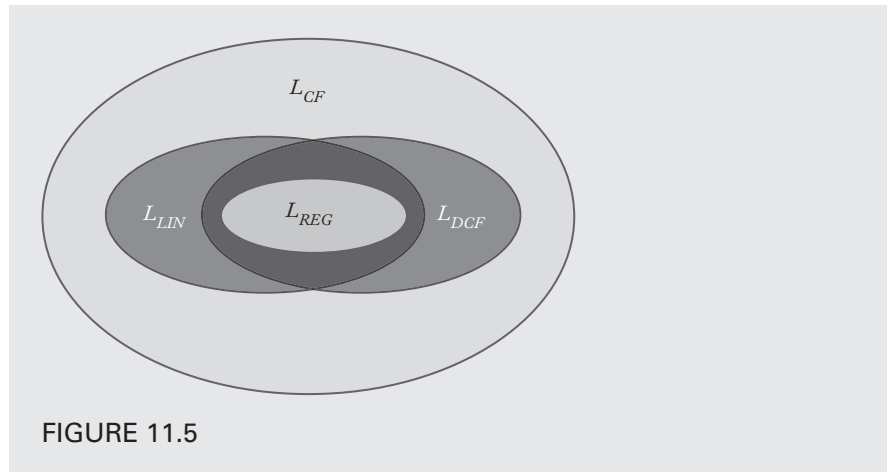


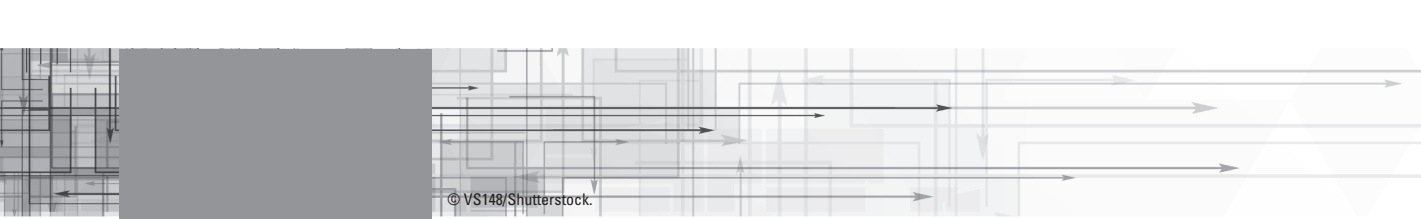
FIGURE 11.5

There is still an unresolved issue. We introduced the concept of a deterministic linear bounded automaton in Exercise 4, Section 10.5. We can now ask the question we asked in connection with other automata: What role does nondeterminism play here? Unfortunately, there is no easy answer. At this time, it is not known whether the family of languages accepted by deterministic linear bounded automata is a proper subset of the context-sensitive languages.

To summarize, we have explored the relationships between several language families and their associated automata. In doing so, we established a hierarchy of languages and classified automata by their power as language accepters. Turing machines are more powerful than linear bounded automata. These in turn are more powerful than pushdown automata. At the bottom of the hierarchy are finite accepters, with which we began our study.

EXERCISES

1. Collect examples given in this book that demonstrate that all the subset relations depicted in Figure 11.4 are indeed proper ones.
2. Find two examples (excluding the one in Example 11.3) of languages that are linear but not deterministic context free.
3. Find two examples (excluding the one in Example 11.3) of languages that are deterministic context free but not linear.



CHAPTER

12

LIMITS OF ALGORITHMIC COMPUTATION

CHAPTER SUMMARY

We have claimed that Turing machines can do anything that computers can do. If we can find a problem that cannot be done by a Turing machine, we have also identified a problem that is not in the power of even the most powerful computer. As we will see, there are many such problems, but instead of dealing with all of them in detail, we find one case to which all the others can be reduced. This is the *halting problem*. While this is a highly abstract issue, several practically important consequences follow.

Having talked about what Turing machines can do, we now look at what they cannot do. Although Turing's thesis leads us to believe that there are few limitations to the power of a Turing machine, we have claimed on several occasions that there could not exist any algorithms for the solution of certain problems. Now we make more explicit what we mean by this claim. Some of the results came about quite simply; if a language is nonrecursive, then by definition there is no membership algorithm for it. If this were all there was to this issue, it would not be very interesting; nonrecursive languages have little practical value. But the problem goes deeper. For example, we have stated (but not yet proved) that there exists no algorithm to determine whether a context-free grammar is

unambiguous. This question is clearly of practical significance in the study of programming languages.

We first define the concepts of **decidability** and **computability** to pin down what we mean when we say that something cannot be done by a Turing machine. We then look at several classical problems of this type, among them the well-known halting problem for Turing machines. From this follow a number of related problems for Turing machines and recursively enumerable languages. After this, we look at some questions relating to context-free languages. Here we find quite a few important problems for which, unfortunately, there are no algorithms.

12.1 SOME PROBLEMS THAT CANNOT BE SOLVED BY TURING MACHINES

The argument that the power of mechanical computations is limited is not surprising. Intuitively we know that many vague and speculative questions require special insight and reasoning well beyond the capacity of any computer that we can now construct or even foresee. What is more interesting to computer scientists is that there are questions that can be clearly and simply stated, with an apparent possibility of an algorithmic solution, but which are known to be unsolvable by any computer.

Computability and Decidability

In Definition 9.4, we stated that a function f on a certain domain is said to be computable if there exists a Turing machine that computes the value of f for all arguments in its domain. A function is uncomputable if no such Turing machine exists. There may be a Turing machine that can compute f on part of its domain, but we call the function computable only if there is a Turing machine that computes the function on the whole of its domain. We see from this that, when we classify a function as computable or not computable, we must be clear on what its domain is.

Our concern here will be the somewhat simplified setting where the result of a computation is a simple “yes” or “no.” In this case, we talk about a problem being **decidable** or **undecidable**. By a *problem* we will understand a set of related statements, each of which must be either true or false. For example, we consider the statement, “For a context-free grammar G , the language $L(G)$ is ambiguous.” For some G this is true, for others it is false, but clearly we must have one or the other. The problem is to decide whether the statement is true for any G we are given. Again, there is an underlying domain, the set of all context-free grammars. We say that a problem is decidable if there exists a Turing machine that gives the correct answer for every statement in the domain of the problem.

When we state decidability or undecidability results, we must always know what the domain is, because this may affect the conclusion. The problem may be decidable on some domain but not on another. Specifically, a single instance of a problem is always decidable, since the answer is either true or false. In the first case, a Turing machine that always answers “true” gives the correct answer, while in the second case one that always answers “false” is appropriate. This may seem like a facetious answer, but it emphasizes an important point. The fact that we do not know what the correct answer is makes no difference; what matters is that there exists some Turing machine that does give the correct response.

The Turing Machine Halting Problem

We begin with some problems that have historical significance and that at the same time give us a starting point for developing later results. The best-known of these is the Turing machine **halting problem**. Simply stated, the problem is: Given the description of a Turing machine M and an input w , does M , when started in the initial configuration q_0w , perform a computation that eventually halts? Using an abbreviated way of talking about the problem, we ask whether M applied to w , or simply (M, w) , halts or does not halt. The domain of this problem is to be taken as the set of all Turing machines and all w ; that is, we are looking for a single Turing machine that, given the description of an arbitrary M and w , will predict whether or not the computation of M applied to w will halt.

We cannot find the answer by simulating the action of M on w , say by performing it on a universal Turing machine, because there is no limit on the length of the computation. If M enters an infinite loop, then no matter how long we wait, we can never be sure that M is in fact in a loop. It may simply be a case of a very long computation. What we need is an algorithm that can determine the correct answer for any M and w by performing some analysis on the machine’s description and the input. But as we now show, no such algorithm exists.

For subsequent discussion, it is convenient to have a precise idea of what we mean by the halting problem; for this reason, we make a specific definition of what we stated somewhat loosely above.

DEFINITION 12.1

Let w_M be a string that describes a Turing machine $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$, and let w be a string in M ’s alphabet. We will assume that w_M and w are encoded as a string of 0’s and 1’s, as suggested in Section 10.4. A solution of the halting problem is a Turing machine H , which for any w_M and w performs the computation

$$q_0w_Mw \vdash^* x_1q_yx_2$$

if M applied to w halts, and

$$q_0 w_M w \vdash^* y_1 q_n y_2$$

if M applied to w does not halt. Here q_y and q_n are both final states of H .

THEOREM 12.1

There does not exist any Turing machine H that behaves as required by Definition 12.1. The halting problem is therefore undecidable.

Proof: We assume the contrary, namely, that there exists an algorithm, and consequently some Turing machine H , that solves the halting problem. The input to H will be the string $w_M w$. The requirement is then that, given any $w_M w$, the Turing machine H will halt with either a yes or no answer. We achieve this by asking that H halt in one of two corresponding final states, say, q_y or q_n . The situation can be visualized by a block diagram like Figure 12.1. The intent of this diagram is to indicate that, if H is started in state q_0 with input $w_M w$, it will eventually halt in state q_y or q_n . As required by Definition 12.1, we want H to operate according to the following rules:

$$q_0 w_M w \vdash_H^* x_1 q_y x_2$$

if M applied to w halts, and

$$q_0 w_M w \vdash_H^* y_1 q_n y_2$$

if M applied to w does not halt.

Next, we modify H to produce a Turing machine H' with the structure shown in Figure 12.2. With the added states in Figure 12.2 we want to

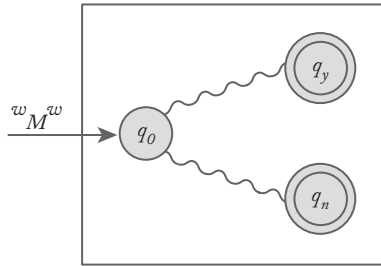


FIGURE 12.1

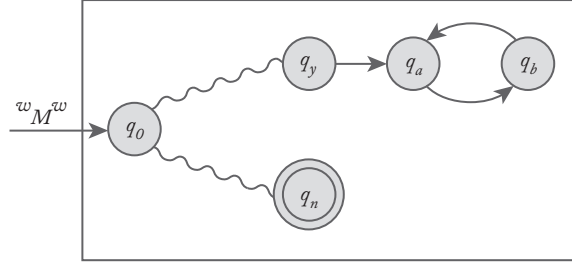


FIGURE 12.2

convey that the transitions between state q_y and the new states q_a and q_b are to be made, regardless of the tape symbol, in such a way that the tape remains unchanged. The way this is done is straightforward. Comparing H and H' we see that, in situations where H reaches q_y and halts, the modified machine H' will enter an infinite loop. Formally, the action of H' is described by

$$q_0 w_M w \vdash_{H'}^* \infty$$

if M applied to w halts and

$$q_0 w_M w \vdash_{H'}^* y_1 q_n y_2$$

if M applied to w does not halt.

From H' we construct another Turing machine $\hat{H}$. This new machine takes as input w_M and copies it, ending in its initial state q_0 . After that, it behaves exactly like H' . Then the action of $\hat{H}$ is such that

$$q_0 w_M \vdash_{\hat{H}}^* q_0 w_M w_M \vdash_{\hat{H}}^* \infty$$

if M applied to w_M halts and

$$q_0 w_M \vdash_{\hat{H}}^* q_0 w_M w_M \vdash_{\hat{H}}^* y_1 q_n y_2$$

if M applied to w_M does not halt.

Now $\hat{H}$ is a Turing machine, so it has a description in $\{0, 1\}^*$, say, $\hat{w}$. This string, in addition to being the description of $\hat{H}$, also can be used as input string. We can therefore legitimately ask what would happen if $\hat{H}$ is applied to $\hat{w}$. From the above, identifying M with $\hat{H}$, we get

$$q_0 \hat{w} \vdash_{\hat{H}}^* \infty$$

if $\hat{H}$ applied to $\hat{w}$ halts and

$$q_0 \hat{w} \vdash_{\hat{H}}^* y_1 q_n y_2$$

if $\widehat{H}$ applied to $\widehat{w}$ does not halt. This is clearly nonsense. The contradiction tells us that our assumption of the existence of H , and hence the assumption of the decidability of the halting problem, must be false. ■

One may object to Definition 12.1, since we required that, to solve the halting problem, H had to start and end in very specific configurations. It is, however, not hard to see that these somewhat arbitrarily chosen conditions play only a minor role in the argument and that essentially the same reasoning could be used with any other starting and ending configurations. We have tied the problem to a specific definition for the sake of the discussion, but this does not affect the conclusion.

It is important to keep in mind what Theorem 12.1 says. It does not preclude solving the halting problem for specific cases; often we can tell by an analysis of M and w whether or not the Turing machine will halt. What the theorem says is that this cannot always be done; there is no algorithm that can make a correct decision for all w_M and w .

The arguments for proving Theorem 12.1 were given because they are classical and of historical interest. The conclusion of the theorem is actually implied in previous results as the following argument shows.

THEOREM 12.2

If the halting problem were decidable, then every recursively enumerable language would be recursive. Consequently, the halting problem is undecidable.

Proof: To see this, let L be a recursively enumerable language on Σ , and let M be a Turing machine that accepts L . Let H be the Turing machine that solves the halting problem. We construct from this the following procedure:

1. Apply H to $w_M w$. If H says “no,” then by definition w is not in L .
2. If H says “yes,” then apply M to w . But M must halt, so it will eventually tell us whether w is in L or not.

This constitutes a membership algorithm, making L recursive. But we already know that there are recursively enumerable languages that are not recursive. The contradiction implies that H cannot exist, that is, that the halting problem is undecidable. ■

The simplicity with which the halting problem can be obtained from Theorem 11.5 is a consequence of the fact that the halting problem and the membership problem for recursively enumerable languages are nearly identical. The only difference is that in the halting problem we do not

distinguish between halting in a final and nonfinal state, whereas in the membership problem we do. The proofs of Theorems 11.5 (via Theorem 11.3) and 12.1 are closely related, both being a version of diagonalization.

Reducing One Undecidable Problem to Another

The above argument, connecting the halting problem to the membership problem, illustrates the very important technique of reduction. We say that a problem A is **reduced** to a problem B if the decidability of A follows from the decidability of B . Then, if we know that A is undecidable, we can conclude that B is also undecidable. Let us do a few examples to illustrate this idea.

EXAMPLE 12.1

The **state-entry problem** is as follows. Given any Turing machine $M = (Q, \Sigma, \Gamma, \delta, q_0, \square, F)$ and any $q \in Q$, $w \in \Sigma^+$, decide whether or not the state q is ever entered when M is applied to w . This problem is undecidable.

To reduce the halting problem to the state-entry problem, suppose that we have an algorithm A that solves the state-entry problem. We could then use it to solve the halting problem. For example, given any M and w , we first modify M to get $\hat{M}$ in such a way that $\hat{M}$ halts in state q if and only if M halts. We can do this simply by looking at the transition function δ of M . If M halts, it does so because some $\delta(q_i, a)$ is undefined. To get $\hat{M}$, we change every such undefined δ to

$$\delta(q_i, a) = (q, a, R),$$

where q is a final state. We apply the state-entry algorithm A to $(\hat{M}, q, w)$. If A answers yes, that is, the state q is entered, then (M, w) halts. If A says no, then (M, w) does not halt.

Thus, the assumption that the state-entry problem is decidable gives us an algorithm for the halting problem. Because the halting problem is undecidable, the state-entry problem must also be undecidable.

EXAMPLE 12.2

The **blank-tape halting problem** is another problem to which the halting problem can be reduced. Given a Turing machine M ,

determine whether or not M halts if started with a blank tape. This is undecidable.

To show how this reduction is accomplished, assume that we are given some M and some w . We first construct from M a new machine M_w that starts with a blank tape, writes w on it, then positions itself in a configuration q_0w . After that, M_w acts like M . Clearly M_w will halt on a blank tape if and only if M halts on w .

Suppose now that the blank-tape halting problem were decidable. Given any (M, w) , we first construct M_w , then apply the blank-tape halting problem algorithm to it. The conclusion tells us whether M applied to w will halt. Since this can be done for any M and w , an algorithm for the blank-tape halting problem can be converted into an algorithm for the halting problem. Since the latter is known to be undecidable, the same must be true for the blank-tape halting problem.

The construction in the arguments of these two examples illustrates an approach common in establishing undecidability results. A block diagram often helps us visualize the process. The construction in Example 12.2 is summarized in Figure 12.3. In that diagram, we first use an algorithm that transforms (M, w) into M_w ; such an algorithm clearly exists. Next, we use the algorithm for solving the blank-tape halting problem, which we assume exists. Putting the two together yields an algorithm for the halting problem. But this is impossible, and we can conclude that A cannot exist.

A decision problem is effectively a function with a range $\{0, 1\}$, that is, a true or false answer. We can look also at more general functions to see if they are computable; to do so, we follow the established method and reduce the halting problem (or any other problem known to be undecidable) to the problem of computing the function in question. Because of Turing's thesis, we expect that functions encountered in practical circumstances will be computable, so for examples of uncomputable functions we must look a little further. Most examples of uncomputable functions are associated with attempts to predict the behavior of Turing machines.

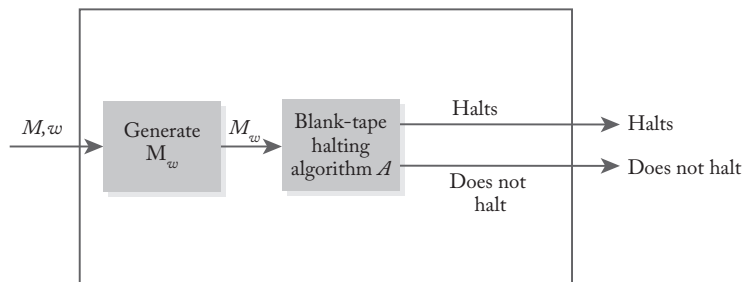


FIGURE 12.3 Algorithm for the halting problem.

EXAMPLE 12.3

Let $\Gamma = \{0, 1, \square\}$. Consider the function $f(n)$ whose value is the maximum number of moves that can be made by any n -state Turing machine that halts when started with a blank tape. This function, as it turns out, is not computable.

Before we set out to demonstrate this, let us make sure that $f(n)$ is defined for all n . Notice first that there are only a finite number of Turing machines with n states. This is because Q and Γ are finite, so δ has a finite domain and range. This in turn implies that there are only a finite number of different δ 's and therefore a finite number of different n -state Turing machines.

Of all of the n -state machines, there are some that always halt, for example machines that have only final states and therefore make no moves. Some of the n -state machines will not halt when started with a blank tape, but they do not enter the definition of f . Every machine that does halt will execute a certain number of moves; of these, we take the largest to give $f(n)$.

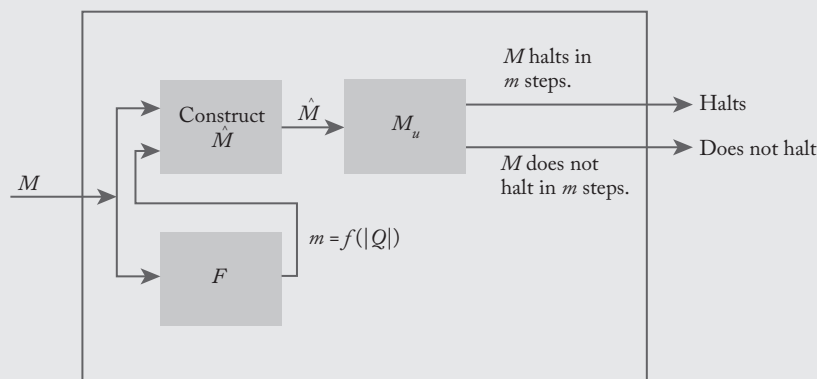


FIGURE 12.4 Algorithm for the blank-tape halting problem.

Take any Turing machine M and positive number m . It is easy to modify M to produce $\hat{M}$ in such a way that the latter will always halt with one of two answers: M applied to a blank tape halts in no more than m moves, or M applied to a blank tape makes more than m moves. All we have to do for this is to have M count its moves and terminate when this count exceeds m . Assume now that $f(n)$ is computable by some Turing machine F . We can then put $\hat{M}$ and F together as shown in Figure 12.4. First we compute $f(|Q|)$, where Q is the state set of M . This tells us the maximum number of moves that M can make if it is to halt. The value we get is then used as m to construct $\hat{M}$ as

outlined, and a description of $\hat{M}$ is given to a universal Turing machine for execution. This tells us whether M applied to a blank tape halts or does not halt in less than $f(|Q|)$ steps. If we find that M applied to a blank tape makes more than $f(|Q|)$ moves, then because of the definition of f , the implication is that M never halts. Thus we have a solution to the blank-tape halting problem. The impossibility of the conclusion forces us to accept that f is not computable.

EXERCISES

1. Describe in detail how H in Theorem 12.1 can be modified to produce H' .
2. Show that the question “Does a Turing machine accept all even-length strings?” is undecidable.
3. Show that the following problem is undecidable. Given any Turing machine M , $a \in \Gamma$, and $w \in \Sigma^+$, determine whether or not the symbol a is ever written when M is applied to w .
4. In the general halting problem, we ask for an algorithm that gives the correct answer for any M and w . We can relax this generality, for example, by looking for an algorithm that works for all M but only a single w . We say that such a problem is decidable if for every w there exists a (possibly different) algorithm that determines whether or not (M, w) halts. Show that even in this restricted setting the problem is undecidable.
5. Show that there is no algorithm to decide whether or not an arbitrary Turing machine halts on all input.
6. Consider the question: “Does a Turing machine in the course of a computation revisit the starting cell (i.e., the cell under the read-write head at the beginning of the computation)?” Is this a decidable question?
7. Show that there is no algorithm for deciding if any two Turing machines M_1 and M_2 accept the same language.
8. How is the conclusion of Exercise 7 affected if M_2 is a finite automaton?
9. Is the halting problem solvable for deterministic pushdown automata; that is, given a pda as in Definition 7.3, can we always predict whether or not the automaton will halt on input w ?
10. Let M be any Turing machine and x and y two possible instantaneous descriptions of it. Show that the problem of determining whether or not

$$x \vdash_M^* y$$

is undecidable.

11. In Example 12.3, give the values of $f(1)$ and $f(2)$.
12. Show that the problem of determining whether a Turing machine halts on any input is undecidable.
13. Let B be the set of all Turing machines that halt when started with a blank tape. Show that this set is recursively enumerable, but not recursive.
14. Consider the set of all n -state Turing machines with tape alphabet $\Gamma = \{0, 1, \square\}$. Give an expression for $m(n)$, the number of distinct Turing machines with this Γ .
15. Let $\Gamma = \{0, 1, \square\}$ and let $b(n)$ be the maximum number of tape cells examined by any n -state Turing machine that halts when started with a blank tape. Show that $b(n)$ is not computable.
16. Determine whether or not the following statement is true: Any problem whose domain is finite is decidable.

12.2 UNDECIDABLE PROBLEMS FOR RECURSIVELY ENUMERABLE LANGUAGES

We have determined that there is no membership algorithm for recursively enumerable languages. The lack of an algorithm to decide on some property is not an exceptional state of affairs for recursively enumerable languages, but rather is the general rule. As we now show, there is little we can say about these languages. Recursively enumerable languages are so general that, in essence, any question we ask about them is undecidable. Invariably, when we ask a question about recursively enumerable languages, we find that there is some way of reducing the halting problem to this question. We give here some examples to show how this is done and from these examples derive an indication of the general situation.

THEOREM 12.3

Let G be an unrestricted grammar. Then the problem of determining whether or not

$$L(G) = \emptyset$$

is undecidable.

Proof: We will reduce the membership problem for recursively enumerable languages to this problem. Suppose we are given a Turing machine M and some string w . We can modify M as follows. M first saves its input on some special part of its tape. Then, whenever it enters a final state, it checks its

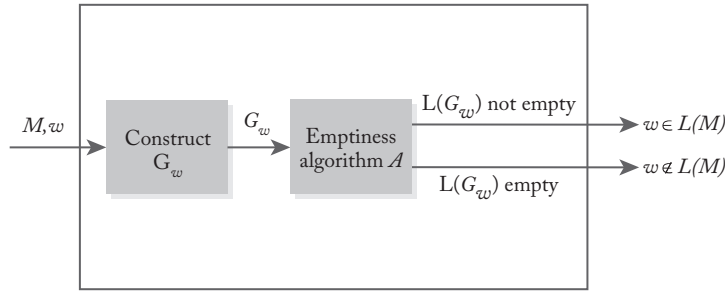


FIGURE 12.5 Membership algorithm.

saved input and accepts it if and only if it is w . We can do this by changing δ in a simple way, creating for each w a machine M_w such that

$$L(M_w) = L(M) \cap \{w\}.$$

Using Theorem 11.7, we then construct a corresponding grammar G_w . Clearly, the construction leading from M and w to G_w can always be done. Equally clear is that $L(G_w)$ is nonempty if and only if $w \in L(M)$.

Assume now that there exists an algorithm A for deciding whether or not $L(G) = \emptyset$. If we let T denote an algorithm by which we generate G_w , then we can put T and A together as shown in Figure 12.5. Figure 12.5 is a Turing machine that for any M and w tells us whether or not w is in $L(M)$. If such a Turing machine existed, we would have a membership algorithm for any recursively enumerable language, in direct contradiction to a previously established result. We conclude therefore that the stated problem “ $L(G) = \emptyset$ ” is not decidable. ■

THEOREM 12.4

Let M be any Turing machine. Then the question of whether or not $L(M)$ is finite is undecidable.

Proof: Consider the halting problem (M, w) . From M we construct another Turing machine $\hat{M}$ that does the following. First, the halting states of M are changed so that if any one is reached, all input is accepted by $\hat{M}$. This can be done by having any halting configuration go to a final state. Second, the original machine is modified so that the new machine $\hat{M}$ first generates w on its tape, then performs the same computations as M , using the newly created w and some otherwise unused space. In other words, the moves made by $\hat{M}$ after it has written w on its tape are the same as would have been made by M had it started in the original configuration q_0w . If M halts in any configuration, then $\hat{M}$ will halt in a final state.

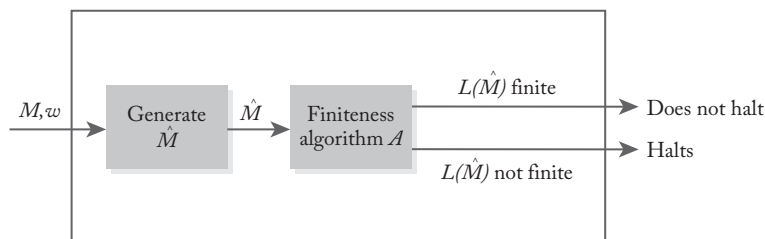


FIGURE 12.6

Therefore, if (M, w) halts, $\hat{M}$ will reach a final state for all input. If (M, w) does not halt, then $\hat{M}$ will not halt either and so will accept nothing. In other words, $\hat{M}$ accepts either the infinite language Σ^+ or the finite language $\emptyset$.

If we now assume the existence of an algorithm A that tells us whether or not $L(\hat{M})$ is finite, we can construct a solution to the halting problem as shown in Figure 12.6. Therefore, no algorithm for deciding whether or not $L(M)$ is finite can exist. ■

Notice that in the proof of Theorem 12.4, the specific nature of the question asked, namely “Is $L(M)$ finite?” is immaterial. We can change the nature of the problem without significantly affecting the argument.

EXAMPLE 12.4

Show that for an arbitrary Turing machine M with $\Sigma = \{a, b\}$, the problem “ $L(M)$ contains two different strings of the same length” is undecidable.

To show this, we use exactly the same approach as in Theorem 12.4, except that when $\hat{M}$ reaches a halting configuration, it will be modified to accept the two strings a and b . For this, the initial input is saved and at the end of the computation compared with a and b , accepting only these two strings. Thus, if (M, w) halts, $\hat{M}$ will accept two strings of equal length, otherwise $\hat{M}$ will accept nothing. The rest of the argument then proceeds as in Theorem 12.4.

In exactly the same manner, we can substitute other questions such as “Does $L(M)$ contain any string of length five?” or “Is $L(M)$ regular?” without affecting the argument essentially. These questions, as well as similar questions, are all undecidable. A general result formalizing this is known as **Rice’s theorem**. This theorem states that any nontrivial property of a

recursively enumerable language is undecidable. The adjective “nontrivial” refers to a property possessed by some but not all recursively enumerable languages. A precise statement and a proof of Rice’s theorem can be found in Hopcroft and Ullman (1979).

EXERCISES

1. Show in detail how the machine $\widehat{M}$ in Theorem 12.4 is constructed.
2. Show that the question “Does a Turing machine accept any even-length strings?” is undecidable.
3. Show that the two problems mentioned at the end of the preceding section, namely
 - (a) $L(M)$ contains any string of length five and
 - (b) $L(M)$ is regular,
 are undecidable.
4. Let M_1 and M_2 be arbitrary Turing machines. Show that the problem “ $L(M_1) \subseteq L(M_2)$ ” is undecidable.
5. Let G be any unrestricted grammar. Does there exist an algorithm for determining whether or not $L(G)^R$ is recursively enumerable?
6. Let G be any unrestricted grammar. Does there exist an algorithm for determining whether or not $L(G) = L(G)^R$?
7. Let G_1 be any unrestricted grammar, and G_2 be any regular grammar. Show that the problem

$$L(G_1) \cap L(G_2) = \emptyset$$
 is undecidable.
8. Let G_1 and G_2 be unrestricted grammars on Σ . Show that the problem

$$L(G_1) \cup L(G_2) = \Sigma^*$$
 is undecidable.
9. Show that the question in Exercise 6 is undecidable for any fixed G_2 , as long as $L(G_2)$ is not empty.
10. For an unrestricted grammar G , show that the question “Is $L(G) = L(G)^*$?” is undecidable. Argue (a) from Rice’s theorem and (b) from first principles.

productions are applicable at each step. Following the analysis leading to Equation (5.2), we see that the maximum number of sentential forms is

$$N = |P| + |P|^2 + \dots |P|^{cn} = O(|P|^{cn+1}).$$

Note, however, that we cannot claim from this that

$$L_{CS} \subseteq DTIME(|P|^{cn+1})$$

because we cannot put an upper bound on $|P|$ and c .

From these examples we note a trend: As $T(n)$ increases, more and more of the families L_{REG} , L_{CF} , L_{CS} are covered. But the connection between the Chomsky hierarchy and the complexity classes is tenuous and not very clear.

EXERCISES

1. Complete the argument in Example 14.4.
2. Show that $L = \{ww^Rw : w \in \{a, b\}^+\}$ is in $DTIME(n)$.
3. Show that $L = \{www : w \in \{a, b\}^+\}$ is in $DTIME(n)$.
4. Show that there are languages that are not in $NTIME(2^n)$.

14.4 THE COMPLEXITY CLASSES P AND NP

At this point, it is instructive to summarize the difficulties we have encountered in trying to find useful complexity classes for formal languages and draw a few conclusions.

1. There exists an infinite number of properly nested complexity classes $DTIME(n^k)$, $k = 1, 2, \dots$. These complexity classes have little connection to the familiar Chomsky hierarchy, and it seems difficult to get any insight into the nature of these classes. Perhaps this is not a good way of classifying languages.
2. The particular model of Turing machine, even if we restrict ourselves to deterministic machines, affects the complexity. It is not clear what kind of Turing machine is the best model of an actual computer, so an analysis should not depend on any particular type of Turing machine.

3. We have found several languages that can be decided efficiently by a nondeterministic Turing machine. For some, there are also reasonable deterministic algorithms, but for others we know only inefficient, brute-force methods. What is the implication of these examples?

Since the attempt to produce meaningful language hierarchies via time complexities with different growth rates appears to be unproductive, let us ignore some factors that are less important, for example by removing some uninteresting distinctions, such as that between $DTIME(n^k)$ and $DTIME(n^{k+1})$. We can argue that the difference between, say, $DTIME(n)$ and $DTIME(n^2)$ is not fundamental, since some of it depends on the specific model of Turing machine we have (e.g., how many tapes). This leads us to consider the famous complexity class

$$\mathbf{P} = \bigcup_{i \geq 1} DTIME(n^i).$$

This class includes all languages that are accepted by some deterministic Turing machine in polynomial time, without any regard to the degree of the polynomial. As we have already seen, L_{REG} and L_{CF} are in $\mathbf{P}$.

Since the distinction between deterministic and nondeterministic complexity classes appears to be fundamental, we also introduce

$$\mathbf{NP} = \bigcup_{i \geq 1} NTIME(n^i).$$

Obviously

$$\mathbf{P} \subseteq \mathbf{NP},$$

but what is not known is if this containment is proper. While it is generally believed that there are some languages in $\mathbf{NP}$ that are not in $\mathbf{P}$, no one has yet found an example of this.

The interest in these complexity classes, particularly in the class $\mathbf{P}$, comes from an attempt to distinguish between realistic and unrealistic computations. Certain computations, although theoretically possible, have such high resource requirements that in practice they must be rejected as unrealistic on existing computers, as well as on supercomputers yet to be designed. Such problems are sometimes called **intractable** to indicate that, while in principle computable, there is no realistic hope of a practical algorithm. To understand this better, computer scientists have attempted to put the idea of intractability on a formal basis. One attempt to define the term intractable is made in what is generally called the **Cook-Karp thesis**. In the Cook-Karp thesis, a problem that is in $\mathbf{P}$ is called tractable, and one that is not is said to be intractable.

Is the Cook-Karp thesis a good way of separating problems we can solve realistically from those we cannot? The answer is not clear-cut. Obviously, any computation that is not in $\mathbf{P}$ has time complexity that grows faster than

any polynomial, and its requirements will increase very quickly with the problem size. Even for a function like $2^{0.1n}$, this will be excessive for large n , say $n \geq 1000$, so we might feel justified in calling a problem with this complexity intractable. But what about problems that are in $DTIME(n^{100})$? While the Cook-Karp thesis calls such a problem tractable, one surely cannot do much with it, even for small n . The justification for the Cook-Karp thesis seems to lie in the empirical observation that most practical problems in $\mathbf{P}$ are in $DTIME(n)$, $DTIME(n^2)$, or $DTIME(n^3)$, while those outside this class tend to have exponential complexities. Among practical problems, a clear distinction exists between problems in $\mathbf{P}$ and those not in $\mathbf{P}$.

14.5 SOME NP PROBLEMS

Computer scientists have studied many **NP** problems, that is, problems that can be solved nondeterministically in polynomial time. Some of the arguments involved in this are very technical, with a number of details that have to be resolved.

Traditionally, complexity questions are studied as languages, in such a way that the cases that satisfy the stated conditions are described by strings in some language L , while those that do not are in $\bar{L}$. So, often the first thing that needs to be done is to rephrase our intuitive understanding of the problem in terms of a language.

EXAMPLE 14.6

Reconsider the SAT problem. We made some rudimentary argument to claim that this problem can be solved efficiently by a nondeterministic Turing machine and, rather inefficiently, by a brute-force exponential search. A number of minor points were ignored in that argument.

Suppose that a CNF expression has length n , with m different literals. Since clearly $m < n$, we can take n as the problem size. Next, we must encode the CNF expression as a string for a Turing machine. We can do this, for example, by taking $\Sigma = \{x, \vee, \wedge, (,), -, 0, 1\}$ and encoding the subscript of x as a binary number. In this system, the CNF expression $(x_1 \vee \bar{x}_2) \wedge (x_3 \vee x_4)$ is encoded as

$$(x1 \vee x - 10) \wedge (x11 \vee x100).$$

Since the subscript cannot be larger than m , the maximum length of any subscript is $\log_2 m$. As a consequence the maximum encoded length of an n -symbol CNF is $O(n \log n)$.

The next step is to generate a trial solution for the variables. Nondeterministically, this can be done in $O(n)$ time. (See Exercise 1 at the end of this section.) This trial solution is then substituted into the input string. This can be done in $O(n^2 \log n)$ time*. The entire process therefore can be done in $O(n^2 \log n)$ or $O(n^3)$ time, and $SAT \in \mathbf{NP}$.

There are a large number of graph problems that have been studied and are known to be in $\mathbf{NP}$.

EXAMPLE 14.7

The Hamiltonian Path Problem Given an undirected graph, with vertices $v_1, v_2, \dots, v_n$, a Hamiltonian path is a simple path that passes through all the vertices. The graph in Figure 14.2 has a Hamiltonian path $(v_2, v_1), (v_1, v_3), (v_3, v_5), (v_5, v_4), (v_4, v_6)$. The Hamiltonian path problem (HAMPATH) is to decide if a given graph has a Hamiltonian path.

A deterministic algorithm is easily found, since any Hamiltonian path is a permutation of the vertices $v_1, v_2, \dots, v_n$. There are $n!$ such permutations, and a brute-force search of all of them will give the answer. Unfortunately, this comes at a great expense, even for modest n .

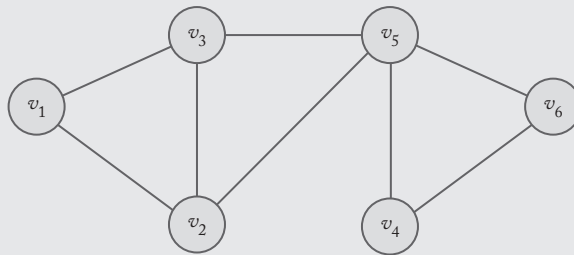


FIGURE 14.2

To explore the nondeterministic solution, we must first find a way to represent a graph by a string. One of the simplest and most convenient ways of encoding graphs is by an *adjacency matrix*. For a directed graph with vertices $v_1, v_2, \dots, v_n$ and edge set E , an adjacency matrix is an $n \times n$ array in which $a(i, j)$, the entry in the i th row and j th column satisfies

$$\begin{aligned} a(i, j) &= 1 \text{ if } (v_i, v_j) \in E, \\ &= 0 \text{ if } (v_i, v_j) \notin E. \end{aligned}$$

An undirected edge can be considered two separate edges in opposite directions. The array

```

0 1 1 0 0 0
1 0 1 0 1 0
1 1 0 0 1 0
0 0 0 0 1 1
0 1 1 1 0 1
0 0 0 1 1 0

```

represents the graph in Figure 14.2.

A graph with n vertices then requires a string of length n^2 for its representation. For an undirected graph the matrix is symmetric, so the storage requirement can be reduced to $(n+1)n/2$, but in any case, the input string will have length $O(n^2)$.

Next, we generate, nondeterministically, a permutation of the vertices. This can be done in $O(n^3)$ time. Finally, we check the permutation to see if it constitutes a path. A time $O(n^4)$ is sufficient for this. Therefore, $HAMPATH \in \mathbf{NP}$.

EXAMPLE 14.8

The Clique Problem Let G be an undirected graph with vertices $v_1, v_2, \dots, v_n$. A k -clique is a subset $V_k \subseteq 2^V$, such that there is an edge between every pair of vertices $v_i, v_j \in V_k$. The clique problem (CLIQ) is to decide if, for a given k , G has a k -clique.

A deterministic search can examine all the elements of 2^V . This is straightforward, but it has exponential time complexity. A nondeterministic algorithm just guesses the correct subset. The representation of the graph and the checking are similar to the previous example, so we claim without further elaboration that the clique problem can be solved in $O(n^4)$ time and that

$$CLIQ \in \mathbf{NP}.$$

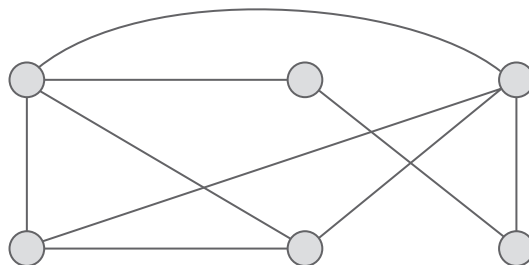
There are many other such problems, some similar to our examples, others quite different, but all sharing the same characteristics.

1. All problems are in $\mathbf{NP}$ and have simple nondeterministic solutions.
2. All problems have deterministic solutions with exponential time complexity, but it is not known if they are tractable.

To get further insight into the connection between these various problems, we need to find some commonality for all these seemingly different cases.

EXERCISES

1. In Example 14.6, show how a trial solution can be generated in $O(n)$ time. This means that all 2^n possibilities must be generated in a decision tree with height $O(n)$.
2. Show how in Example 14.6 the checking of the trial solution can be done in $O(n^2 \log n)$ time.
3. Discuss how in HAMPATH a permutation can be generated nondeterministically in $O(n^4)$ time.
4. In HAMPATH, how can the checking for a Hamiltonian path be done in $O(n^4)$ time?
5. Show that a k -clique must have exactly $k(k-1)/2$ edges.
6. Find a 4-clique in the graph below. Prove that the graph has no 5-clique.



7. Give the details of the argument in Example 14.8.
8. Show that $\mathbf{P}$ is closed under union, intersection, and complementation.

14.6 POLYNOMIAL-TIME REDUCTION

One way to unify different cases is to see if we can reduce them to each other, in the sense that if one is tractable, the others will be tractable also.

DEFINITION 14.3

A language L_1 is said to be **polynomial-time reducible** to another language L_2 if there exists a deterministic Turing machine by which any w_1 in

the alphabet of L_1 can be transformed in polynomial time to a w_2 in the alphabet of L_2 in such a way that $w_1 \in L_1$ if and only if $w_2 \in L_2$.

EXAMPLE 14.9

In the satisfiability problem we put no restriction on the length of a clause. A restricted type of satisfiability is the *three-satisfiability* problem (3SAT) in which each clause can have at most three literals. The SAT problem is polynomial-time reducible to 3SAT.

We illustrate the reduction with the simple 4-literal expression

$$e_1 = (x_1 \vee x_2 \vee x_3 \vee x_4).$$

We introduce a new variable z and construct

$$e_2 = (x_1 \vee x_2 \vee z) \wedge (x_3 \vee x_4 \vee \bar{z}).$$

If e_1 is true, one of the x_1, x_2, x_3, x_4 must be true. If $x_1 \vee x_2$ is true, we choose $z = 0$, and e_2 is true. If $x_3 \vee x_4 = 1$, we can choose $z = 1$ to satisfy e_2 . Conversely, if e_2 is true, e_1 must also be true, so for satisfiability, e_1 and e_2 are equivalent.

We claim that this pattern can be extended to clauses with more than four literals, but we will leave the argument as an exercise. We include an exercise to show that the conversion of a CNF expression from the SAT form to 3SAT form can be done deterministically in polynomial time.

EXAMPLE 14.10

The 3SAT problem is polynomial-time reducible to CLIQUE.

We can assume that in any 3SAT expression each clause has exactly three literals. If in some expression this is not the case, we just add extra literals that do not change the satisfiability. For example, $(x_1 \vee x_2)$ is equivalent to $(x_1 \vee x_1 \vee x_2)$. Consider now the expression

$$e = (x_1 \vee \bar{x}_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee x_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_3).$$

We draw a graph in which each clause is represented by a group of three vertices and each literal is associated with one of the vertices (Figure 14.3).

For each vertex in a group, we put in an edge to all vertices of the other groups, unless the two associated literals are complements. So in

Figure 14.3 we draw an edge* from $(\bar{x}_2)_1$ to $(x_3)_2$, and from $(\bar{x}_2)_1$ to $(x_3)_3$, but not from $(\bar{x}_2)_1$ to $(x_2)_2$. Figure 14.3 shows a subset of the edges (the full set looks very messy). Notice that the subgraph with vertices $(\bar{x}_2)_1, (x_3)_2, (x_3)_3$, and $(x_1)_4$ is a 4-clique and that

$$\bar{x}_2 = x_3 = x_1 = 1$$

is a variable assignment that satisfies e .

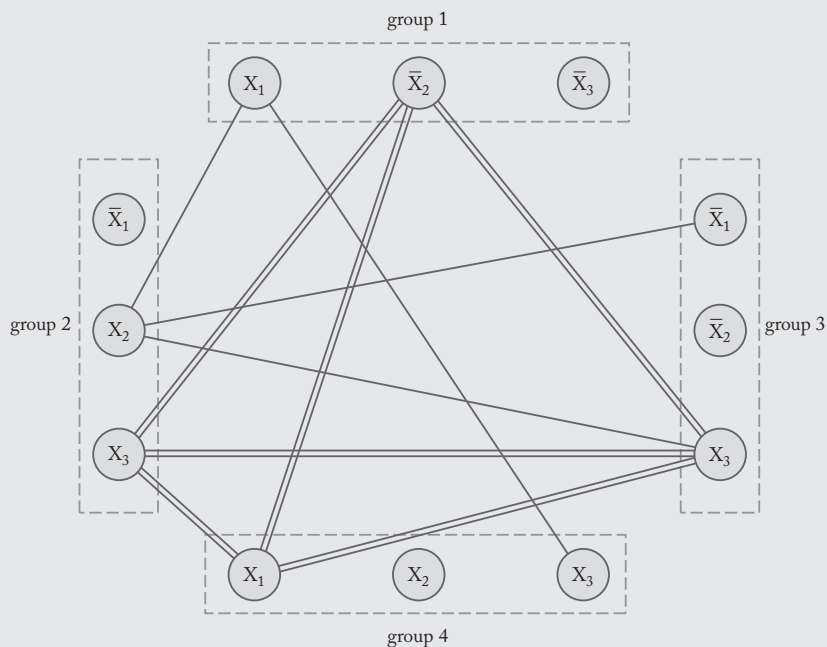


FIGURE 14.3

This approach can be generalized for any 3SAT expression with k clauses. It can be shown that the 3SAT problem can be satisfied if and only if the associated graph has a k -clique. Furthermore, it is not hard to see that the transformation from the 3SAT expression to the graph can be done deterministically in polynomial time.

The point of these reductions is that we can now look at a given problem in several ways. Suppose we conjecture that SAT is tractable. If this is difficult to prove, we might try the simpler 3SAT case. If this does not

*The second subscripts identify the group to which the vertices belong.

work either, we can try to find an efficient algorithm for the clique problem or some other related graph problem. If any of the options can be shown to be tractable, we can claim that SAT is tractable.

EXERCISES

1. Show how a CNF expression with clauses of five literals can be reduced to the 3SAT form.
Generalize your method for clauses with an arbitrary number of literals.
2. Show how the reduction of SAT to 3SAT can be done in polynomial time.
3. Justify the statement in Example 14.10 that a 3SAT expression with k clauses is satisfiable if and only if the associated graph has a k -clique.
4. Show that the construction of the graph associated with a 3SAT expression can be done deterministically in polynomial time.
5. The Traveling Salesman Problem (TSP) can be stated as follows. Let G be a complete graph, whose edges are assigned nonnegative weights. The weight of a simple path is the sum of all the edge weights. The TSP problem is to decide if, for any given $k \geq 0$, the graph G has a Hamiltonian path with a weight less than or equal to k .

Show that HAMPATH is polynomial-time reducible to TSP.

14.7 NP-COMPLETENESS AND AN OPEN QUESTION

There are a number of problems that are central to complexity study and are such that, if we completely understood one of them, we would understand the major issue involved in tractability.

DEFINITION 14.4

A language L is said to be **NP**-complete if $L \in \mathbf{NP}$ and every $L_1 \in \mathbf{NP}$ is polynomial-time reducible to L .

It follows from this definition that if L is **NP**-complete and polynomial-time reducible to L_1 , then L_1 is also **NP**-complete. So if we can find one deterministic polynomial-time algorithm for any **NP**-complete language, then every language in **NP** is also in **P**, that is,

$$\mathbf{P} = \mathbf{NP}.$$

Here we hold out the hope that efficient algorithms exist for such problems, even if none have been found yet. On the other hand, if one could prove that any of the many known **NP**-complete problems is intractable, then many interesting problems are not practically solvable. This puts **NP**-completeness in the center of the question of the tractability of many important problems. At this point then, we need to study some **NP**-complete problems. The next result, known as **Cook's theorem**, provides the entry point to this study.

THEOREM 14.5

The Satisfiability Problem (SAT) is **NP**-complete.

Proof: The idea behind the proof is that for every configuration sequence of a Turing machine one can construct a CNF expression that is satisfiable if and only if there is a sequence of configurations leading to acceptance. The details, unfortunately, are long and complicated. They are technical and shed little light on the **NP** problem, so we will omit them here. Extensive discussions of Cook's theorem can be found in books devoted to complexity. ■

If we accept Cook's theorem, we immediately have a number of **NP**-complete problems.

EXAMPLE 14.11

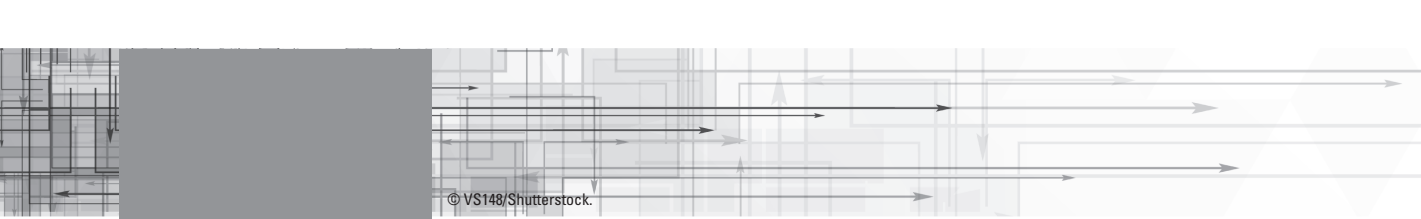
We have shown that SAT can be reduced to 3SAT, and that 3SAT can be reduced to CLIQ. Therefore 3SAT and CLIQ are both **NP**-complete.

It turns out that HAMPATH is also **NP**-complete, but the reductions needed to show this are less obvious.

In addition to SAT, 3SAT, CLIQ, and HAMPATH, there are many more problems that are known to be **NP**-complete. A good deal of effort has been expended in trying to find efficient algorithms for any of these, so far without success. This leads us to conjecture that

$$\mathbf{P} \neq \mathbf{NP}$$

and that many important problems are intractable. But while this is a reasonable working conjecture, it has not been proved. It remains the fundamental open problem in complexity theory.



CHAPTER

15

COMPILERS AND PARSING

CHAPTER SUMMARY

Previous chapters covered regular and context-free languages in a formal and detailed way. This chapter shows how such languages fit into the more applied area of compilers. We show how a compiler works and how parsing fits into the compilation process. We introduce two efficient parsing methods: top-down and bottom-up parsing. We also introduce the `FIRST()` and `FOLLOW()` functions that are used in subsequent chapters to explore parsing in more depth.

We begin the study of parsing by looking at a familiar application: a compiler. Compilers are an important application we interact with when writing and developing programs. We present a big picture view of the compiler to highlight three notions. First, a compiler is an elegant process that translates a program into code that runs on a computer. Second, theoretical constructs of previous chapters underpin key aspects of a compiler in ways that reduce its overall complexity and design. We use finite automata and regular expressions in the first phases of a compiler, and later chapters use other theoretical concepts as we expand on parsing. Third, the parser is the most important part of a compiler because it determines whether a program is syntactically correct. The compilation process cannot complete if the program is not syntactically correct.

The remainder of the chapter covers parsing in general. We study two types of parsers: top-down parsers and bottom-up parsers. In later chapters, we go into more depth on each type of parser. This chapter also introduces two functions: `FIRST()` and `FOLLOW()`. When parsing a string, one needs to determine which grammar rule to apply at each step. These functions help in determining which rule to apply. The `FIRST()` function for a rule calculates possible terminals that could be the first terminal resulting from that rule in a derivation. The `FOLLOW()` function determines the possible terminals that could follow a variable in some derivation. We use these functions in parsing algorithms in later chapters.

15.1 COMPILERS

A compiler is a translator that converts a computer program written in one programming language (the source program) into another language (the target program). There are many types of compilers, but the typical compiler translates a program in a high-level language (e.g., C++, Java, Python) into a low-level language (e.g., machine code). High-level languages are easier for humans to write programs in than low-level languages. Low-level languages such as machine code are what a computer understands, but they are tedious for humans to write programs in.

As an example, consider a C++ program in a file named **simple.cpp**. A C++ compiler would convert the program into a file of machine code matching the type of computer the user is on. That machine code, also known as an executable file, is system dependent. On a Windows machine, for example, executable files are typically given **.exe** extensions to signify they contain machine code, and our example would be named **simple.exe**. On another type of computer, a file might be named **simple** with no extension. In recent years, compilers have begun producing **bytecode** rather than machine code. Bytecode is machine-independent code that is interpreted or run on a virtual machine, meaning the code is platform independent and can be run on any machine. A Python program named **simple.py** would be compiled into bytecode in a file named **simple.pyc**.

How does the source program get translated to the target program? Figure 15.1 shows the general steps source code goes through as its being translated into machine code. We will describe each part and give some examples.

The scanner begins by reading the source program and identifying individual words. In our example, words are separated by white space (blanks, tabs, and newlines). Each word in the program is called a **token**, and the token is assigned a type that reflects its purpose (e.g. integer, keyword, variable). The token name and type are then passed as a pair to the next phase of processing (i.e., the parser).

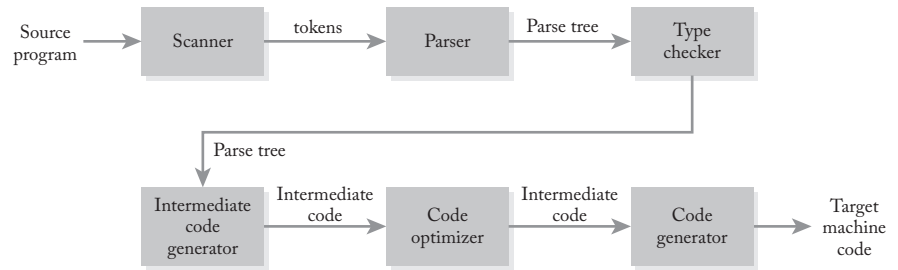


FIGURE 15.1

EXAMPLE 15.1

What are the tokens and their types in the following two lines of code?

```
sum = total + 10
green if 6.5 87
```

The first line contains five tokens: **sum** (type variable), **=** (type assignment operator), **total** (type variable), etc. Each token is considered by itself, without considering the context of the tokens around them. The second line does not make syntactic sense in our programming language, but the scanner would still identify each of the words. The full stream of tokens and their types would be (sum, variable), (=, assignment operator), (total, variable), (+, plus operator), (10, integer), (green, variable), (if, keyword), (6.5, decimal), (87, integer).

The scanner passes a stream of tokens to the parser in the order they appear in the file. How does one write a scanner? One approach is to build a dfa that recognizes all the valid tokens in a particular programming language. Each final state would represent a different token that has been recognized. Given an input program, the dfa would process each word in the program and, when a final state is reached, label the token as the type associated with the final state. Another approach would be to build a regular expression for each valid token. Then for each word, compare it to each regular expression until the word matches an expression. The order of the regular expressions is important. The regular expressions for keywords, such as **if** and **for**, should be matched before the regular expression for a variable name, as keywords are formed of letters and would also match the regular expression for a variable name.

After the scanner, the next step in the compilation process is the parser. For parsing, a context-free grammar is needed that defines the programming language. The parser then uses the grammar to attempt to derive the program from the stream of input tokens (and their types) received from the scanner. If the program can be derived, then the program is syntactically

correct. While determining correctness, the parser builds a derivation tree of the input program. Whenever a rule is recognized, the parser creates a new node in the derivation tree. If the program is syntactically correct, then the derivation tree will be complete and represents the program. The derivation tree is then passed to the next phase (the type checker).

EXAMPLE 15.2

Consider the following simple program with a main function consisting of a sequence of statements, where each statement is either an assignment statement or a call to a print function. For simplicity, we assume all words are separated by white space.

```
main ( )
    a = 4
    b = 8
    print ( a + b )
```

What is a context-free grammar for this programming language? Based on that grammar, what is a derivation tree for this program?

The context-free grammar is quite large, so we show a partial grammar. We leave parts of the grammar as exercises.

```
<program> ::= main ( ) <sequence>
<sequence> ::= <line> | <line> <sequence>
<line> ::= <variable> = <integer>
<line> ::= print ( <expression> )
```

We saw rules for the variable <expression> in an expression grammar in Section 5.3. We leave rules to expand <variable> and <integer> as exercises. The start symbol is <program>. The start of a derivation of the program above would be:

```
<program> ⇒ main ( ) <sequence>
⇒ main ( ) <line> <sequence>
⇒ main ( ) <variable> = <integer> <sequence>
⇒ ...
```

While deriving the program with the grammar, a derivation tree is built. One possible derivation tree for the program above is shown in Figure 15.2. Each node in the tree identifies the type of node as indicated by variables in the grammar (e.g., line, variable, sequence). Each node has pointers to other nodes or values that are part of the rule. For example, the sequence node has a pointer to a line node and to a sequence node because it has two parts in the rule:

```
<sequence> ::= <line> <sequence>
```

We do not provide all the details of how to build the derivation tree. Instead we only show how the program above might appear in a derivation tree. Starting at the root (see Figure 15.2), a preorder traversal shows the order in which the lines from the program would be processed. The leftmost bottom node would be processed first, assigning a the value 4. Next the other assignment node would be reached, assigning b the value 8. Then, the print node is reached, which would then reach the plus (+) node. The plus node adds a and b to produce the value 12. Finally, the print node can actually display the value 12. A compiler would not execute the program in this phase, but rather, it would pass the derivation tree to the next phase (i.e., type checker).

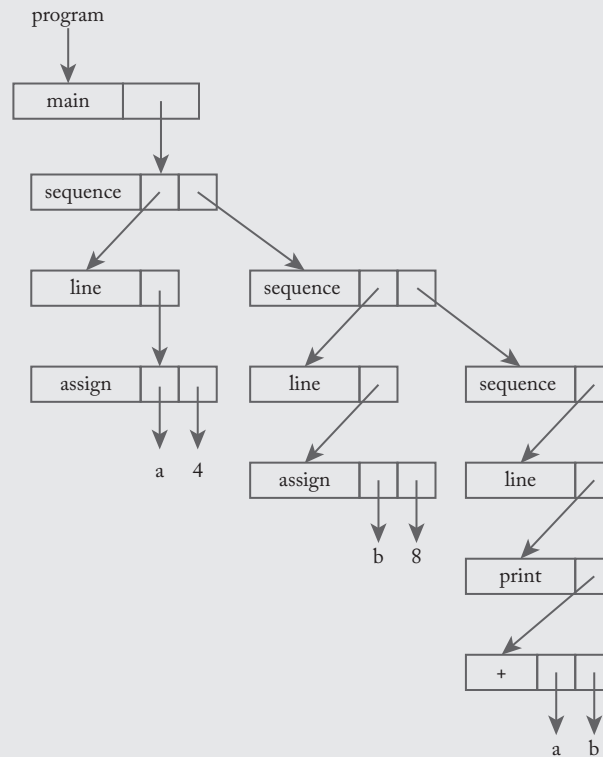


FIGURE 15.2

The type checker phase checks the derivation tree for semantic errors. For example, if the function call *process(a, b)* is defined to take two arguments of type *int* and *float* (in that order), type checking ensures exactly that. For programming languages that allow the passing of an integer where a float argument is required, the type checker might be designed to modify the derivation tree to add a node that converts the integer argument to a

float before passing it to the called function. The result of this phase is a modified derivation tree that is passed to the next phase (i.e., intermediate code generator).

The intermediate code generator phase walks through the derivation tree, generating intermediate code at each node. Intermediate code is simpler code, where each line of code has at most three addresses. An address refers to a storage location.

EXAMPLE 15.3

In the following two lines of code, each line references four variables, implying four different address references to memory. Convert each line into three-address code.

```
sum = a + b + c
sum2 = a + b + d
```

This code could be translated into the following lines in which there are at most three addresses per line. Each line above is replaced with two lines below.

```
temp1 = a + b
sum = temp1 + c
temp2 = a + b
sum2 = temp2 + d
```

The next phase is the code optimizer. When writing a computer program for a course, the program is not run many times, and execution speed is not usually a primary concern. Indeed, once the program runs correctly and gets handed in, it may never be run again. In the real world, efficiency is important, and code that is executed repeatedly should be efficient. The optimization phase scans through the intermediate code looking for code sequences it can rewrite in a more optimal fashion.

EXAMPLE 15.4

Optimize the code from Example 15.3.

In the code in Example 15.3, the expression $a + b$ is calculated twice. Code optimization can recognize such inefficiencies and change such code so that $a + b$ is only calculated once, as in the following:

```
temp1 = a + b
sum = temp1 + c
sum2 = temp1 + d
```

While reducing code by one line may not seem like much, the benefits of optimization can easily be magnified, for example, if the code at issue

is run frequently, e.g., within a loop or a frequently invoked function. The majority of code written is typically inside a loop.

The final phase of a compiler is translating the intermediate code into machine code or assembly code. Machine code typically applies operations to registers.

EXAMPLE 15.5

Translate the line **temp1 = a + b** into machine code.

The machine code might be:

```
MOV a, R1
MOV b, R2
ADD R1, R2
MOV R2, temp1
```

The first line moves the value stored in variable *a* to register 1. The second line moves the value stored in variable *b* to register 2. In line 3 register 1 is added to register 2. In line 4, the result in register 2 is moved into the storage location for variable *temp1*.

We have covered the six phases of a compiler. A program flows through these phases starting as a stream of tokens, then as a derivation tree, then as intermediate code, and finally resulting in machine code. Two additional compiler parts interact with each of the phases: a symbol table and error checking. The symbol table stores information about identifiers, such as variables and names of functions. In the scanner, when an identifier is recognized, it is added to the symbol table if not already present. In later phases, information about identifiers are added to the symbol table or referred to when needed (e.g., to determine the type of a variable). We discussed in Section 5.3 how a context-free grammar does not understand the concept of type checking. The symbol table can help in this case, as the type information would normally be available there. The symbol table stores the name of the variable in the scanner phase, the type of the variable in the parsing phase (e.g., as the result of a declaration statement) and then provides the variable type during the type checker phase.

Error checking also takes place during every phase. In the scanner phase, a word may not match any token type. In the parser phase, a group of words may not fit a rule in the context-free grammar. When errors are found, they need to be reported in a useful way to the programmer. For example, the error needs to identify the line where the error occurs (if possible).

For the rest of the book, we will focus on the parsing phase of the compiler. To learn more about compilers and other translators see books such as *Compilers: Principles, Techniques, & Tools* by Aho, Sethi, and Ullman, 1986. To learn more about intermediate code and machine code see books on computer architecture such as *Computer Organization and Design: the Hardware/Software Interface* by Patterson and Hennessy, 2013.

EXERCISES

1. Consider the following lines of code from different programming languages. Each word is a token. Identify the type of token for each word.

- (a) if count < 10
- (b) if (stuff >= value)
- (c) d = (a + b) * c ;
- (d) while (a != 7) { cnt = cnt + 1 ; a = a + 1 ; }
- (e) amount = 27 * check (a , b)

2. Assume the name of a variable in a new programming language must follow the following rules:

- The name is formed from the letters A, B or C, and the numbers 1 or 2.
- The name is of length 1 or greater.
- The name cannot start with a number.

For example, valid variable names include A1B2, CAB, and BBB2A. Invalid variable names are 3AB (starts with a number) and ABCD (includes an invalid letter).

- (a) Construct a dfa that accepts valid variable names.
- (b) Construct a regular expression that accepts valid variable names.

3. A non-negative integer is defined as follows:

- It is formed from the digits 0, 1, 2, 3, 4, 5, 6, 7, 8 and 9.
- It is of length 1 or greater.
- If it starts with 0, then it is of length 1.

For example, non-negative integers include 0, 34, and 78231. Invalid non-negative integers are 056 (starts with 0 so should be of length 1) and 67F (includes an invalid symbol F).

- (a) Construct a dfa that accepts valid non-negative integers.
- (b) Construct a regular expression that accepts valid non-negative integers.

4. Consider the following three keywords from Python: if, else, elif.

- (a) Construct a dfa that accepts these three keywords.
- (b) Construct a regular expression that accepts these three keywords.

5. Consider the following grammar for some simple programming language with `<program>` as the start variable.

```

<program> ::= begin  <sequence> end
<sequence> ::= <line> | <line> <sequence>
<line> ::= <variable> = <integer>

```

```

<line> ::= add <integer> to <variable>
<line> ::= print <variable>
<variable> ::= A | B
<integer> ::= 1 | 2

```

Which of the following are derivable from this grammar? If they are derivable, show the derivation.

- (a) begin
 B = 1
 add 2 to B
 print B
 end
- (b) begin
 B = 1
 add 2 to B
 end
print B
- (c) begin
 A = 2
 B = 1
 add B to A
 print A
 end
- (d) begin
 A = 1
 end = 2
 add 1 to end
 print end
 end
- (e) begin
 add 2 to B
 print B
 B = 1
 end

6. In the following, the line shown references six variables, implying six different address references to memory. Convert the line into three-address code.

```
total = e + d + c + b + a
```

7. In the following, each line shown is three address code. Optimize the code and show the resulting optimized code.

```

temp1 = d + e
temp2 = temp1 + c
temp3 = d + e
temp4 = temp3 + c
result = temp2 + temp4

```

15.2 TOP-DOWN VS. BOTTOM-UP PARSING

There are different approaches to parsing algorithms. The exhaustive search algorithm (Section 5.2) is a form of top-down parsing. In top-down parsing, the parser begins with the grammar's start symbol and replaces the left-most variable in each step of the derivation until the input string is derived. A derivation tree for top-down parsing produces derivation trees like we saw in Section 5.1. With exhaustive search parsing, if a variable A has more than one rule, the parser tries all possible rules in each step, an inefficient approach. LL parsing is another type of top-down parsing. With LL parsing, if a variable A has more than one rule, the algorithm inspects the symbols in the input string from left to right to decide which A rule to apply. LL parsing is more efficient than trying every A rule.

A second approach, bottom-up parsing, works in the opposite way. In bottom-up parsing, parsing starts with the symbols in the string, examining them from left to right, and replaces the right-hand side of a grammar rule with its left-hand side, until the start symbol is derived. If we apply the rules in the reverse order they are found, the string is derived from the start symbol with a rightmost derivation. With respect to derivation trees, bottom-up parsing produces an inverted derivation tree. LR parsing is a type of bottom-up parsing. Determining when to replace a right-hand side has more details, but first we look at some examples that compare the derivation trees for these two types of parsing.

EXAMPLE 15.6

Given the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow aABc, \\ A &\rightarrow aA \mid b, \\ B &\rightarrow bBb \mid c. \end{aligned}$$

Show the steps in top-down parsing of both the derivation tree and the derivation of the string $aabbcbc$.

Here is the leftmost derivation: $S \Rightarrow aABc \Rightarrow aaABc \Rightarrow aabBc \Rightarrow aabbBbc \Rightarrow aabbcbc$.

Figure 15.3 shows the steps in building the derivation tree. Steps a–e show the order the variables are replaced in the derivation tree for the derivation of $aabbcbc$. Whenever there is a choice of more than one variable, the leftmost variable is replaced.

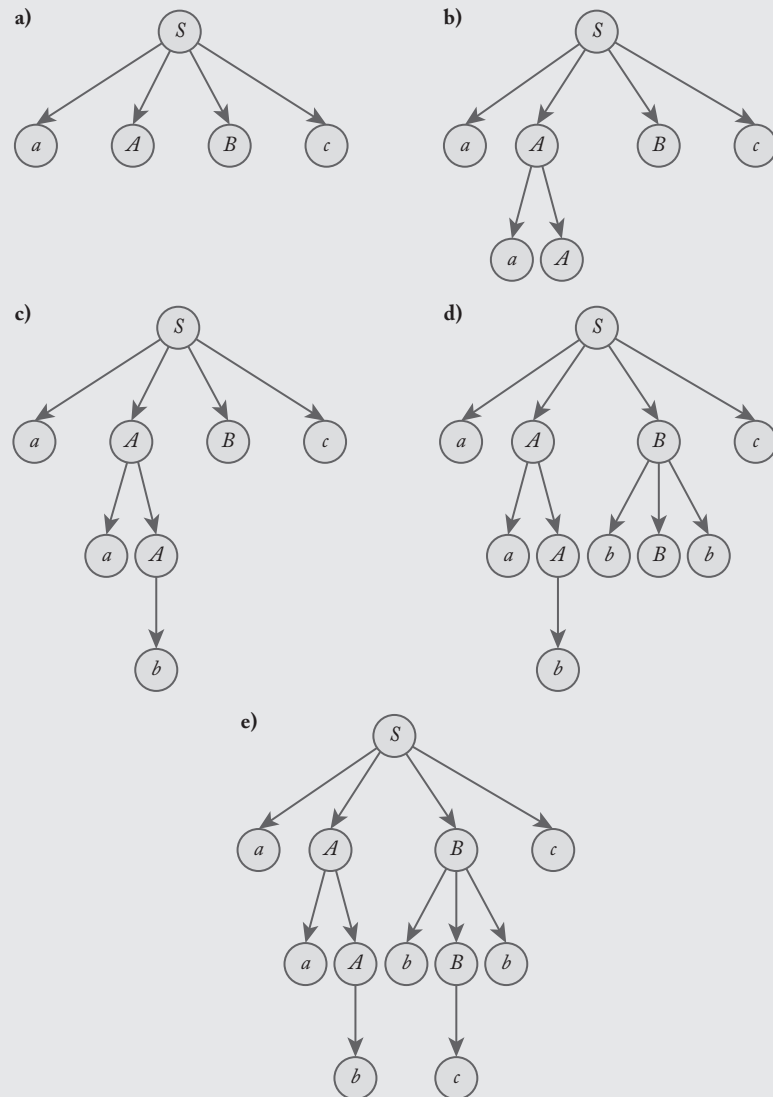


FIGURE 15.3

EXAMPLE 15.7

Using the same grammar as Example 15.6, show the steps in bottom-up parsing of both the inverted derivation tree and the derivation of the string *aabbcb*.

Here is the rightmost derivation: $S \Rightarrow aABc \Rightarrow aAbBbc \Rightarrow aAbcbcb \Rightarrow aaAbcbcb \Rightarrow aabbcbcb$.

Figure 15.4 shows the steps in building the inverted derivation tree and the order the variables are derived from right-hand sides. This figure also shows only those symbols from the input string from left to right that are needed until a right-hand side is recognized. In (a), the first three symbols aab are examined, and the rule $A \rightarrow b$ is applied by replacing b by A . In the inverted tree, we draw an arrow from the A node to the b node. Note that the first rule we found, $A \rightarrow b$, is the last rule applied in the rightmost derivation. In (b), aA is a right-hand side of a rule, so the rule $A \rightarrow aA$ is applied. In (c), two more symbols b and c are considered, and then the rule $B \rightarrow c$ is applied. In (d), the next symbol b is considered, and the rule $B \rightarrow bBb$ is applied. In (e), the last symbol c is considered, and the rule $S \rightarrow aABc$ is applied. This inverted derivation tree shows that the string $aabbcbcb$ is replaced by S . If you invert the tree, it will look like the tree in Example 15.6.

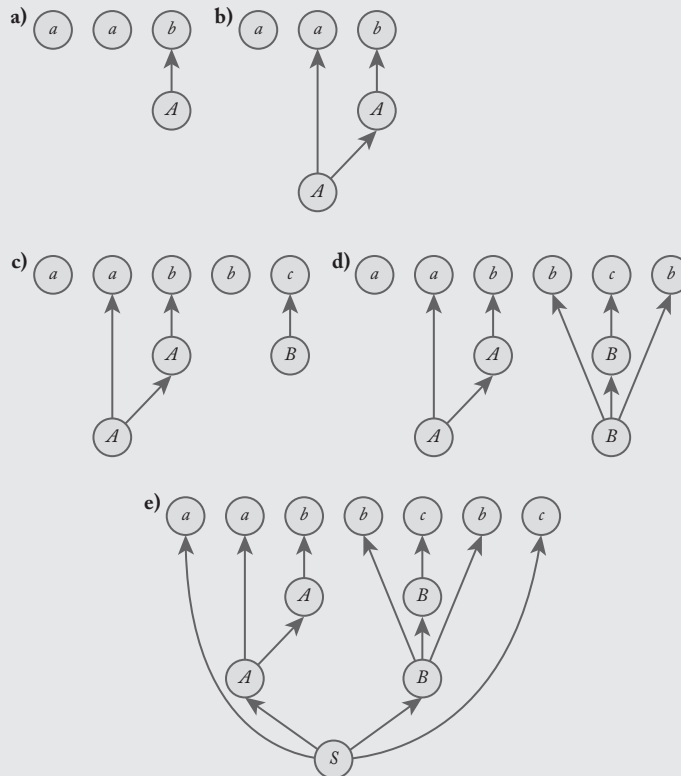


FIGURE 15.4

EXERCISES

1. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S\}$, $T = \{a, b, c\}$, and P consists of

$$S \rightarrow aaS \mid bS \mid c.$$

- (a) Show the derivation tree resulting from top-down parsing of the string $baabc$.
 - (b) Show the leftmost derivation of the string $baabc$.
 - (c) Show the derivation tree resulting from top-down parsing of the string $aabbaac$.
 - (d) Show the leftmost derivation of the string $aabbaac$.
2. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A\}$, $T = \{a, b\}$, and P consists of

$$S \rightarrow aSb \mid aAb,$$

$$A \rightarrow aA \mid \lambda.$$

- (a) Show the derivation tree resulting from top-down parsing of the string $aaab$.
 - (b) Show the leftmost derivation of the string $aaab$.
 - (c) Show the derivation tree resulting from top-down parsing of the string $aaaabb$.
 - (d) Show the leftmost derivation of the string $aaaabb$.
3. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c\}$, and P consists of

$$S \rightarrow aSb \mid BAc,$$

$$A \rightarrow aA \mid b,$$

$$B \rightarrow bBb \mid c.$$

- (a) Show the derivation tree resulting from top-down parsing of the string $bbcbbbaabc$.
 - (b) Show the leftmost derivation of the string $bbcbbbaabc$.
 - (c) Show the derivation tree resulting from top-down parsing of the string $aabcababcb$.
 - (d) Show the leftmost derivation of the string $aabcababcb$.
4. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B, C\}$, $T = \{a, b, c\}$, and P consists of

$$S \rightarrow ABC,$$

$$A \rightarrow aA \mid b,$$

$$\begin{aligned} B &\rightarrow bB \mid a, \\ C &\rightarrow cCa \mid b. \end{aligned}$$

- (a) Show the derivation tree resulting from top-down parsing of the string *aaabacba*.
 - (b) Show the leftmost derivation of the string *aaabacba*.
 - (c) Show the derivation tree resulting from top-down parsing of the string *aabbaccbaa*.
 - (d) Show the leftmost derivation of the string *aabbaccbaa*.
5. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S\}$, $T = \{a, b, c\}$, and P consists of

$$S \rightarrow aaS \mid bS \mid c.$$

- (a) Show the derivation tree resulting from bottom-up parsing of the string *baabc*. The start symbol would be at the bottom of the tree.
 - (b) Show the rightmost derivation of the string *baabc*.
 - (c) Show the derivation tree resulting from bottom-up parsing of the string *abaabc*.
 - (d) Show the rightmost derivation of the string *abaabc*.
6. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow BA, \\ A &\rightarrow aA \mid \lambda, \\ B &\rightarrow bBc \mid c. \end{aligned}$$

- (a) Show the derivation tree resulting from bottom-up parsing of the string *caa*.
 - (b) Show the rightmost derivation of the string *caa*.
 - (c) Show the derivation tree resulting from bottom-up parsing of the string *bbcccaa*.
 - (d) Show the rightmost derivation of the string *bbcccaa*.
7. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow aSb \mid BAc, \\ A &\rightarrow aA \mid b, \\ B &\rightarrow bBb \mid c, \end{aligned}$$

- (a) Show the derivation tree resulting from bottom-up parsing of the string *bcbbaabc*.
 - (b) Show the rightmost derivation of the string *bcbbaabc*.
 - (c) Show the derivation tree resulting from bottom-up parsing of the string *aabcabcb*.
 - (d) Show the rightmost derivation of the string *aabcabcb*.
8. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B, C\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow ABC, \\ A &\rightarrow aA \mid b, \\ B &\rightarrow bB \mid a, \\ C &\rightarrow cCa \mid b. \end{aligned}$$

- (a) Show the derivation tree resulting from bottom-up parsing of the string *aaabacba*.
- (b) Show the rightmost derivation of the string *aaabacba*.
- (c) Show the derivation tree resulting from bottom-up parsing of the string *aabbaccbaa*.
- (d) Show the rightmost derivation of the string *aabbaccbaa*.

15.3 FIRST FUNCTION

We have not given many details on how to choose a rule if a variable has more than one rule, or how to choose a righthand side to replace if there is more than one choice. Before giving those details, we define two functions useful in both *LL* and *LR* parsing: **FIRST** and **FOLLOW**. The **FIRST** function calculates a set of terminals from a sentential form that could be the first terminal in the string after the sentential form has replaced all the variables. Knowing the first terminal helps guide the parsing, that is, helps choose the next rule to apply.

DEFINITION 15.1

Given a context-free grammar $G = (V, T, S, P)$, $a \in T$ and $w, v \in (V \cup T)^*$, the **FIRST**(w) is the set of terminals that can be the first terminal a in $w \xRightarrow{*} av$. λ is in **FIRST**(w) if $w \xRightarrow{*} \lambda$.

We show how to calculate **FIRST** for variables and terminals in the grammar, for λ and for strings.

Algorithm for FIRST

Given a grammar $G = (V, T, S, P)$, calculate $\text{FIRST}(w)$ for w in $(V \cup T)^*$,

1. For $a \in T$, $\text{FIRST}(a) = \{a\}$.
2. $\text{FIRST}(\lambda) = \{\lambda\}$.
3. For $A \in V$, set $\text{FIRST}(A) = \{\}$.
4. Repeat these steps until no more terminals or λ can be added to any FIRST set for variables.

For every production $A \rightarrow w$

$$\text{FIRST}(A) = \text{FIRST}(A) \cup \text{FIRST}(w)$$

5. For $w = x_1x_2x_3 \dots x_n$ where $x_i \in (V \cup T)$

$$(a) \text{ FIRST}(w) = \text{FIRST}(x_1) - \{\lambda\}$$

(b) For i from 2 to n do:

if $x_j \xRightarrow{*} \lambda$ for all j from 1 to $i - 1$ then

$$\text{FIRST}(w) = \text{FIRST}(w) \cup \text{FIRST}(x_i) - \{\lambda\}$$

(c) If $x_i \xRightarrow{*} \lambda$ for all i from 1 to n then

$$\text{FIRST}(w) = \text{FIRST}(w) \cup \{\lambda\}$$

For example, for $A \rightarrow w$ with $w = x_1x_2x_3 \dots x_n$ every $a \in T$ in $\text{FIRST}(x_1)$ is put in $\text{FIRST}(A)$. If $x_1 \xRightarrow{*} \lambda$, then every $a \in T$ in $\text{FIRST}(x_2)$ is put in $\text{FIRST}(A)$. If $x_1 \xRightarrow{*} \lambda$ and $x_2 \xRightarrow{*} \lambda$, then every $a \in T$ in $\text{FIRST}(x_3)$ is added to $\text{FIRST}(A)$. This continues until some x_i does not derive λ . Step (c) checks if λ should be added to the $\text{FIRST}(w)$. Note that λ is only put in the $\text{FIRST}(w)$ if λ is in the FIRST set for every symbol in w , which means $w \xRightarrow{*} \lambda$.

EXAMPLE 15.8

Calculate the FIRST function for each of the variables in $G = (V, T, S, P)$, where $V = \{S, A\}$, $T = \{a, b, c\}$, and P consists of

$$S \rightarrow aSb \mid A,$$

$$A \rightarrow bAa \mid c.$$

Start with $\text{FIRST}(S) = \{\}$. The variable S has two productions so we compute the FIRST of both of them and then union them together.

$$\text{FIRST}(S) = \text{FIRST}(S) \cup \text{FIRST}(aSb) \cup \text{FIRST}(A) = \{\} \cup \{a\} \cup \text{FIRST}(A).$$

We need to calculate the $\text{FIRST}(A)$ in order to complete $\text{FIRST}(S)$. Start with $\text{FIRST}(A) = \{\}$. The variable A has two productions.

$$\text{FIRST}(A) = \text{FIRST}(A) \cup \text{FIRST}(bAa) \cup \text{FIRST}(c) = \{\} \cup \{b\} \cup \{c\} = \{b, c\}.$$

Note that to calculate $\text{FIRST}(bAa)$, we take the FIRST of the first symbol of bAa , $\text{FIRST}(b) = \{b\}$. Since b is a terminal, we do not need to look past the b in bAa .

Now we can complete $\text{FIRST}(S)$.

$$\begin{aligned}\text{FIRST}(S) &= \text{FIRST}(S) \cup \text{FIRST}(aSb) \cup \text{FIRST}(A) = \{\} \cup \{a\} \cup \{b, c\} \\ &= \{a, b, c\}.\end{aligned}$$

EXAMPLE 15.9

Calculate the FIRST function for each of the variables in $G = (V, T, S, P)$, where $V = \{S\}$, $T = \{a, b\}$, and P consists of

$$S \rightarrow Sa \mid b.$$

Start with $\text{FIRST}(S) = \{\}$. The variable S has two productions so we compute the FIRST of both of them and then union them together.

$$\begin{aligned}\text{FIRST}(S) &= \text{FIRST}(S) \cup \text{FIRST}(Sa) \cup \text{FIRST}(b) \\ \text{FIRST}(b) &= \{b\}\end{aligned}$$

For $\text{FIRST}(Sa)$, the first symbol is S . S does not derive λ so we do not consider the second symbol a in Sa . Thus $\text{FIRST}(Sa) = \text{FIRST}(S)$.

$$\begin{aligned}\text{So, } \text{FIRST}(S) &= \text{FIRST}(S) \cup \text{FIRST}(Sa) \cup \text{FIRST}(b) = \text{FIRST}(S) \cup \\ &\text{FIRST}(S) \cup \{b\} = \{b\}\end{aligned}$$

Neither Example 15.8 or Example 15.9 had a λ -production. In both examples when computing $\text{FIRST}(x_1x_2x_3 \dots x_n)$ for any production $A \rightarrow x_1x_2x_3 \dots x_n$ where $A \in V$ and $x_i \in (V \cup T)$, we never have to consider any symbol to the right of x_1 . Let us look at another example that has a λ -production.

EXAMPLE 15.10

Calculate the FIRST function for each of the variables from $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned}
S &\rightarrow aASc \mid BAd, \\
A &\rightarrow cA \mid d, \\
B &\rightarrow bBd \mid \lambda.
\end{aligned}$$

Start with $\text{FIRST}(S) = \{\}$, $\text{FIRST}(A) = \{\}$ and $\text{FIRST}(B) = \{\}$. Since $\text{FIRST}(S)$ depends on $\text{FIRST}(B)$, and both A and B productions do not start with a variable, calculate $\text{FIRST}(A)$ and $\text{FIRST}(B)$ first.

$$\begin{aligned}
\text{FIRST}(A) &= \text{FIRST}(A) \cup \text{FIRST}(cA) \cup \text{FIRST}(d) = \{\} \cup \{c\} \cup \{d\} = \{c, d\}. \\
\text{FIRST}(B) &= \text{FIRST}(B) \cup \text{FIRST}(bBd) \cup \text{FIRST}(\lambda) = \{\} \cup \{b\} \cup \{\lambda\} = \{b, \lambda\}. \\
\text{FIRST}(S) &= \text{FIRST}(S) \cup \text{FIRST}(aASc) \cup \text{FIRST}(BAd). \\
\text{FIRST}(aASc) &= \{a\}.
\end{aligned}$$

For $\text{FIRST}(BAd)$, $\text{FIRST}(B) = \{b, \lambda\}$, so b is put in $\text{FIRST}(BAd)$. Since $B \xRightarrow{*} \lambda$, we consider $\text{FIRST}(A)$. $\text{FIRST}(A) = \{c, d\}$. Both c and d are added to $\text{FIRST}(BAd)$, A does not derive λ , so we do not add any more symbols to $\text{FIRST}(BAd)$. $\text{FIRST}(BAd) = \{b, c, d\}$.

Then, $\text{FIRST}(S) = \text{FIRST}(S) \cup \text{FIRST}(aASc) \cup \text{FIRST}(BAd) = \{\} \cup \{a\} \cup \{b, c, d\} = \{a, b, c, d\}$.

Given a context-free grammar $G = (V, T, S, P)$, for every symbol a in $\text{FIRST}(A)$, where $a \in T$ and $A \in V$, there should be at least one derivation $A \xRightarrow{*} aw$ where $w \in (V \cup T)^*$. If $\lambda \in \text{FIRST}(A)$, then there should be at least one derivation $A \xRightarrow{*} \lambda$. Let us look at some examples.

EXAMPLE 15.11

For the grammar in Example 15.10, show the derivation of a sentential form for each symbol $a \in \text{FIRST}(A)$ that shows a as the first symbol of the sentential form, for all $A \in V$.

$$\text{FIRST}(S) = \{a, b, c, d\}.$$

$S \Rightarrow aASc$ shows a first.

$S \Rightarrow BAd \Rightarrow bBdAd$ shows b first.

$S \Rightarrow BAd \Rightarrow Ad \Rightarrow cAd$ shows c first.

$S \Rightarrow BAd \Rightarrow Ad \Rightarrow dd$ shows d first.

For variables A and B , one rule for each symbol suffices.

$$\text{FIRST}(A) = \{c, d\}.$$

$A \Rightarrow cA$ shows c first.

$A \Rightarrow d$ shows d first.

$$\text{FIRST}(B) = \{b, \lambda\}.$$

$B \Rightarrow bBd$ shows b first.

$B \Rightarrow \lambda$ shows λ .

EXERCISES

1. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S\}$, $T = \{a, b, c\}$, and P consists of

$$S \rightarrow aaS \mid bS \mid c.$$

- (a) Compute $\text{FIRST}(S)$.
- (b) Compute $\text{FIRST}(baaS)$.
- (c) Compute $\text{FIRST}(aaS)$.

2. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A\}$, $T = \{a, b\}$, and P consists of

$$S \rightarrow aSb \mid aAb,$$

$$A \rightarrow aA \mid \lambda.$$

- (a) Compute $\text{FIRST}(A)$.
- (b) Compute $\text{FIRST}(S)$.
- (c) Compute $\text{FIRST}(aaAbb)$.
- (d) Compute $\text{FIRST}(Ab)$.

3. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c\}$, and P consists of

$$S \rightarrow aSb \mid BAc,$$

$$A \rightarrow aA \mid b,$$

$$B \rightarrow bBb \mid c.$$

- (a) Compute $\text{FIRST}(A)$.
- (b) Compute $\text{FIRST}(B)$.
- (c) Compute $\text{FIRST}(S)$.
- (d) Compute $\text{FIRST}(bA)$.
- (e) Compute $\text{FIRST}(Ac)$.

4. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B, C\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow ABC, \\ A &\rightarrow aA \mid b, \\ B &\rightarrow bB \mid a, \\ C &\rightarrow cCa \mid b. \end{aligned}$$

- (a) Compute $\text{FIRST}(A)$.
 - (b) Compute $\text{FIRST}(B)$.
 - (c) Compute $\text{FIRST}(C)$.
 - (d) Compute $\text{FIRST}(S)$.
 - (e) Compute $\text{FIRST}(BC)$.
 - (f) Compute $\text{FIRST}(Ca)$.
5. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow BA, \\ A &\rightarrow aA \mid \lambda, \\ B &\rightarrow bBc \mid c. \end{aligned}$$

- (a) Compute $\text{FIRST}(A)$.
 - (b) Compute $\text{FIRST}(B)$.
 - (c) Compute $\text{FIRST}(S)$.
 - (d) Compute $\text{FIRST}(AS)$.
 - (e) Compute $\text{FIRST}(Bc)$.
6. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B, C\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned} S &\rightarrow ABCBd, \\ A &\rightarrow aA \mid \lambda, \\ B &\rightarrow bbB \mid \lambda, \\ C &\rightarrow cC \mid AB. \end{aligned}$$

- (a) Compute $\text{FIRST}(A)$.
- (b) Compute $\text{FIRST}(B)$.
- (c) Compute $\text{FIRST}(C)$.
- (d) Compute $\text{FIRST}(S)$.

- (e) Compute $\text{FIRST}(CB)$.
 (f) Compute $\text{FIRST}(Bc)$.
7. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B, C\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow BACB, \\ A &\rightarrow CaA \mid \lambda, \\ B &\rightarrow bBb \mid \lambda, \\ C &\rightarrow cC \mid BB. \end{aligned}$$

- (a) Compute $\text{FIRST}(B)$.
 (b) Compute $\text{FIRST}(C)$.
 (c) Compute $\text{FIRST}(A)$.
 (d) Compute $\text{FIRST}(S)$.
 (e) Compute $\text{FIRST}(CA)$.
 (f) Compute $\text{FIRST}(BC)$.
8. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B, C\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned} S &\rightarrow BCD \mid AS, \\ A &\rightarrow aAa \mid \lambda, \\ B &\rightarrow ABb \mid \lambda, \\ C &\rightarrow cC \mid \lambda. \end{aligned}$$

- (a) $\text{FIRST}(A) = \{a, \lambda\}$. Show the derivation of a sentential form that shows a as the first symbol of the sentential form.
 (b) $\text{FIRST}(B) = \{a, b, \lambda\}$. Show the derivation of a sentential form for each terminal $a \in \text{FIRST}(B)$ that shows a as the first symbol of the sentential form.
 (c) $\text{FIRST}(C) = \{c, \lambda\}$. Show the derivation of a sentential form that shows c as the first symbol of the sentential form.
 (d) $\text{FIRST}(S) = \{a, b, c, d\}$. Show the derivation of a sentential form for each terminal $a \in \text{FIRST}(S)$ that shows a as the first symbol of the sentential form.
9. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B, C\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned} S &\rightarrow BAdC, \\ A &\rightarrow Aa \mid \lambda, \\ B &\rightarrow Acb \mid \lambda, \\ C &\rightarrow ABc. \end{aligned}$$

- (a) $\text{FIRST}(A) = \{a, \lambda\}$. Show the derivation of a sentential form that shows a as the first symbol of the sentential form.
- (b) $\text{FIRST}(B) = \{a, c, \lambda\}$. Show the derivation of a sentential form for each terminal $a \in \text{FIRST}(B)$ that shows a as the first symbol of the sentential form.
- (c) $\text{FIRST}(C) = \{a, c\}$. Show the derivation of a sentential form for each terminal $a \in \text{FIRST}(C)$ that shows a as the first symbol of the sentential form.
- (d) $\text{FIRST}(S) = \{a, c, d\}$. Show the derivation of a sentential form for each terminal $a \in \text{FIRST}(S)$ that shows a as the first symbol of the sentential form.

15.4 FOLLOW FUNCTION

Next we define the FOLLOW function. FOLLOW makes use of a special symbol $\$$ that indicates an end-of-string marker. The $\$$ symbol is not a terminal or a variable, but rather it is a special symbol that appears to the immediate right of a string to help determine that all the terminals in an input string have been processed. This is a similar idea to a bottom of stack marker in an npda, which is a special symbol one can look at to see if all the symbols on the stack have been processed.

The FOLLOW function is applied to a variable and calculates the set of terminals or $\$$ that can immediately follow that variable in some sentential form. Note that $\$$ can always follow the start symbol S of a context-free grammar. Suppose $S \xRightarrow{*} w$. That is, starting with the start symbol S , S will derive some string w in $L(G)$. Then $S\$ \xRightarrow{*} w\$$. The FOLLOW function helps parsing algorithms choose a production $A \rightarrow w$ to expand, based on those terminals that can follow the variable A .

DEFINITION 15.2

Given a context-free grammar $G = (V, T, S, P)$, $A \in V$, $a \in T$ and $w, v \in (V \cup T)^*$, **FOLLOW**(A) is the set of terminals that can be the first terminal a immediately following A in some sentential form $vAaw$. $\$$ is always in **FOLLOW**(S).

Here is an algorithm to calculate FOLLOW for the variables in $G = (V, T, S, P)$. Let $A, B \in V$ and $v, w \in (V \cup T)^*$.

1. $\$$ is in **FOLLOW**(S).
2. For $A \rightarrow vB$, **FOLLOW**(A) is in **FOLLOW**(B).

3. For $A \rightarrow vBw$:

- (a) $\text{FIRST}(w) - \{\lambda\}$ is in $\text{FOLLOW}(B)$.
- (b) If $\lambda \in \text{FIRST}(w)$, then $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$.

EXAMPLE 15.12

Calculate the FOLLOW function for each of the variables from Example 15.8, shown here again. $G = (V, T, S, P)$, where $V = \{S, A\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow aSb \mid A, \\ A &\rightarrow bAa \mid c. \end{aligned}$$

The variable S appears in one right-hand side, $S \rightarrow aSb$.

$\text{FOLLOW}(S) = \{b, \$\}$, b because b immediately follows S in the right-hand side aSb , and $\$$ because S is the start symbol.

The variable A appears in two right-hand sides. From $A \rightarrow bAa$, we add a to the $\text{FOLLOW}(A)$ since a immediately follows A .

From $S \rightarrow A$, A is at the right end of the production, that means anything that can follow S , can also follow A . We add everything in the $\text{FOLLOW}(S)$ to $\text{FOLLOW}(A)$.

Thus $\text{FOLLOW}(A) = \{a, b, \$\}$.

As an example where something from $\text{FOLLOW}(S)$ is also in $\text{FOLLOW}(A)$, you can see in this partial derivation that the b following S is next following A after applying the rule $S \rightarrow A$. The derivation is $S \Rightarrow aSb \Rightarrow aAb \dots$

EXAMPLE 15.13

Calculate the FOLLOW function for each of the variables from Example 15.10, shown here again. $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned} S &\rightarrow aASc \mid BAd, \\ A &\rightarrow cA \mid d, \\ B &\rightarrow bBd \mid \lambda. \end{aligned}$$

The variable S appears in one right-hand side, $S \rightarrow aASc$.

$\text{FOLLOW}(S) = \{c, \$\}$, c because c immediately follows S in the right-hand side $aASc$, and $\$$ because S is the start symbol.

Next calculate FOLLOW(A). A appears in three right-hand sides. For $S \rightarrow aASc$, we calculate FIRST(Sc), the part that follows A . FIRST(Sc) = $\{a, b, c, d\}$ so we set FOLLOW(A) = $\{a, b, c, d\}$. For $S \rightarrow BAd$, we calculate FIRST(d) = $\{d\}$, and we add d to FOLLOW(A), but note d is already in FOLLOW(A). For $A \rightarrow cA$, since A is at the end of the right-hand side, we calculate that everything in FOLLOW(A) is in FOLLOW(A). That is redundant and does not add anything to FOLLOW(A). Thus, FOLLOW(A) = $\{a, b, c, d\}$.

Now, calculate FOLLOW(B). B appears in two right-hand sides. For $S \rightarrow BAd$, we calculate FIRST(Ad), the part that follows B . FIRST(Ad) = $\{c, d\}$ so we put c and d in FOLLOW(B). For $B \rightarrow bBd$, we calculate FIRST(d) = $\{d\}$, and we add d to FOLLOW(B), but it is already there. Thus, FOLLOW(B) = $\{c, d\}$.

We display the FIRST and FOLLOW for all the variables in this grammar in a table.

	FIRST	FOLLOW
S	a, b, c, d	$c, \$$
A	c, d	a, b, c, d
B	b, λ	c, d

EXAMPLE 15.14

For each variable A in the grammar in Example 15.10, also shown here, and for each a in FOLLOW(A), show a derivation to a sentential form that shows a immediately following A .

$G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned} S &\rightarrow aASc \mid BAd, \\ A &\rightarrow cA \mid d, \\ B &\rightarrow bBd \mid \lambda. \end{aligned}$$

FOLLOW(S) = $\{c, \$\}$.

$S \Rightarrow aASc$ shows c following S .

$\$$ always follows the start symbol.

FOLLOW(A) = $\{a, b, c, d\}$.

$S \Rightarrow aASc \Rightarrow aAaAScc$ using $S \rightarrow aASc$ shows a following A .

$S \Rightarrow aASc \Rightarrow aABAdc \Rightarrow aAbBdAdc$ using $S \rightarrow BAd$ followed by $B \rightarrow bBd$ shows b following A .

$S \Rightarrow aASc \Rightarrow aABAdc \Rightarrow aAAdc \Rightarrow aAcAdc$ using $S \rightarrow BAd$, $B \rightarrow \lambda$
 and $A \rightarrow cA$, shows c following A .
 $S \Rightarrow BAd$, shows d following A .

$\text{FOLLOW}(B) = \{c, d\}$.
 $S \Rightarrow BAd \Rightarrow BcAd$ using $A \rightarrow cA$ shows c following B .
 $S \Rightarrow BAd \Rightarrow bBdAd$ using $B \rightarrow bBd$ shows d following B .

We will see in the next two chapters how FIRST and FOLLOW sets are helpful with two different parsing algorithms.

EXERCISES

1. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S\}$, $T = \{a, b, c\}$, and P consists of

$$S \rightarrow aaS \mid bS \mid c.$$

Compute $\text{FOLLOW}(S)$.

2. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A\}$, $T = \{a, b\}$, and P consists of

$$\begin{aligned} S &\rightarrow aSb \mid aAb, \\ A &\rightarrow aA \mid \lambda. \end{aligned}$$

(a) Compute $\text{FOLLOW}(S)$.

(b) Compute $\text{FOLLOW}(A)$.

3. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow aSb \mid BAc, \\ A &\rightarrow aA \mid b, \\ B &\rightarrow bBb \mid c. \end{aligned}$$

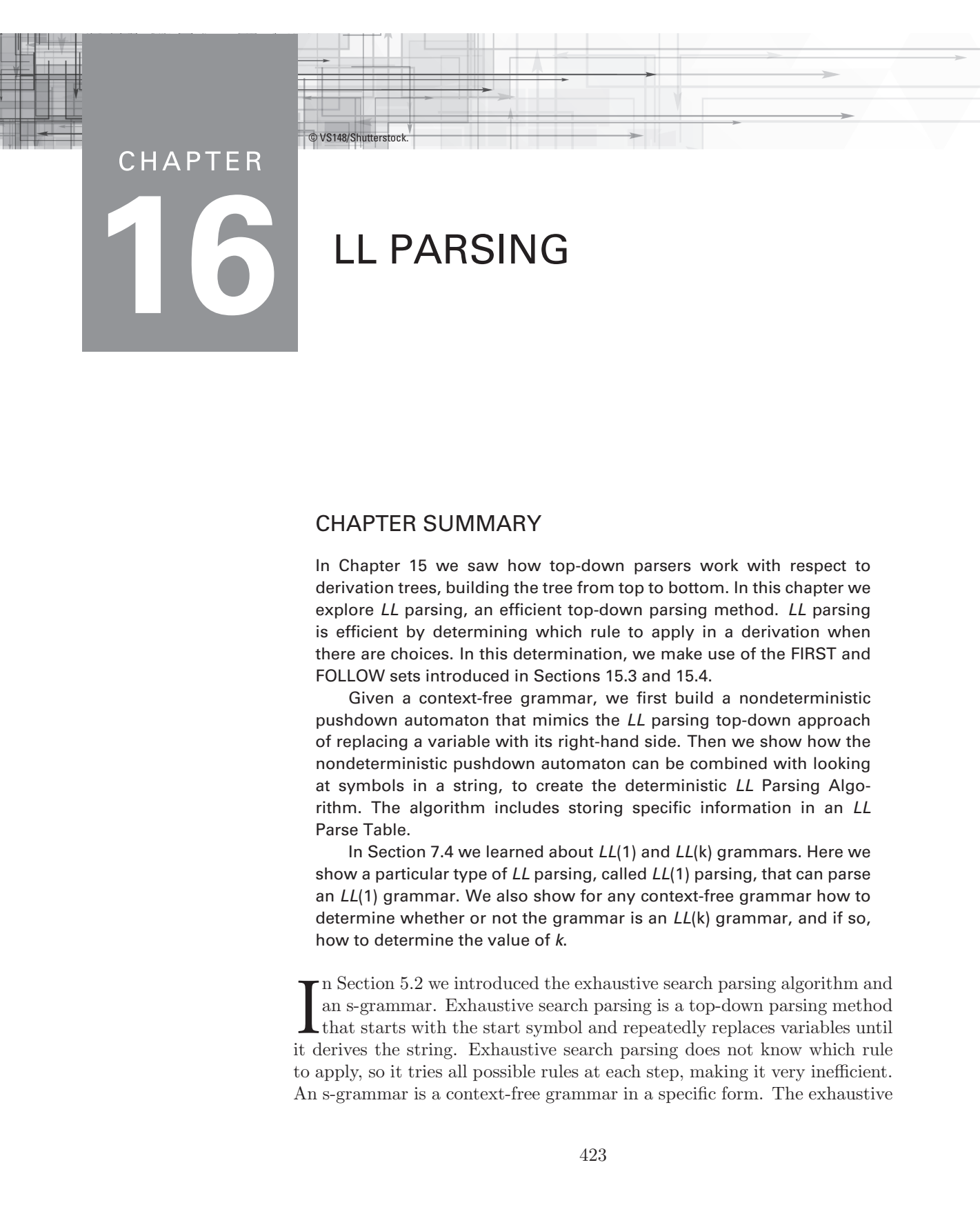
(a) Compute $\text{FOLLOW}(S)$.

(b) Compute $\text{FOLLOW}(A)$.

(c) Compute $\text{FOLLOW}(B)$.

4. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B, C\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow ABC, \\ A &\rightarrow aA \mid b, \\ B &\rightarrow bB \mid a, \\ C &\rightarrow cCa \mid b. \end{aligned}$$



CHAPTER 16

LL PARSING

CHAPTER SUMMARY

In Chapter 15 we saw how top-down parsers work with respect to derivation trees, building the tree from top to bottom. In this chapter we explore *LL* parsing, an efficient top-down parsing method. *LL* parsing is efficient by determining which rule to apply in a derivation when there are choices. In this determination, we make use of the FIRST and FOLLOW sets introduced in Sections 15.3 and 15.4.

Given a context-free grammar, we first build a nondeterministic pushdown automaton that mimics the *LL* parsing top-down approach of replacing a variable with its right-hand side. Then we show how the nondeterministic pushdown automaton can be combined with looking at symbols in a string, to create the deterministic *LL* Parsing Algorithm. The algorithm includes storing specific information in an *LL* Parse Table.

In Section 7.4 we learned about *LL*(1) and *LL*(*k*) grammars. Here we show a particular type of *LL* parsing, called *LL*(1) parsing, that can parse an *LL*(1) grammar. We also show for any context-free grammar how to determine whether or not the grammar is an *LL*(*k*) grammar, and if so, how to determine the value of *k*.

In Section 5.2 we introduced the exhaustive search parsing algorithm and an s-grammar. Exhaustive search parsing is a top-down parsing method that starts with the start symbol and repeatedly replaces variables until it derives the string. Exhaustive search parsing does not know which rule to apply, so it tries all possible rules at each step, making it very inefficient. An s-grammar is a context-free grammar in a specific form. The exhaustive

search algorithm applied to an s-grammar parses strings in linear time since there is at most one rule that can be applied at each step. The parsing of s-grammars is efficient, but the s-grammar definition is too restrictive to be practical. In this chapter we will study *LL* parsing, an efficient top-down method that can be applied to any context-free grammar with no restrictions on the grammar's form. However, we will see that *LL* parsing does not work on all context-free grammars.

In Section 7.4 we explored deterministic context-free grammars, focusing on *LL*(1) and *LL*(*k*) grammars in a formal way, but did not look at parsing algorithms for such grammars. *LL* parsing can be applied to such grammars. *LL*(*k*) parsing looks at *k* symbols in the string called **lookaheads** to help guide the parsing process. The first *L* in *LL*(*k*) stands for processing the input from left to right, the second *L* stands for a leftmost derivation, and the *k* indicates the minimum number of lookaheads needed to parse the string deterministically. *LL*(*k*) parsing works on *LL*(*k*) grammars. In this chapter we first focus on *k* = 1. We apply *LL*(1) parsing to context-free grammars and use the *LL*(1) Parsing Algorithm to identify which context-free grammars are *LL*(1) grammars. We then explore *LL*(*k*) parsing and determine the value of *k*, if any, that works for a specific context-free grammar.

16.1 CONTEXT-FREE GRAMMAR CONVERSION TO NONDETERMINISTIC PUSHDOWN AUTOMATON

Pushdown Automata for *LL* Parsing

Given a context-free grammar, we create an equivalent nondeterministic pushdown automaton (npda) that models the *LL* parsing process. We previously saw in Section 7.2 an algorithm for converting any context-free grammar to an npda. In that algorithm, we first had to convert the context-free grammar to Greibach normal form. Here we use a different approach where the context-free grammar is not restricted to any form.

We design the npda in the following way. Given a grammar $G = (V, T, S, P)$, parsing starts with the start symbol and repeatedly replaces a variable by one of its rules. For a rule $A \rightarrow w \in P$ with $A \in V$ and $w \in (V \cup T)^*$, the npda mimics the parsing process by popping the variable *A* off the npda's stack and pushing its right-hand side *w* on the stack. For the terminals, the order they appear from left to right in the string must be the same order they appear on top of the stack in the npda. As a terminal appears on top of the stack, the npda confirms it matches the next symbol in the string and pops the terminal off the stack.

Algorithm to Convert Context-Free Grammar to NPDA for *LL* Parsing

Let $G = (V, T, S, P)$ be a context-free grammar. Construct an equivalent npda $M = (Q, \Sigma, \Gamma, \delta, q_0, z, F)$ that follows the *LL* parsing process.

M has three states, $Q = \{q_0, q_1, q_2\}$ with initial state q_0 and $F = \{q_2\}$. $\Sigma = T$. $\Gamma = V \cup T \cup \{z\}$, where z is the stack start symbol. We assume z is the only symbol on the stack when we start parsing. Next we define δ . The initial state q_0 's purpose is to push the start symbol S on the stack. S is put on the stack, and we transition to state q_1 with

$$\delta(q_0, \lambda, z) = \{(q_1, Sz)\}.$$

The middle state q_1 has two purposes. First for each production, q_1 handles the replacement of the left-hand side with its right-hand side. For every rule $A \rightarrow x_1x_2 \dots x_n \in P$, with $A \in V$ and x_i is a symbol either in V or T , add the following:

$$(q_1, x_1x_2 \dots x_n) \in \delta(q_1, \lambda, A).$$

Second, q_1 matches the symbols in the input string in the order they appear on top of the stack. For every $a \in T$ add the following:

$$(q_1, \lambda) \in \delta(q_1, a, a).$$

If all the symbols in the input string have been matched and the stack only has the stack start symbol on it, then the string can be accepted by moving to the final state q_2 . Add the following:

$$\delta(q_1, \lambda, z) = \{(q_2, z)\}.$$

EXAMPLE 16.1

Given a context-free grammar $G = (V, T, S, P)$, where $V = \{S, A\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow aSb \mid A, \\ A &\rightarrow bAa \mid c, \end{aligned}$$

create the equivalent npda that models the *LL* parsing process.

We create the three states $Q = \{q_0, q_1, q_2\}$ with initial state q_0 and $F = \{q_2\}$.

We push the start symbol S on top of the stack by adding the transition rule:

$$\delta(q_0, \lambda, z) = \{(q_1, Sz)\}.$$

We add one transition rule for each of the four productions resulting in

$$\begin{aligned} \delta(q_1, \lambda, S) &= \{(q_1, aSb), (q_1, A)\}, \\ \delta(q_1, \lambda, A) &= \{(q_1, bAa), (q_1, c)\}. \end{aligned}$$

We add transition rules to shift the input symbols onto the stack:

$$\begin{aligned} \delta(q_1, a, a) &= \{(q_1, \lambda)\}, \\ \delta(q_1, b, b) &= \{(q_1, \lambda)\}, \\ \delta(q_1, c, c) &= \{(q_1, \lambda)\}. \end{aligned}$$

Figure 16.2 shows a trace of parsing the string $abcb$ with the constructed npda, showing the current stack, state and lookahead at each step. Step 1 shows the starting configuration in state q_0 with the lookahead as the first symbol in the string, a , and the stack empty with z on it. Pop z off the stack, push Sz on the stack and move into state q_1 shown in step 2. Use the top of the stack, S , and the lookahead a to determine that we need to pop S off the stack and push down the righthand side aSb , shown in step 3. We choose aSb because we need an a to match the lookahead. With a on top of the stack and a the lookahead, pop a off the stack and set the lookahead to the next symbol in the string, b , shown in step 4. With S on top of the stack and b the lookahead, pop S off the stack and replace it with A shown in step 5. We do this since A can generate a string that starts with b . Pop A off the stack and replace it with bAa , since b is the lookahead, shown in step 6. With b on top of the stack and b the lookahead, pop b off the stack and replace the lookahead with the next symbol in the string, c , shown in step 7. With c the lookahead, pop A off the stack and replace it with c , shown in step 8. Steps 9–11 pop c , a , and b off the stack, match them with the lookaheads c , a , and b , and set the lookahead to $\$$. At this point the stack only has the stack start symbol z . Move into the final state q_2 in step 12 and accept $abcb$.

What are the differences between the construction of a context-free grammar to an npda here and in Section 7.2? In Section 7.2 the context-free grammar must be in Greibach Normal form. That means every production is in the form $A \rightarrow ax$ where $A \in V$, $a \in T$ and $x \in V^*$. There is only one terminal in each right-hand side, and the terminal is the first symbol. The construction in Section 7.2 replaces the right-hand side by the left-hand side, but instead of putting the one terminal on the stack and then immediately popping it, the terminal is read as input. Thus the construction in Section 7.2 does not need the transitions $\delta(q_1, a, a) = \{q_1, \lambda\}$ for all a .

EXERCISES

1. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned} S &\rightarrow ABd, \\ A &\rightarrow aA \mid c, \\ B &\rightarrow bbB \mid \lambda. \end{aligned}$$

- (a) Construct the equivalent npda that models the *LL* parsing process.
 - (b) Use the npda constructed and one lookahead in the string to parse the string *aacbbd*. Show the stack contents, the current state in the npda, and the current lookahead in each step.
 - (c) The npda is nondeterministic. For the string *aacbbd*, if no lookahead is used, when is the first choice of transitions to apply in the npda, and what are those transitions?
2. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, C\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned} S &\rightarrow aSb \mid Ccd, \\ C &\rightarrow \lambda. \end{aligned}$$

- (a) Construct the equivalent npda that models the *LL* parsing process.
 - (b) Use the npda constructed and one lookahead in the string to parse the string *cd*. Show the stack contents, the current state in the npda, and the current lookahead in each step.
 - (c) Use the npda constructed and one lookahead in the string to parse the string *acdb*. Show the stack contents, the current state in the npda, and the current lookahead in each step.
3. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow aSb \mid cA, \\ A &\rightarrow cAb \mid Bb, \\ B &\rightarrow aBc \mid \lambda. \end{aligned}$$

- (a) Construct the equivalent npda that models the *LL* parsing process.
 - (b) Use the npda constructed and one lookahead in the string to parse the string *ccbb*. Show the stack contents, the current state in the npda, and the current lookahead in each step.
 - (c) Use the npda constructed and one lookahead in the string to parse the string *acccacbbbb*. Show the stack contents, the current state in the npda, and the current lookahead in each step.
4. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned} S &\rightarrow ABc \mid dAcS, \\ A &\rightarrow aA \mid \lambda, \\ B &\rightarrow bBd \mid \lambda. \end{aligned}$$

- (a) Construct the equivalent npda that models the LL parsing process.
- (b) Use the npda constructed and one lookahead in the string to parse the string $dcac$. Show the stack contents, the current state in the npda, and the current lookahead in each step.
- (c) Use the npda constructed and one lookahead in the string to parse the string $dacaabdc$. Show the stack contents, the current state in the npda, and the current lookahead in each step.

16.2 $LL(1)$ PARSE TABLE

To simplify the parsing process, information from the npda combined with lookaheads can be stored in a table called the $LL(k)$ Parse Table. For now we focus on $k = 1$, and show how to build an $LL(1)$ Parse Table.

$LL(1)$ Parse Table

For a context-free grammar $G = (V, T, S, P)$, the rows of the table are labeled with $A \in V$ and the columns of the table are labeled with $a \in T \cup \{\$, \}$, the symbols that could be a lookahead.

To construct the $LL(1)$ Parse Table. Let $A \in V$, and $w \in (V \cup T)^*$.

1. For each production $A \rightarrow w$.
 - (a) For each a in $\text{FIRST}(w)$, add w to $LL[A, a]$.
 - (b) If $\lambda \in \text{FIRST}(w)$, add w to $LL[A, b]$ for each b in $\text{FOLLOW}(A)$.
2. Each undefined entry is error.

EXAMPLE 16.3

Build the $LL(1)$ Parse Table for the grammar in Example 16.1, and also shown below.

$G = (V, T, S, P)$, where $V = \{S, A\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow aSb \mid A, \\ A &\rightarrow bAa \mid c. \end{aligned}$$

First calculate the FIRST and FOLLOW sets for all variables.

	FIRST	FOLLOW
S	a, b, c	$b, \$$
A	b, c	$a, b, \$$

Build the $LL(1)$ Parse Table. For each production $A \rightarrow w$, determine where w goes in the table.

For the three rules that start with a terminal, the right-hand side is placed in the table in the column for that terminal and in the row for its left-hand side. For example, for $S \rightarrow aSb$, the entry $LL[S, a]$ is set to aSb .

For $S \rightarrow A$, $LL[S, x]$ is set to A for all x in $\text{FIRST}(A)$. Thus, $LL[S, b] = A$ and $LL[S, c] = A$.

Here is the completed $LL(1)$ Parse Table.

	a	b	c	$\$$
S	aSb	A	A	
A		bAa	c	

EXERCISES

1. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned} S &\rightarrow ABd, \\ A &\rightarrow aA \mid c, \\ B &\rightarrow bbB \mid \lambda. \end{aligned}$$

- (a) Calculate the FIRST and FOLLOW sets for all variables in the grammar.
 - (b) Build the $LL(1)$ Parse Table. For each production $A \rightarrow w$ determine where w goes in the table.
2. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, C\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned} S &\rightarrow aSb \mid Ccd, \\ C &\rightarrow \lambda. \end{aligned}$$

- (a) Calculate the FIRST and FOLLOW sets for all variables in the grammar.
 - (b) Build the $LL(1)$ Parse Table. For each production $A \rightarrow w$ determine where w goes in the table.
3. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow aSb \mid cA, \\ A &\rightarrow cAb \mid Bb, \\ B &\rightarrow aBc \mid \lambda. \end{aligned}$$

- (a) Calculate the FIRST and FOLLOW sets for all variables in the grammar.
 - (b) Build the $LL(1)$ Parse Table. For each production $A \rightarrow w$ determine where w goes in the table.
4. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned} S &\rightarrow ABc \mid dAcS, \\ A &\rightarrow aA \mid \lambda, \\ B &\rightarrow bBd \mid \lambda. \end{aligned}$$

- (a) Calculate the FIRST and FOLLOW sets for all variables in the grammar.
- (b) Build the $LL(1)$ Parse Table. For each production $A \rightarrow w$ determine where w goes in the table.

16.3 $LL(1)$ PARSING ALGORITHM

We present the following algorithm for $LL(1)$ parsing that uses the $LL(1)$ Parse Table and a stack to parse strings instead of using an npda. For this stack we do not use a stack start symbol, but instead assume this stack can determine whether it is empty or not. We first define several functions for the algorithm.

1. $\text{push}(w)$ pushes w on the stack. If $w = x_1x_2 \dots x_n$, where x_i is a symbol either from V or T , then x_n is pushed on the stack first, and x_1 is pushed on the stack last.
2. $\text{top}()$ returns a copy of the top symbol on the stack.
3. $\text{pop}()$ removes and returns the top symbol on the stack.
4. $\text{get}()$ returns the next lookahead symbol in the input string. If there is no such symbol, it returns the end of string marker \$.

LL(1) Parsing Algorithm

```

push(S)
lookahead = get()
while the stack is not empty do:
    symbol = top()
    if symbol is a terminal then:
        if symbol == lookahead then:
            pop()
            lookahead = get()
        else:
            error
    else if symbol is a variable then:
        if LL[symbol, lookahead] is not error then:
            pop()
            push(LL[symbol, lookahead])
        else:
            error
if lookahead ≠ $, then:
    error
else:
    accept

```

If accept is reached, then the string is in the language of the grammar. If any error is reached instead, then the string is not in the language of the grammar.

EXAMPLE 16.4

For the grammar in Example 16.3, apply the LL(1) Parsing Algorithm to the string *abcab*, and show the stack and lookahead in each step.

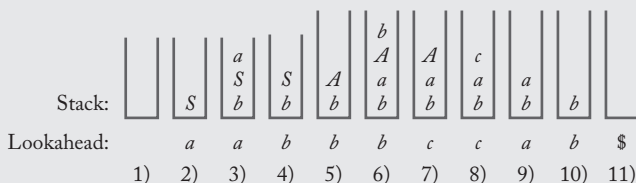


FIGURE 16.3

Figure 16.3 shows the stack and lookahead for the steps in tracing *abcab*. Start in step 1 before starting the algorithm with an empty stack. With this algorithm, we do not use a start stack symbol as we did with an npda, so an empty stack has no symbols. Push *S* on the

stack and set the lookahead to the first symbol in the input string, a , shown in step 2. The top of the stack is S . $LL[S, a]$ is aSb , so pop S off the stack and push aSb on the stack, shown in step 3. The top of the stack is a . Pop a off the stack and set lookahead to the next symbol in the input string, b , as shown in step 4. The top of the stack is S . $LL[S, b]$ is A . Pop S off the stack, and push A on the stack as shown in step 5. The top of the stack is A . $LL[A, b]$ is bAa . Pop A off the stack push bAa on, as shown in step 6. The top of the stack is b . Pop b off the stack and set lookahead to c as shown in step 7. The top of the stack is A . $LL[A, c]$ is c . Pop A off the stack and push c on the stack as shown in step 8. This continues with c , a and b popped off the stack and the lookahead set to $\$$ as shown in step 11. This results in an empty stack and accepting the input string $abcab$.

EXERCISES

- Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned} S &\rightarrow AB \mid cS, \\ A &\rightarrow aA \mid \lambda, \\ B &\rightarrow bBAd \mid \lambda. \end{aligned}$$

Here is the $LL(1)$ Parse Table for this grammar.

	a	b	c	d	$\$$
S	AB	AB	cS		AB
A	aA	λ		λ	λ
B	λ	$bBAd$		λ	λ

- Calculate the FIRST and FOLLOW sets for all variables in the grammar.
 - Apply the $LL(1)$ Parsing Algorithm to the string $cbad$, and show the stack and lookahead in each step.
 - Apply the $LL(1)$ Parsing Algorithm to the string $abbadd$, and show the stack and lookahead in each step.
- Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned} S &\rightarrow BdA \mid aSB, \\ A &\rightarrow a \mid \lambda, \\ B &\rightarrow bBAd \mid c. \end{aligned}$$

Here is the $LL(1)$ Parse Table for this grammar.

	a	b	c	d	$\$$
S	aSB	BdA	BdA		
A	a	λ	λ	λ	λ
B		$bBAd$	c		

- (a) Calculate the FIRST and FOLLOW sets for all variables in the grammar.
 - (b) Apply the $LL(1)$ Parsing Algorithm to the string $acdc$, and show the stack and lookahead in each step.
 - (c) Apply the $LL(1)$ Parsing Algorithm to the string $acdbcad$, and show the stack and lookahead in each step.
3. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned}
 S &\rightarrow BdA, \\
 A &\rightarrow cAB \mid \lambda, \\
 B &\rightarrow bB \mid aA.
 \end{aligned}$$

Here is the $LL(1)$ Parse Table for this grammar.

	a	b	c	d	$\$$
S	BdA	BdA			
A	λ	λ	cAB	λ	λ
B	aA	bB			

- (a) Calculate the FIRST and FOLLOW sets for all variables in the grammar.
 - (b) Apply the $LL(1)$ Parsing Algorithm to the string bad , and show the stack and lookahead in each step.
 - (c) Apply the $LL(1)$ Parsing Algorithm to the string $adcba$, and show the stack and lookahead in each step.
4. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned}
 S &\rightarrow aA \mid bS, \\
 A &\rightarrow aS \mid cB, \\
 B &\rightarrow bBd \mid \lambda.
 \end{aligned}$$

Here is the $LL(1)$ Parse Table for this grammar.

	a	b	c	d	$\$$
S	aA	bS			
A	aS		cB		
B		bBd		λ	λ

- Calculate the FIRST and FOLLOW sets for all variables in the grammar.
- Apply the $LL(1)$ Parsing Algorithm to the string $aaac$, and show the stack and lookahead in each step.
- Apply the $LL(1)$ Parsing Algorithm to the string $bacbd$, and show the stack and lookahead in each step.

16.4 $LL(k)$ PARSING

As we saw earlier in Section 7.4, not all context-free grammars are $LL(1)$, and some context-free grammars are not $LL(k)$ for any k . We can build the $LL(1)$ Parse Table for any context-free grammar. However, if there is a conflict in the table, meaning at least one entry in the table has more than one right-hand side, then the grammar is not $LL(1)$.

EXAMPLE 16.5

Build the $LL(1)$ Parse Table for the following grammar.

$G = (V, T, S, P)$, where $V = \{S\}$, $T = \{a, b\}$, and P consists of

$$S \rightarrow aSb \mid ab.$$

First calculate the FIRST and FOLLOW sets.

	FIRST	FOLLOW
S	a	$b, \$$

Build the $LL(1)$ Parse Table.

	a	b	$\$$
S	$aSb \quad ab$		

In this case there are right-hand sides from two rules that are placed in $LL[S, a]$. This means that we don't know which rule to pick, $S \rightarrow aSb$ or $S \rightarrow ab$ when the lookahead is a . This is called a conflict. The grammar is not an $LL(1)$ grammar.

We saw back in Section 7.4 that the grammar in Example 16.5 is an $LL(2)$ grammar. The grammar needs two lookaheads to determine which S rule to apply.

EXAMPLE 16.6

Build an $LL(2)$ Parse Table for the grammar in Example 16.5.

In the $LL(2)$ Parse Table, there are two lookaheads for each column heading. Note that the last column is only $\$$ since no symbol follows $\$$. Here is the table.

	aa	ab	$a\$$	ba	bb	$b\$$	$\$$
S	aSb	ab					

There are no conflicts when two lookaheads are used. This is an $LL(2)$ grammar.

EXAMPLE 16.7

The following grammar is $LL(k)$ for what value of k ?

$G = (V, T, S, P)$, where $V = \{S, B, C\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow aB \mid aaC, \\ B &\rightarrow aB \mid bc, \\ C &\rightarrow abC \mid a. \end{aligned}$$

For each variable, we determine the maximum number of lookaheads needed to determine which rule to apply. Then k is the maximum number of lookaheads needed over all variables.

For C there are two rules. Both rules start with the same terminal a . One lookahead of a cannot determine which rule to apply. Since the second rule's right-hand side is of length one, two symbols can determine between ab and a , $k = 2$.

For B there are two rules. Both rules start with a different terminal, $k = 1$.

For S there are two rules. Both rules start with the same terminal a , so k is at least 2. Here we need to look further into what the variables in the right-hand sides can derive. We need to calculate two strings, one derived from each rule, that have the most letters in common.

$S \rightarrow aB$ can derive aaB or aa^nbc with $n \geq 0$ or abc .

$S \rightarrow aaC$ can derive $aaabC$ or $aa(ab)^na$ with $n \geq 0$, or aaa .

The two strings with the longest same prefix are $aaabc$ from $S \rightarrow aB$ and $aaaba$ from $S \rightarrow aaC$ that both have prefix $aaab$. Five lookaheads are needed to distinguish the first character that is different, and thus which S rule to apply. For $S, k = 5$.

The grammar is $LL(5)$, since 5 is the largest value of k over all the variables.

EXAMPLE 16.8

The following grammar is $LL(k)$ for what value of k ?

$G = (V, T, S, P)$, where $V = \{S, A\}$, $T = \{a, b, c\}$, and P consists of

$$\begin{aligned} S &\rightarrow Ab \mid aAc, \\ A &\rightarrow aA \mid aAa \mid \lambda. \end{aligned}$$

For A there are three rules. The first two rules both can generate an infinite number of a 's. $A \rightarrow aA$ derives $a^n A$ with $n > 0$, and $A \rightarrow aAa$ derives $a^n Aa^n$ with $n > 0$. Since both rules can generate a large number of a 's, there is no k that can distinguish which rule to use.

The grammar is not $LL(k)$ for any k .

EXERCISES

1. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned} S &\rightarrow ASdB \mid cB, \\ A &\rightarrow aA \mid \lambda, \\ B &\rightarrow bBb \mid \lambda. \end{aligned}$$

- (a) Calculate the FIRST and FOLLOW sets for all variables in the grammar.
 - (b) Calculate the $LL(1)$ Parse Table for this grammar.
 - (c) Explain why this grammar is not $LL(1)$.
2. Consider the context-free grammar $G = (V, T, S, P)$, where $V = \{S, A, B, C\}$, $T = \{a, b, c, d\}$, and P consists of

$$\begin{aligned} S &\rightarrow AbCS \mid BA, \\ A &\rightarrow aA \mid \lambda, \\ B &\rightarrow bcB \mid \lambda, \\ C &\rightarrow d. \end{aligned}$$